Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mcode functions

Synopsis

The functions prefixed with Mcode make up the Code module. The Code module allows you to read, write, and grade symbols (codes) from most popular one-dimensional (1D) and two-dimensional (2D) symbologies (code types). The supported 1D code types include 4-state, BC412, Codeabar, Code 39, Code 93, Code 128, EAN 8, EAN 13, EAN 14, GS1-128, GS1 Databar, Industrial 2 of 5 (ITF-14), Interleaved 2 of 5 (ITF-14), Pharmacode, Planet, Postnet, UPC-A, and UPC-E. The supported 2D code types are Aztec, Data Matrix, Maxicode, QR code, Micro QR code, PDF417, MicroPDF417, and Truncated PDF417. Composite codes, which are combinations of 1D and 2D code types, are also supported. The module easily handles rotated, scaled, and degraded code occurrences, even in complex scenes. To help setup for reading or grading, the Code module can detect the type of most 1D code occurrences in an image. In addition, the module can optimize the read or grading operation settings by training them using a set of sample images.

Functions

McodeAlloc
McodeAllocResult
McodeControl
McodeDetect
McodeDraw
McodeFree
McodeGetResult
McodeGrade
McodeInquire
McodeModel
McodeRead
McodeRestore
McodeSave
McodeStream
McodeTrain
McodeWrite

McodeAlloc

Synopsis

Allocate a code context.

Syntax

```
AIL_ID McodeAlloc(
    AIL_ID SysId,            //in
    AIL_INT64 ContextType,   //in
    AIL_INT64 ControlFlag,   //in
    AIL_ID *ContextCodeIdPtr //out
)
```

Description

This function allocates a code context. A code context is an Aurora Imaging Library object that stores one or more code models. The code model is used by the McodeRead(), McodeGrade(), McodeTrain(), and McodeWrite() operations.

To add code models to the context, use McodeModel(). A code context can contain multiple code models of 1D code types (excluding GS1 databar, Planet, Postnet, and 4-state); for other code types, a code context can contain at most one code model. To identify the types of code occurrences in an image, you can perform a McodeDetect() operation; note that a code context is not required for a McodeDetect() operation.

To adjust code model settings, use McodeControl().

When allocating a code context, specify whether the context should be initialized for a robust or a typical McodeRead(), McodeGrade(), or McodeTrain() operation. The initial configuration of the code context establishes the default values for some code context and code model settings. To change the initial configuration at a later date, use McodeControl() with M_INITIALIZATION_MODE. The explicit restrictions are indicated within the description of each code model setting; see McodeControl() for details.

To read and grade code occurrences in an image, use McodeRead() and McodeGrade(), respectively. The module only supports searching for multiple occurrences of 1D codes (excluding 4-State, GS1 Databar, Planet, and Postnet code types) and 2D Data Matrix codes. For other types of code occurrences, the module only supports searching for a single occurrence. When reading or grading multiple code occurrences, all results can be retrieved using McodeGetResult() with M_ALL. Each occurrence has a set of results.

To establish the best possible control type settings given your images, use McodeTrain().

When the code context is no longer required, release it using McodeFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the code context's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextType

Specifies the context type. Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

ControlFlag

Specifies the initial configuration for the code context. The restrictions are indicated within each control, see McodeControl() for details.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the initial configuration of the code context

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_TYPICAL_RECOGNITION.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMPROVED_RECOGNITION	Specifies a code context that might provide a more robust McodeRead(), McodeGrade(), or McodeTrain() operation. This initialization mode is recommended if your target images lack similarity (for example, some but not all include complex or noisy backgrounds), contain distorted code occurrences, or contain code occurrences that are at an angle or even flipped. It is also recommended when searching for 2D matrix codes that are composed of dots. Note that when selecting this mode, the default of the affected code context and code model settings might change between Aurora Imaging Library releases to improve robustness, given new features or standard recommendations. This initialization mode is not recommended with M_PHARMACODE code types.	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPICAL_RECOGNITION	Specifies a code context that might provide a quicker McodeRead() or McodeGrade() operation, but might potentially produce less robust results.	c		e	f	g	a		i		j	k	l	m	n	b

ContextCodeIdPtr

Specifies the address of the variable in which to write the code context identifier or specifies the data type that the function should use to return the code context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated code context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated code context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_CODE_ID* is returned instead of a standard Aurora Imaging Library identifier. This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the code context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated code context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the code context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_CODE_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcode11.lib.

DLL	Requires ail11.dll; ailcode11.dll.

McodeAllocResult

Synopsis

Allocate a code result buffer.

Syntax

```
AIL_ID McodeAllocResult(
    AIL_ID SysId,           //in
    AIL_INT64 ControlFlag,  //in
    AIL_ID *ResultCodeIdPtr //out
)
```

Description

This function allocates a code result buffer on the specified system to store results obtained from an McodeTrain(), McodeDetect() McodeRead(), McodeGrade(), or McodeWrite() operation.

When the code result buffer is no longer required, release it using McodeFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the code result buffer's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the result buffer. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies the type of result buffer to allocate. This parameter must be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the result buffer type																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_CODE_GRADE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_DETECT_RESULT	Specifies to allocate a code detect result buffer to be used with McodeDetect().	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_RESULT	Specifies to allocate a code grade result buffer to be used with McodeGrade().	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_READ_RESULT	Specifies to allocate a code read result buffer to be used with McodeRead().	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_TRAIN_RESULT	Specifies to allocate a code train result buffer to be used with McodeTrain().	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_WRITE_RESULT	Specifies to allocate a code write result buffer to be used with McodeWrite().	c		e	f	g	a		i		j	k	l	m	n	b

ResultCodeIdPtr

Specifies the address of the variable in which to write the code result buffer identifier or specifies the data type that the function should use to return the code result buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l –

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated code result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated code result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_CODE_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the code result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the code detect result identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated code detect result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the code grade result identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated code grade result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the code read result identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated code read result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the code train result identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated code train result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the code write result identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated code write result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the code result buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_CODE_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcode11.lib.
DLL	Requires ail11.dll; ailcode11.dll.

McodeControl

Synopsis

Control a code context, code model, or code result buffer setting.

Syntax

```
void McodeControl(
    AIL_ID CodeId,          //out
    AIL_INT64 ControlType,  //in
    AIL_DOUBLE ControlValue //in
)
```

Description

This function changes the setting of a specified code context, code model, or code result buffer.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **McodeControl()** internally calls the *AIL_DOUBLE* version of this function (McodeControlDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (McodeControlInt64()).

Parameters

CodeId

Specifies the identifier of the code context, code model, or code result buffer.

ControlType

Specifies the setting to change.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the setting's new value.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- Code context settings for an McodeRead(), McodeGrade(), or McodeTrain() operation
- Code context settings for an McodeGrade() operation
- Code context settings for an McodeTrain() operation
- Code model settings for an McodeRead(), McodeGrade(), or McodeTrain() operation
- Code model settings for an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation
- Code model settings for an McodeGrade() operation
- Code model settings for an McodeWrite() operation
- Result buffer settings dealing with an McodeRead(), McodeGrade(), McodeDetect(), McodeTrain(), or McodeWrite() operation
- Result buffer settings dealing with an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation
- Result buffer settings dealing with McodeDetect() operation

The following ControlType and corresponding ControlValue parameter settings can be specified for a code context, to control an McodeRead(), McodeGrade(), or McodeTrain() operation. In this case, a code context must be passed to the CodeId parameter.

> Note that besides code type restrictions listed explicitly in the values below, McodeGrade() does not support M_4_STATE, M_PHARMACODE, M_POSTNET, and M_PLANET code types.

Although all these control types affect the results of McodeTrain(), see the description of the control types to determine if they can be activated for training; you can also use filters to limit the control types in the table to those that can be trained.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Code context settings for an McodeRead(), McodeGrade(), or McodeTrain() operation																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

ControlValue															
M_INITIALIZATION_MODE	Changes the initialization mode of the context; this changes the default of several code context and code model settings. The default value of the setting in each mode is indicated within the description of the control type. When you change the initialization mode, control types that have already been set to some value (other than default) are not changed. Instead, the initialization mode only changes those control values that are set to their default value (M_DEFAULT). INQ														
M_DEFAULT	Same as M_TYPICAL_RECOGNITION.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMPROVED_RECOGNITION	Specifies a code context that might provide a more robust McodeRead(), McodeGrade(), or McodeTrain() operation. This initialization mode is recommended if your target images lack similarity (for example, some but not all include complex or noisy backgrounds), contain distorted code occurrences, or contain code occurrences that are at an angle or even flipped. It is also recommended when searching for 2D matrix codes that are composed of dots. > Note that when selecting this mode, the default of the affected code context and code model settings might change between Aurora Imaging Library releases to improve robustness, given new features or standard recommendations. This initialization mode is not recommended with M_PHARMACODE code types.	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPICAL_RECOGNITION	Specifies a code context that might provide a quicker McodeRead() or McodeGrade() operation, but might potentially produce less robust results.	c		e	f	g	a		i		j	k	l	m	n	b
M_MINIMUM_CONTRAST +	Sets the minimum possible contrast between the foreground and background in the target image for 1D code types (excluding M_PLANET and M_POSTNET), the M_DOTCODE code type, and the M_MICROPDF417 code type when using the M_ADAPTIVE threshold mode. For 1D code types (except for M_GS1_DATABAR, M_PLANET, and M_POSTNET) and 2D cross-row code types, regardless of the threshold mode, M_MINIMUM_CONTRAST is also used to help the localization. When using the M_ADAPTIVE threshold mode for 1D code types, the M_DOTCODE code type, and the M_MICROPDF417 code type, increasing the minimum contrast will typically improve McodeRead() operations, particularly in the presence of noise and non-uniform lighting. However, if the minimum contrast is higher than the contrast of a code, the code will not be read correctly. For 2D code types (excluding M_DOTCODE and M_MICROPDF417), the minimum contrast is determined automatically, so the adaptive threshold mode does not use the M_MINIMUM_CONTRAST setting. For 1D code types (excluding M_PLANET and M_POSTNET) and 2D cross-row code types, regardless of threshold mode, you should set this control type if the contrast is less than 50 and there are difficulties decoding the code. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. Note that training this control type (M_TRAIN) is only supported for the M_DOTCODE code type. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50. > Note that setting M_MINIMUM_CONTRAST to M_DEFAULT is not equivalent to setting M_MINIMUM_CONTRAST to 50. When M_DEFAULT is specified, some internal steps might use a minimum contrast other than 50.	c		e	f	g	a		i		j	k	l	m	n	b

			c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 255	Specifies the minimum contrast. Only integer values are accepted. For the M_DOTCODE code type, this setting should be set to a value greater than or equal to 10. Only integer values are accepted.															
M_SCANLINE_HEIGHT		Sets the height (or thickness) of the scanlines. A scanline is a line along which the code occurrence is read. When locating a code occurrence, a scanline is usually first taken across the middle of the image or the image's region of interest (ROI), if one is associated. If a code occurrence is not found, a scanline is taken either higher or lower in the image or its ROI. Usually, setting this control type to a higher value increases the robustness of McodeRead(), McodeGrade(), and McodeTrain() operations. This is especially true in noisy images. However, defects in the code can cause the need for a lower value. Using the default value is recommended. Note that during an McodeGrade() operation, after locating the code, when taking scanlines within the inspection band, Aurora Imaging Library uses a scanline height of a single pixel regardless of the value of M_SCANLINE_HEIGHT, as per the ISO standard. This control type is only available for 1D code types (excluding M_4_STATE, M_PLANET, and M_POSTNET), 2D cross-row code types, and M_COMPOSITECODE code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies a default scanline height that is based on the code type. The table below shows the default value for some code types. Code type: Default value (in pixel units) Linear bar codes (excluding M_4_STATE, M_PLANET, and M_POSTNET): 5 MicroPDF 417: 1 PDF 417/Truncated PDF417: 3 Composite codes: 1	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the scanline height, relative to the input coordinate system specified using M_SCANLINE_INPUT_UNITS. When entering a value in pixel units, this value must be between 1 and 256, inclusive. The value entered is rounded to the nearest integer. The recommended range of values for linear bar codes is: 1 < Value <= (BarcodeHeight / 2). For M_MICROPDF417 and M_COMPOSITECODE, the recommended range of values is: 1 <= Value <= (RowHeight / 2). When entering a value in world units, the number that results from the internal transformation of world units to pixel units must still respect the value's pixel unit constraint. To find the range of allowed values in world units, according to your associated camera calibration, use McalTransformResult().	c		e	f	g	a		i		j	k	l	m	n	b

M_SCANLINE_INPUT_UNITS	Sets the units with which to interpret the M_SCANLINE_HEIGHT and M_SCANLINE_STEP control types. This essentially sets the input coordinate system to use. This control type is only available for 1D code types (excluding M_4_STATE, M_PLANET, and M_POSTNET), 2D cross-row code types, and M_COMPOSITECODE code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the values in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the values in world units, with respect to the relative coordinate system. If world units are specified, calling McodeRead(), McodeGrade(), or McodeTrain() generates an error if the operation is not performed on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCANLINE_STEP	Sets the scanline step. The scanline step is the distance, or gap, between scanlines taken in the image or the image's region of interest (ROI), if one is associated. In general, the higher the value for this control type, the faster the code is read, but the greater the chance of missing a readable line of code. Using the default value is recommended; the default value is dependent upon the search speed. Note that during an McodeGrade() operation, after locating the code, when taking scanlines within the inspection band, Aurora Imaging Library uses a scanline step distance based on the height of the inspection band and the required number of scanlines, regardless of the value of M_SCANLINE_STEP, as per the ISO standard; the ISO standard requires ten evenly spaced scanlines within the inspection band. This control type is only available for 1D code types (excluding M_4_STATE, M_PLANET, and M_POSTNET), 2D cross-row code types, and M_COMPOSITECODE code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies a default scanline step that is based on the value of M_SPEED. When dealing with 1D code types (excluding M_4_STATE, M_PLANET, and M_POSTNET) and M_COMPOSITECODE code types, the following values are associated with M_DEFAULT. M_SPEED value: Default value M_LOW: 5 M_VERY_LOW: 5 M_MEDIUM: 10 M_HIGH: 15 M_VERY_HIGH: 25 When dealing with 2D cross-row code types, the default value is 2, regardless of the value of M_SPEED.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the scanline step, relative to the input coordinate system specified using M_SCANLINE_INPUT_UNITS. When entering a value in pixel units, this value must be greater than or equal to 1. The value entered is rounded to the nearest integer. When entering a value in world units, the number that results from the internal transformation of world units to pixel units must still respect the value's pixel unit constraint. To find the range of allowed values in world units, according to your associated camera calibration, use McalTransformResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_MODE	Sets whether to enable the search angular range algorithm for the code context. When enabled, codes are sought at the angular range defined by M_SEARCH_ANGLE - M_SEARCH_ANGLE_DELTA_NEG to M_SEARCH_ANGLE + M_SEARCH_ANGLE_DELTA_POS. For 1D codes, the Code module will not use the search angular range algorithm if M_SEARCH_ANGLE_DELTA_NEG and M_SEARCH_ANGLE_DELTA_POS both have a value of less than 5°, regardless of M_SEARCH_ANGLE_MODE. For linear code types, if you expect that the codes sought are close to their nominal angle, you can try disabling the search angular range algorithm to see if the Code module can still find the required code occurrences. Disabling this algorithm might speed up the search depending on the code type and the target image. For the M_AZTEC, M_DATAMATRIX, M_DOTCODE, M_QRCODE, M_MICROQRCODE, M_MAXICODE, M_PDF417, and M_TRUNCATED_PDF417 code types, the angular range does not affect the speed of the operation. For M_PDF417 and M_TRUNCATED_PDF417, you can set M_DECODE_ALGORITHM to M_SCANLINE_AT_ANGLE if you are having issues reading your code at an angle. Note that although this improves robustness, this might affect the speed of the operation. You can try disabling M_SEARCH_ANGLE_MODE to see if the Code module can still find the required code occurrences; this might improve the speed. Note that M_SEARCH_ANGLE_MODE is automatically activated for training if you activate M_SEARCH_ANGLE for training. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the search angular range algorithm is not used.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the search angular range algorithm is used.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPEED +	Sets the search speed. The faster the search speed, the less robust the operation. For 1D code types, consider reducing the search speed when the pixel height of the bar code is small. For 2D code types, consider reducing the search speed when the cell size is small relative to the image size. This control type is available for all code types except M_DOTCODE. For 1D code types, M_COMPOSITECODE, and M_MICROPDF417, you can also affect the search speed by adjusting the value for M_SCANLINE_STEP. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value, which is dependent on the initialization mode. When using a code context with M_INITIALIZATION_MODE set to M_IMPROVED_RECOGNITION, the default value is M_VERY_LOW. When using a code context with M_INITIALIZATION_MODE set to M_TYPICAL_RECOGNITION, the default value is M_MEDIUM.	c		e	f	g	a		i		j	k	l	m	n	b
	M_HIGH	Specifies a high search speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOW	Specifies a low search speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEDIUM	Specifies a medium search speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_VERY_HIGH	Specifies a very high search speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_VERY_LOW	Specifies a very low search speed.	c		e	f	g	a		i		j	k	l	m	n	b
M_STOP_READ		Stops the current McodeRead() or McodeGrade() operation being performed on the code context passed to CodeId. You must call **McodeControl**() with M_STOP_READ from another thread, typically of higher priority. After making this call, the results returned to the result buffer of the McodeRead() or McodeGrade() operation are invalid and discarded.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_MODE +		Sets the threshold mode used to internally binarize the source image. If the lighting is non-uniform across the code, use M_ADAPTIVE mode or process the image before performing the McodeRead() or McodeGrade() operation so that the background is strictly darker or strictly lighter than the code. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value, which is dependent on the code type and the initialization mode. When using a code context with M_INITIALIZATION_MODE set to M_IMPROVED_RECOGNITION, the default value for M_MICROPDF417 and linear (1D) code types (except M_PHARMACODE) is M_GLOBAL_SEGMENTATION, and the default value for the remaining code types is M_ADAPTIVE. When using a code context with M_INITIALIZATION_MODE set to M_TYPICAL_RECOGNITION, the default value for M_DOTCODE is M_ADAPTIVE, and the default value for the remaining code types is M_GLOBAL_SEGMENTATION.	c		e	f	g	a		i		j	k	l	m	n	b

M_ADAPTIVE	Specifies to use a fast dynamic local threshold. This mode selects an individual threshold for each pixel based on the range of intensity values in its local neighborhood. For linear (1D) code types (excluding M_PLANET and M_POSTNET), the M_DOTCODE code type, and the M_MICROPDF417 code type, you must specify the minimum contrast (**McodeControl**() with M_MINIMUM_CONTRAST) between the foreground and background in the target image when using the M_ADAPTIVE mode. It is highly recommended that M_CELL_SIZE_MIN is set to a valid value that is not M_DEFAULT; if your code is not decoded properly, increase the value of M_CELL_SIZE_MIN. For 2D code types (excluding M_DOTCODE and M_MICROPDF417), the minimum contrast is determined automatically, so the adaptive threshold mode does not use the M_MINIMUM_CONTRAST setting. 2D and 1D code types (excluding M_PLANET and M_POSTNET) are the code types that support M_ADAPTIVE.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_SEGMENTATION	Specifies to use a global threshold value. Specify the value using M_THRESHOLD_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_WITH_LOCAL_RESEGMENTATION	Specifies that the source image will be globally thresholded and then the edges in the binarized image are resegmented according to the intensities of the surrounding bars and spaces in the original source image. These individual local thresholds, calculated per region, help enhance the edge positions in the code. Specify the global threshold value using M_THRESHOLD_VALUE. This binarization method is compatible with the methods described in the various ISO code grading standards. For more information, refer to the Technical code information subsection of the Basic concepts for the Aurora Imaging Library Code module section of Chapter 17: Codes. This control value is available for all 1D code types (except M_4_STATE, M_POSTNET, and M_PLANET) and the M_MICROPDF417 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_VALUE	Sets the threshold value used to internally binarize the source image, depending on the specified threshold mode. Note that if M_THRESHOLD_MODE is set to M_ADAPTIVE, this control type is ignored. Note that this control type is automatically activated for training if you activate M_THRESHOLD_MODE for training. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO_COMPUTE.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_COMPUTE	Specifies the threshold value automatically.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 255	Specifies the threshold value. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT	Specifies the maximum decoding time for an McodeRead() or McodeGrade() operation, in msec. Note that this control type can still affect McodeTrain(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2000 msec.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that there is no maximum decoding time.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum decoding time, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

M_TOTAL_NUMBER	Sets the maximum number of code occurrences to read or grade in one source image. Note that this number is limited by the maximum number of occurrences of each code model to read or grade (M_NUMBER). To avoid obtaining false positives when searching for a specific number of occurrences, set M_CODE_SEARCH_MODE to M_BEST. Note that M_BEST is not supported for 2D code types other than the M_DOTCODE code type. The following table lists the code types that support searching for multiple occurrences: Code type — Multiple occurrences Linear bar codes (excluding M_GS1_DATABAR, M_4_STATE, M_PLANET, and M_POSTNET) — Yes M_DATAMATRIX — Yes M_DOTCODE — Yes Only the code types in the table above support this control type. When dealing with all other code types, M_TOTAL_NUMBER must be set to M_DEFAULT. Note that for M_DATAMATRIX and M_DOTCODE code types, M_TOTAL_NUMBER should be set to the same value as M_NUMBER or to M_DEFAULT because a code context can only contain multiple code models of the above-mentioned 1D code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to read/grade all code occurrences in the source image, up to the maximum number of occurrences to read/grade of each code model (M_NUMBER).	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the maximum number of codes to read/grade in one source image. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings can be specified for a code context, to control an McodeGrade() operation. In this case, a code context must be passed to the CodeId parameter.

> Note that besides code type restrictions listed explicitly in the values below, McodeGrade() does not support M_4_STATE, M_PHARMACODE, M_POSTNET, and M_PLANET code types.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Code context settings for an McodeGrade() operation																	
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_ABSOLUTE_APERTURE_SIZE	Sets the absolute size (diameter) of the aperture. Use this control type when using an absolute aperture mode (using M_APERTURE_MODE set to M_ABSOLUTE). INQ															

	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the absolute aperture size, relative to the input coordinate system specified using M_ABSOLUTE_APERTURE_SIZE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_ABSOLUTE_APERTURE_SIZE_INPUT_UNITS		Sets the units with which to interpret the M_ABSOLUTE_APERTURE_SIZE control type. This essentially sets the input coordinate system to use. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL	Specifies to interpret the value in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD	Specifies to interpret the value in world units, with respect to the relative coordinate system. If world units are specified, calling McodeGrade() generates an error if the operation is not performed on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b
M_APERTURE_MODE		Sets the way in which the aperture size is determined. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_RELATIVE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ABSOLUTE	Specifies to use the absolute aperture size, set using M_ABSOLUTE_APERTURE_SIZE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable the aperture. Note that this is provided for backwards compatibility with older code. In this case, the McodeGrade() operation's returned results will more closely resemble the results of an McodeRead() operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RELATIVE	Specifies to use a relative aperture size, based on the cell size (using M_CELL_SIZE...) and the relative aperture factor (using M_RELATIVE_APERTURE_FACTOR).	c		e	f	g	a		i		j	k	l	m	n	b
M_DPM_CALIBRATION_RESULTS		Sets the expected reflectance and mean light values derived during the reflectance calibration phase. This control type is an alternative to separately obtaining the reflectance and mean light values from the result buffer used during the reflectance calibration phase, and setting M_REFLECTANCE_CALIBRATION and M_MEAN_LIGHT_CALIBRATION to these two values, respectively, in the code context associated with the target code. This control type is available for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_MICROQRCODE code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Result buffer identifier	Specifies the result buffer used during the reflectance calibration phase.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_REFLECTANCE_CHECK		Sets whether the grading must perform an additional reflectance check over an area containing the code occurrence and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This control type is only available for M_AZTEC, M_DATAMATRIX, M_DOTCODE, M_QRCODE, and M_MICROQRCODE code types. Note that this control type must be set before an McodeRead() operation if you intend to use the result for an McodeGrade() operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable the additional reflectance check.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable the additional reflectance check. > Note that you can only enable this control type when performing an ISO grading (that is, when M_GRADING_STANDARD is set to M_ISO_GRADING).	c		e	f	g	a		i		j	k	l	m	n	b
M_GRADE_QUIET_ZONE		Sets whether to include the quiet zone when performing an McodeGrade() operation. This control type is available for M_DATAMATRIX, M_DOTCODE, M_QRCODE, and M_MICROQRCODE code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to exclude the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to include the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRADING_STANDARD		Sets the grading standard to use when performing an McodeGrade() operation. To specify a particular grading standard edition, use M_GRADING_STANDARD_EDITION. The results of the McodeGrade() operation are dependent on the standard used; some control types have a different implementation or meaning depending on the standard used. These differences are outlined in the control types' description. Note that this control type must be set before an McodeRead() operation if you intend to use the result for an McodeGrade() operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ISO_GRADING.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ISO_DPM_GRADING	Specifies to use an ISO/IEC 29158 specification. Note that this is the replacement for the AIM DPM-1-2006 standard. This control value must be used at the beginning of the target grading phase, and should not be used during the reflectance calibration phase. This control value is available for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_MICROQRCODE code types.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ISO_GRADING	Specifies to use an ISO/IEC 15416 or ISO/IEC 15415 specification.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SEMI_T10_GRADING	Specifies to use a Semi T10 specification. This control value is available for the M_DATAMATRIX code type. This control type requires that M_APERTURE_MODE, M_MAXIMUM_CALIBRATED_REFLECTANCE, and M_MINIMUM_CALIBRATED_REFLECTANCE are set to M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_INSPECTION_BAND_RATIO		Sets the height of the inspection band as a percentage of the average bar height. The inspection band is the area of the code occurrence from which the scanlines to grade the code occurrence are taken, after it has been located. This control type is only available for 1D code types, 2D cross-row code types, and M_COMPOSITECODE code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default percentage. The default value is 80% of average bar height.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 < *Value* < 100.0	Specifies the percentage of the average bar height to set as the height of the inspection band.	c		e	f	g	a		i		j	k	l	m	n	b
M_LIGHTING_CONFIGURATION		Sets the tilted coaxial lighting and camera angle position (TCL) relative to the marked object. There is a tolerance of +/- 3 degrees between the angle of the light and the angle of the camera. See the ISO/IEC 29518:2020 specifications for further details. This control type is only available for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_MICROQRCODE code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_UNSPECIFIED.	c		e	f	g	a		i		j	k	l	m	n	b
	M_90_DEGREE	Specifies the TCL is 90 degrees relative to the marked object. If M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020, this will cause M_MINIMUM_REFLECTANCE and M_MINIMUM_REFLECTANCE_GRADE to return M_CODE_GRADE_NOT_AVAILABLE. If M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2011, this setting will have no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSPECIFIED	Specifies that the TCL is none of the other values.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAXIMUM_CALIBRATED_REFLECTANCE		Sets the maximum possible grayscale value in the target image. This control type is used in the scan reflectance profile analysis. If this control type does not accurately represent the actual maximum grayscale value from the target image, the resulting values from the scan reflectance profile are invalid. Note that this value must be higher than the value of M_MINIMUM_CALIBRATED_REFLECTANCE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 255.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 255	Specifies the maximum calibrated reflectance. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN_LIGHT_CALIBRATION		Sets the expected mean light (MLcal). This corresponds to the expected mean intensity of the centers of the white elements of the code occurrence. This control type is used during the target grading phase of the ISO/IEC 29158 specification. Note that the value of this control type can also be set using M_DPM_CALIBRATION_RESULTS. This control type is available for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_MICROQRCODE code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is equal to the setting of M_MAXIMUM_CALIBRATED_REFLECTANCE.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 255	Specifies the mean intensity. This value is typically derived during the reflectance calibration phase.	c		e	f	g	a		i		j	k	l	m	n	b

M_MINIMUM_CALIBRATED_REFLECTANCE		Sets the minimum possible grayscale value in the target image. This control type is used in the scan reflectance profile analysis. If this control type does not accurately represent the actual minimum grayscale value from the target image, the resulting values from the scan reflectance profile are invalid. Note that this value must be lower than the value of M_MAXIMUM_CALIBRATED_REFLECTANCE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 255	Specifies the minimum calibrated reflectance. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_SCANLINES		Sets the number of scanlines inside the inspection band to inspect during the McodeGrade() operation, after the code occurrence has been located. Set the height of the inspection band using M_INSPECTION_BAND_RATIO. This control type is only available for 1D code types, 2D cross-row code types, and M_COMPOSITECODE code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value. The default value is 10 scanlines.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies to use all possible scanlines inside the inspection band.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the number of scanlines to inspect. This value is equivalent to M_ALL if it exceeds the inspection band height. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_SIZE_IN_MM		Sets the scale between a pixel and its physical measurement in mm. To simplify the computation of this value, measure the length of the entire code and divide it by its pixel length; this yields a good approximation. Alternatively, calculate the scale using the following formula: (*size of the object*) / (*number of pixels in object*). For more information, see point 3 in the Relative aperture setting subsection of the Settings for a code grade operation section of Chapter 18: More on grading codes. Code specifications generally specify constraints in mm units. To compare values known in Aurora Imaging Library (for example, the cell size in pixels) to the values in the code specifications, this control type is used. Note that this control type can impact other control types, for example, the aperture size selection, and the M_SCAN_INTERCHARACTER_GAP_GRADE result (retrieved using McodeGetResult()). Note that this value is needed even when operating on calibrated images because Aurora Imaging Library does not know the relationship between mm and the world units of the calibrated context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_UNKNOWN.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNKNOWN	Specifies that the scale between a pixel and its physical measurement is not known, in mm per pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the scale between a pixel and its physical measurement, in mm per pixel units.	c		e	f	g	a		i		j	k	l	m	n	b

M_REFLECTANCE_CALIBRATION	Sets the expected reflectance value (Rcal). This control type is used during the target grading phase of the ISO/IEC 29158 specification. Note that the value of this control type can also be set using M_DPM_CALIBRATION_RESULTS. This control type is available for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_MICROQRCODE code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is equal to the setting of M_MAXIMUM_CALIBRATED_REFLECTANCE.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 255	Specifies the reflectance value. This value is typically derived during the reflectance calibration phase. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_APERTURE_FACTOR	Sets the aperture factor to use when M_APERTURE_MODE is set to M_RELATIVE. The aperture factor is multiplied by the M_CELL_SIZE_MIN to determine the aperture's diameter. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the aperture factor is chosen according to the ISO/IEC 15416 specification for all supported code types except 2D matrix code types; for the latter, the aperture factor is chosen according to the ISO/IEC 15415 or ISO/IEC 29158 specification (depends on M_GRADING_STANDARD). Note that if the nominal bar width is less than 0.1 mm, the aperture factor is 0.5. When dealing with EAN/UPC code types, the aperture factor is 0.15. When dealing with 2D matrix code types, the aperture factor is 0.8. Note that the recommended aperture size in the ISO/IEC 15415 specification has changed. The 2004 specification states that the aperture size should be 0.8 times the diameter. The 2011 specification recommends an aperture size between 50% and 80% of the smallest X-dimension encountered. M_AUTO complies with the 2004 version of the specification. Since the new specification now permits alternative aperture sizes, using an aperture factor of 0.5 with 2D matrix code types can often lead to better results.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 2	Specifies the aperture factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_SYSTEM_RESPONSE_CALIBRATION	Sets the System Response value derived during the reflectance calibration phase (SRcal) of the ISO/IEC 29158 specification. The System Response value is a user-defined aggregate of the settings of environmental factors, such as exposure time and/or gain factor, that create the conditions for an acceptable image of the code. For example, you could define the System Response value as: (*Exposure time* * *Gain factor*). During the reflectance calibration phase, adjust the exposure time and/or gain factor until you obtain an acceptable image of the code, and then multiply the two values together. At the beginning of the target grading phase, set this result as the System Response value of the reflectance calibration phase. Note that you should use the same formula to calculate the System Response value, in both the reflectance calibration phase and the target grading phase. This control type is only available for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_MICROQRCODE code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b

	Value > 0.0	Specifies the System Response value for the reference code.	c		e	f	g	a		i		j	k	l	m	n	b
M_SYSTEM_RESPONSE_TARGET		Sets the System Response value derived during the target grading phase (SRtarget) of the ISO/IEC 29158 specification. To set this control type, use the same aggregate of environmental factors that was used to calculate M_SYSTEM_RESPONSE_CALIBRATION. For example, if you set the System Response value of the reflectance calibration phase to the multiplication of the exposure time by the gain factor, set the System Response value of the target grading phase to the multiplication of the exposure time by the gain factor used to create the conditions for an acceptable image of the target code. During the target grading phase, you should not alter any environmental factors other than those used to calculate the System Response value. This control type is only available for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_MICROQRCODE code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the System Response value for the target code.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings can be specified when dealing with an McodeTrain() operation. In this case, a code context must be passed to the CodeId parameter.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Code context settings for an McodeTrain() operation																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_RESET_FROM_TRAINED_RESULTS		Resets the code context using the results in the code train result buffer. Existing code models are discarded and code models used for training are added to the context. If a control type was not trained, it is set to its value used during training; if a control type was trained, it is set to its trained value. The results in the code train result buffer can be applied to any code context. To reconfigure the code context that was used during training, you might find it useful to selectively change some control types to their recommended setting, established during training. To do so, you can use McodeGetResult() to retrieve their recommended setting, and then use **McodeControl**() to set them explicitly. Note that some control types are inter-dependent. Changing the setting of one control type to its trained value might not be optimal if the settings of other control types are not also changed. This is especially true for control types that are automatically activated for training if another control type is activated (for example, M_CELL_NUMBER_...).															
	CodeTrainResultBufId	Specifies the identifier of the code train result buffer to use to reset the code context. The result buffer must have been previously allocated using McodeAllocResult() with M_CODE_TRAIN_RESULT and must contain results from a successful McodeTrain(). Use McodeGetResult() with M_STATUS to ensure that the result buffer contains results from a successful operation (M_STATUS_TRAIN_OK).	c		e	f	g	a		i		j	k	l	m	n	b

M_SET_TRAINING_STATE_ALL	Resets whether all trainable control types will be trained when McodeTrain() is called. Using this control type is equivalent to using the M_TRAIN combination value with every trainable control type. After activating or deactivating all trainable control types using M_SET_TRAINING_STATE_ALL, you can then activate or deactivate the training of specific control types using M_TRAIN, if necessary. M_SET_TRAINING_STATE_ALL modifies the training status of M_CELL_NUMBER_X, M_CELL_NUMBER_Y, M_CELL_SIZE_MAX, M_CELL_SIZE_MIN, M_CODE_FLIP, M_DATAMATRIX_SHAPE, M_DECODE_ALGORITHM, M_DOT_SPACING_MAX, M_DOT_SPACING_MIN, M_ENCODING, M_ERROR_CORRECTION, M_FOREGROUND_VALUE, M_SEARCH_ANGLE, M_SPEED, M_THRESHOLD_MODE, and M_USE_PRESEARCH.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default activation; the default activation depends on the control type and code model type. For a given trainable control type, if an unsupported code model is included in the training, the default value will be M_DISABLE, otherwise it will be M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to not train any of the trainable control types.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to train all trainable control types.	c		e	f	g	a		i		j	k	l	m	n	b
M_STOP_TRAIN	Stops the current McodeTrain() operation being performed on the code context passed to CodeId. This must be called from another thread, typically of higher priority. After making this call, the results returned to the result buffer of the McodeTrain() operation are invalid and discarded.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_TIMEOUT	Sets the maximum training time for an McodeTrain() operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the timeout.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the timeout limit, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings can be specified for a code model to control an McodeRead(), McodeGrade(), or McodeTrain() operation. If you pass a code context to the CodeId parameter, the specified control type setting is applied to all the code models in the context. If none of the code models support the specified control type, an error will occur. If only some code models do not support the specified control type, the control type setting is ignored for these code models.

> Note that besides code type restrictions listed explicitly in the values below, McodeGrade() does not support M_4_STATE, M_PHARMACODE, M_POSTNET, and M_PLANET code types.

Although all these control types affect the results of McodeTrain(), see the description of the control types to determine if they can be activated for training; you can also use filters to limit the control types in the table to those that can be trained.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Code model settings for an McodeRead(), McodeGrade(), or McodeTrain() operation																	
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																	

M_BEARER_BAR	Sets whether bearer bars run along the top and bottom of the code occurrences to read (such as, the edge of a sticker). Note that this control type is only taken into account when the search angle is not specified. This value is available only for 1D code types. In situations where bearer bars don't run along both the top and the bottom of the code occurrence to read, setting M_SPEED to M_VERY_LOW will provide more robust results. INQ														
M_DEFAULT	Same as M_ABSENT.	c		e	f	g	a		i		j	k	l	m	n	b
M_ABSENT	Specifies that no bearer bars are above and below the code.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESENT	Specifies that there are bearer bars above and below the code.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_NUMBER_X_MAX	Sets the maximum number of cells for which to search, in the X-direction of a 2D code. Note that **McodeControl**() generates an error if you set both a range of cells for which to search (using M_CELL_NUMBER_X_MAX, M_CELL_NUMBER_X_MIN, M_CELL_NUMBER_Y_MAX, and/or M_CELL_NUMBER_Y_MIN) and an explicit number of cells for which to search (using M_CELL_NUMBER_X and/or M_CELL_NUMBER_Y). This control type is only available for 2D code types. Note that this control type is automatically activated for training if you activate M_CELL_NUMBER_X for training. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to search for code occurrences with any number of cells.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the maximum number of cells for which to search. If a value is specified, it must be larger than M_CELL_NUMBER_X_MIN, otherwise an error occurs. For the M_DOTCODE code type, this setting should be set to a value greater than 4. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_NUMBER_X_MIN	Sets the minimum number of cells for which to search, in the X-direction of a 2D code. Note that **McodeControl**() generates an error if you set both a range of cells for which to search (using M_CELL_NUMBER_X_MAX, M_CELL_NUMBER_X_MIN, M_CELL_NUMBER_Y_MAX, and/or M_CELL_NUMBER_Y_MIN) and an explicit number of cells for which to search (using M_CELL_NUMBER_X and/or M_CELL_NUMBER_Y). This control type is only available for 2D code types. Note that this control type is automatically activated for training if you activate M_CELL_NUMBER_X for training. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to search for code occurrences with any number of cells.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the minimum number of cells for which to search. If a value is specified, it must be smaller than M_CELL_NUMBER_X_MAX, otherwise an error occurs. For the M_DOTCODE code type, this setting should be set to a value greater than 4. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b

M_CELL_NUMBER_Y_MAX	Sets the maximum number of cells for which to search, in the Y-direction of a 2D code. Note that **McodeControl**() generates an error if you set both a range of cells for which to search (using M_CELL_NUMBER_X_MAX, M_CELL_NUMBER_X_MIN, M_CELL_NUMBER_Y_MAX, and/or M_CELL_NUMBER_Y_MIN) and an explicit number of cells for which to search (using M_CELL_NUMBER_X and/or M_CELL_NUMBER_Y). This control type is only available for 2D code types. Note that this control type is automatically activated for training if you activate M_CELL_NUMBER_Y for training. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to search for code occurrences with any number of cells.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the maximum number of cells for which to search. If a value is specified, it must be larger than M_CELL_NUMBER_Y_MIN, otherwise an error occurs. For the M_DOTCODE code type, this setting should be set to a value greater than 4. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_NUMBER_Y_MIN	Sets the minimum number of cells for which to search, in the Y-direction of a 2D code. Note that **McodeControl**() generates an error if you set both a range of cells for which to search (using M_CELL_NUMBER_X_MAX, M_CELL_NUMBER_X_MIN, M_CELL_NUMBER_Y_MAX, and/or M_CELL_NUMBER_Y_MIN) and an explicit number of cells for which to search (using M_CELL_NUMBER_X and/or M_CELL_NUMBER_Y). This control type is only available for 2D code types. Note that this control type is automatically activated for training if you activate M_CELL_NUMBER_Y for training. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to search for code occurrences with any number of cells.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the minimum number of cells for which to search. If a value is specified, it must be smaller than M_CELL_NUMBER_Y_MAX, otherwise an error occurs. For the M_DOTCODE code type, this setting should be set to a value greater than 4. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE_MAX +	Sets the maximum cell size. Note that setting the maximum and minimum cell size increases both the speed and robustness of the operation. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies to automatically select an appropriate size, which is dependent on the code type and the initialization mode. For an M_DOTCODE code type, the default is always 255 when M_INITIALIZATION_MODE is set to M_IMPROVED_RECOGNITION. For all code types, the default is always 24 when M_INITIALIZATION_MODE is set to M_TYPICAL_RECOGNITION.	c		e	f	g	a		i		j	k	l	m	n	b

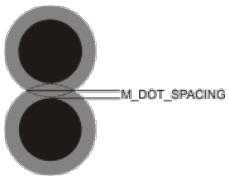
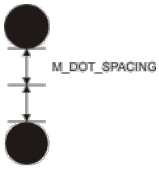
Value	Specifies the maximum cell size, relative to the input coordinate system specified using M_CELL_SIZE_INPUT_UNITS. When entering a value in pixel units, this value must be between 1 and 256, inclusive. When entering a value in world units, the number that results from the internal transformation of world units to pixel units must still respect the value's pixel unit constraint. To find the range of allowed values in world units, according to your associated camera calibration, use McalTransformResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE_MIN +	Sets the minimum cell size. Note that setting the maximum and minimum cell size increases both the speed and robustness of the operation. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default cell size, in pixels, which is dependent on the operation and code type. Setting the default ignores the input unit type chosen with M_CELL_SIZE_INPUT_UNITS. For an McodeRead() operation, M_DEFAULT is equivalent to 1, except for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_MICROQRCODE (which have a default of 3). For an M_DOTCODE code type regardless of the operation, the default is always 2.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the minimum cell size, relative to the input coordinate system specified using M_CELL_SIZE_INPUT_UNITS. When entering a value in pixel units, this value must be between 1 and 256, inclusive. When entering a value in world units, the number that results from the internal transformation of world units to pixel units must still respect the value's pixel unit constraint. To find the range of allowed values in world units, according to your associated camera calibration, use McalTransformResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_CHECK_FINDER_PATTERN	Sets whether checking for a false Data Matrix pattern is enabled. If this control type is enabled, McodeRead() and McodeGrade() operations are more robust, and false Data Matrix code types are not read. This control type should only be enabled if it is possible that parts of your image could be falsely interpreted as a Data Matrix code types. This control type is only available for the M_DATAMATRIX code type. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the Code module will not check for false M_DATAMATRIX code types.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the Code module will check for false M_DATAMATRIX code types.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHECK_QUIET_ZONE	Sets whether the presence of the quiet zone is necessary for a successful McodeRead() or McodeGrade() operation of this code type. Note that the specifications of a code's quiet zone are dependent upon the code type. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value, which is dependent on the code type. The default value for M_DOTCODE is M_DISABLE. For all other code types, the default is M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that a quiet zone is not necessary. When set to M_DISABLE, the McodeRead() or McodeGrade() operation can be less robust, especially for 1D code types.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that a quiet zone is necessary.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_FLIP +		Sets whether code occurrences need to be flipped or read in the opposite direction to be read properly. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value, which is dependent on the initialization mode. When using a code context with M_INITIALIZATION_MODE set to M_IMPROVED_RECOGNITION, the default value is M_ANY. When using a code context with M_INITIALIZATION_MODE set to M_TYPICAL_RECOGNITION, the default value is M_NO_FLIP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ANY	Allows Aurora Imaging Library to decide whether a code occurrence needs to be flipped or read in the opposite direction to be read properly.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FLIP	Specifies that code occurrences need to be flipped or read in the opposite direction to be read properly. When dealing with 1D code types, a code occurrence will be read from left-to-right and from right-to-left. When dealing with 2D code types, a code occurrence will be flipped before being read.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_FLIP	Specifies that code occurrences don't need to be flipped or read in the opposite direction. When dealing with 1D code types, a code occurrence will be read from left-to-right. When dealing with 2D code types, a code occurrence will not be flipped.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_SEARCH_MODE +		Sets the behavior of the decoding algorithm. Note that the default setting is M_FAST in the majority of cases. To avoid obtaining false positives when searching for a specific number of occurrences (M_NUMBER and M_TOTAL_NUMBER), set M_CODE_SEARCH_MODE to M_BEST. This control type is available for 1D code types (excluding M_4_STATE, M_PLANET, and M_POSTNET), the M_DOTCODE code type, and M_COMPOSITECODE code types. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value, which is dependent on the code type. The default value is M_FAST for all code types except M_PHARMACODE, in which case, the default value is M_BEST.	c		e	f	g	a		i		j	k	l	m	n	b
	M_BEST	Specifies to search and return the best quality code occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FAST	Specifies to search and return the first code occurrences decoded.	c		e	f	g	a		i		j	k	l	m	n	b

M_DATAMATRIX_SHAPE +	Sets the shape of the Data Matrix code type. This control type is only available for the M_DATAMATRIX code type. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies that the M_DATAMATRIX code type can be any shape.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECTANGLE	Specifies that the M_DATAMATRIX code has a rectangular shape. This shape can only use the M_ECC_200 error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b
M_SQUARE	Specifies that the M_DATAMATRIX code has a square shape. The number of cells in X and Y must be equal, and the aspect ratio of cell sizes in the X and Y direction must be near 1:1.	c		e	f	g	a		i		j	k	l	m	n	b
M_DECODE_ALGORITHM +	Sets the decoding algorithm to use to read the code occurrences. This control type is available for M_AZTEC, M_DATAMATRIX, M_DOTCODE, M_QRCODE, M_PDF417, and M_TRUNCATED_PDF417 code types. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. Note that training this control type (M_TRAIN) is not supported for the M_DOTCODE code type. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type and the initialization mode. When using a code context with M_INITIALIZATION_MODE set to M_IMPROVED_RECOGNITION, the default value for M_PDF417 and M_TRUNCATED_PDF417 is M_CODE_NOT_DEFORMED, and the default value for the remaining code types is M_CODE_DEFORMED. When using a code context with M_INITIALIZATION_MODE set to M_TYPICAL_RECOGNITION, the default value is M_CODE_NOT_DEFORMED for all code types except M_DOTCODE, in which case, the default value is M_CODE_DEFORMED.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_DEFORMED	Specifies to use the algorithm to decode deformed code occurrences. This algorithm is only available for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_DOTCODE code types. > Note that selecting this algorithm might lead to longer processing times.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_NOT_DEFORMED	Specifies to use the algorithm to decode non-deformed code occurrences.	c		e	f	g	a		i		j	k	l	m	n	b

	M_SCANLINE_AT_ANGLE	Specifies to use the algorithm to decode M_PDF417 or M_TRUNCATED_PDF417 code occurrences using rotated scanlines, without any localization. The angle of the scanlines is determined using M_SEARCH_ANGLE. Select this algorithm if you have codes that do not completely respect their specification (for example, they are missing their quiet zone). To get the best performance, specify a precise M_SEARCH_ANGLE or use M_SEARCH_ANGLE_STEP; also, ensure M_SCANLINE_HEIGHT is not set to a value higher than the row height. This algorithm is only available for M_PDF417 and M_TRUNCATED_PDF417 code types.	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SIZE_INPUT_UNITS		Sets the units with which to interpret the M_DOT_SIZE_MIN and M_DOT_SIZE_MAX control types. This essentially sets the input coordinate system to use. This control type is only available for the M_DOTCODE code type. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL	Specifies to interpret the values in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD	Specifies to interpret the values in world units, with respect to the relative coordinate system. If world units are specified, calling McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() generates an error if the operation is not performed on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SIZE_MAX +		Sets the maximum diameter of the dots for which to search. This control type is only available for the M_DOTCODE code type. Note that setting the maximum and minimum dot size increases both the speed and robustness of the operation. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value, which is dependent on the initialization mode. The default is always 255 when M_INITIALIZATION_MODE is set to M_IMPROVED_RECOGNITION. Whereas the default is always 24 when M_INITIALIZATION_MODE is set to M_TYPICAL_RECOGNITION.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies to select an appropriate size, automatically.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the maximum dot size, relative to the input coordinate system specified using M_DOT_SIZE_INPUT_UNITS. When entering a value in world units, the number that results from the internal transformation of world units to pixel units must still respect the value's pixel unit constraint. To find the range of allowed values in world units, according to your associated camera calibration, use McalTransformResult().	c		e	f	g	a		i		j	k	l	m	n	b

M_DOT_SIZE_MIN +	Sets the minimum diameter of the dots for which to search. This control type is only available for the M_DOTCODE code type. Note that setting the maximum and minimum dot size increases both the speed and robustness of the operation. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to select an appropriate size, automatically.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the minimum dot size, relative to the input coordinate system specified using M_DOT_SIZE_INPUT_UNITS. When entering a value in world units, the number that results from the internal transformation of world units to pixel units must still respect the value's pixel unit constraint. To find the range of allowed values in world units, according to your associated camera calibration, use McalTransformResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SPACING_INPUT_UNITS	Sets the units with which to interpret the M_DOT_SPACING_MIN and M_DOT_SPACING_MAX control types. This essentially sets the input coordinate system to use. This control type is only available for M_AZTEC, M_DATAMATRIX, M_MAXICODE, M_QRCODE, and M_MICROQRCODE code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies what convention is used, automatically. Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the value in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the value in world units, with respect to the relative coordinate system. If world units are specified, calling McodeRead() or McodeGrade() generates an error if the operation is not performed on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b

M_DOT_SPACING_MAX +	Sets the maximum distance between 2 dots in a matrix code type composed of dots. Note that the value entered represents half the distance between two dark cells (dots). To set the minimum distance between 2 dots, use M_DOT_SPACING_MIN. When the dark cells in a printed code are oversized (for example, due to printing artifacts) and overlap, set M_DOT_SPACING_MAX to a negative value equal to half the width of the overlap. The image below illustrates dark cells overlapping, the gray circle represents the actual cell size, while the black dot represents the expected cell size.  By contrast, when the dots are too far apart, specify a positive dot spacing equal to half the distance between the two dots.  This control type is only available for M_AZTEC, M_DATAMATRIX, M_MAXICODE, M_QRCODE, and M_MICROQRCODE code types. Note that overestimating this value will severely impair reading results. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the initialization mode. When using a code context with M_INITIALIZATION_MODE set to M_IMPROVED_RECOGNITION, the default value is 2. When using a code context with M_INITIALIZATION_MODE set to M_TYPICAL_RECOGNITION, the default value is the same as M_DOT_SPACING_MIN. Note that if M_DOT_SPACING_MIN is also set to M_DEFAULT, the result is no spacing.	c		e	f	g	a		i		j	k	l	m	n	b
-256 <= *Value* <= 256	Specifies the distance, relative to the input coordinate system specified using M_DOT_SPACING_INPUT_UNITS. Specifying a positive value means that you are specifying half the distance between the edges of the dots. Specifying a negative value means you are specifying the half-depth of the overlap. If M_DOT_SPACING_INPUT_UNITS is set to M_WORLD, the distance is specified in world units. Aurora Imaging Library transforms the value to pixel units according to the camera calibration of the target image that is specified when reading the code occurrence. > Whether to use a negative or positive dot spacing is a situation best determined by testing. For more information, refer to the Dot spacing subsection of the Customizing read and grade operation settings section of Chapter 17: Codes. When entering a value in world units, the number that results from the internal transformation of world units to pixel units must still respect the value's pixel unit constraint. To find the range of allowed values in world units, according to your associated camera calibration, use McalTransformResult().	c		e	f	g	a		i		j	k	l	m	n	b

M_DOT_SPACING_MIN +	Sets the minimum distance between 2 dots in a matrix code type composed of dots. Note that the value entered represents half the distance between two dark cells (dots). For more information, refer to M_DOT_SPACING_MAX. This control type is only available for M_AZTEC, M_DATAMATRIX, M_MAXICODE, M_QRCODE, and M_MICROQRCODE code types. Note that overestimating this value will severely impair reading results. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the initialization mode. When using a code context with M_INITIALIZATION_MODE set to M_IMPROVED_RECOGNITION, the default value is the same as -1. When using a code context with M_INITIALIZATION_MODE set to M_TYPICAL_RECOGNITION, the default value is the same as M_DOT_SPACING_MAX. Note that if M_DOT_SPACING_MAX is also set to M_DEFAULT, the result is no spacing.	c		e	f	g	a		i		j	k	l	m	n	b
-256 <= *Value* <= 256	Specifies the distance, relative to the input coordinate system specified using M_DOT_SPACING_INPUT_UNITS. Specifying a positive value means that you are specifying half the distance between the edges of the dots. Specifying a negative value means you are specifying the half-depth of the overlap. If M_DOT_SPACING_INPUT_UNITS is set to M_WORLD, the distance is specified in world units. Aurora Imaging Library transforms the value to pixel units according to the camera calibration of the target image that is specified when reading the code occurrence. > Whether to use a negative or positive dot spacing is a situation best determined by testing. For more information, refer to the Dot spacing subsection of the Customizing read and grade operation settings section of Chapter 17: Codes. When entering a value in pixel units, this value must be between -256 and 256, inclusive. When entering a value in world units, the number that results from the internal transformation of world units to pixel units must still respect the value's pixel unit constraint. To find the range of allowed values in world units, according to your associated camera calibration, use McalTransformResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_CORRECTED_NUMBER	Sets whether McodeRead() or McodeGrade() are forced to perform a more robust operation to minimize the number of errors to correct. Enabling M_ECC_CORRECTED_NUMBER decreases performance, particularly in good quality images or in images where the error count is not important. This control type is only available for M_PDF417 and M_TRUNCATED_PDF417 code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform a more robust operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform a more robust operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_FINDER_PATTERN_EXHAUSTIVE_SEARCH	Sets whether to search for the L-shaped finder pattern (the gray boxed area in the following image) to help localize the Data Matrix code. Note that enabling this control type increases the operation time, but is more reliable when a Data Matrix code has little contrast between foreground and background colors or when dealing with complex images. This control type is only available when M_DECODE_ALGORITHM is set to M_CODE_DEFORMED or when M_USE_PRESEARCH is set to M_FINDER_PATTERN_BASE. In addition, this control type is only available for the M_DATAMATRIX code type. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to not perform an exhaustive search.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform an exhaustive search.	c		e	f	g	a		i		j	k	l	m	n	b
M_FINDER_PATTERN_INPUT_UNITS	Sets the units with which to interpret the M_FINDER_PATTERN_MAX_GAP and M_FINDER_PATTERN_MINIMUM_LENGTH control types. This essentially sets the input coordinate system to use. This control type is only available when M_DECODE_ALGORITHM is set to M_CODE_DEFORMED or when M_USE_PRESEARCH is set to M_FINDER_PATTERN_BASE. In addition, this control type is only available for the M_DATAMATRIX code type. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the values in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the values in world units, with respect to the relative coordinate system. If world units are specified, calling McodeRead() or McodeGrade() generates an error if the operation is not performed on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b
M_FINDER_PATTERN_MAX_GAP	Sets the maximum tolerable gap in the finder pattern of a Data Matrix code. A gap is an unintentional space (area of background color), in the solid foreground color area of an M_DATAMATRIX code's finder pattern. A code occurrence that has a finder pattern with a gap larger than the specified value is not read. This control type is only available when M_DECODE_ALGORITHM is set to M_CODE_DEFORMED or when M_USE_PRESEARCH is set to M_FINDER_PATTERN_BASE. In addition, this control type is only available for the M_DATAMATRIX code type. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUTO	Specifies that the maximum tolerable gap in the finder pattern is 6 times the minimum cell size specified using M_CELL_SIZE_MIN.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum gap allowed, in input units specified using M_FINDER_PATTERN_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_FINDER_PATTERN_MINIMUM_LENGTH	Sets the shortest acceptable length of either "arm" of the finder pattern of an M_DATAMATRIX code occurrence. If either "arm" is shorter than the minimum, the code occurrence is not read. This control type is only available when M_DECODE_ALGORITHM is set to M_CODE_DEFORMED or when M_USE_PRESEARCH is set to M_FINDER_PATTERN_BASE. In addition, this control type is only available for the M_DATAMATRIX code type. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the minimum acceptable finder pattern "arm" length is 6 times the minimum cell size specified using M_CELL_SIZE_MIN.	c		e	f	g	a		i		j	k	l	m	n	b
Value	The minimum acceptable finder pattern "arm" length, in input units specified using M_FINDER_PATTERN_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER +	Sets the maximum number of code occurrences of the specified code model to read, grade, or train. Note that this number is limited by the specified maximum total number of code occurrences in the image to read, grade, or train (**McodeControl**() with M_TOTAL_NUMBER). To avoid obtaining false positives when searching for a specific number of occurrences, set M_CODE_SEARCH_MODE to M_BEST. Note that M_BEST is not supported for 2D code types other than the M_DOTCODE code type. The following table lists the code types that support searching for multiple occurrences: Code type — Multiple occurrences Linear bar codes (excluding M_GS1_DATABAR, M_4_STATE, M_PLANET, and M_POSTNET) — Yes M_DATAMATRIX — Yes M_DOTCODE — Yes Only the code types in the table above support this control type. When dealing with all other code types, M_NUMBER must be set to 0 or 1 (or M_DEFAULT). Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. Note that training this control type (M_TRAIN) is only supported for the M_DOTCODE code type. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value, which is dependent on the code type and the initialization mode. When using a code context with M_INITIALIZATION_MODE set to M_IMPROVED_RECOGNITION, the default value is 1 for all code types except M_DOTCODE, in which case, the default value is M_ALL. When using a code context with M_INITIALIZATION_MODE set to M_TYPICAL_RECOGNITION, the default value is 1. Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies that all code model occurrences are read/graded up to the maximum number limited by M_TOTAL_NUMBER.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the maximum number of code occurrences of the specified code model to read or grade. Only integer values are accepted. Only 1D code types (excluding M_GS1_DATABAR, M_4_STATE, M_PLANET, and M_POSTNET code types), the M_DOTCODE code type, and the M_DATAMATRIX code type support specifying a value greater than 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_ACCURACY		Sets the accuracy of positional results. Accuracy depends on the settings of the code context and its models. It is always recommended to set this control type to M_HIGH when performing an McodeGrade() operation; for higher accuracy, this is also recommended when performing an McodeRead() on multiple code occurrences. Note that this can result in longer read times. This control type must be set to M_HIGH before an McodeRead() operation if you want to use its result for an McodeGrade() operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value, which is dependent on the code type and the initialization mode. When using a code context with M_INITIALIZATION_MODE set to M_IMPROVED_RECOGNITION, the default value is M_HIGH for all code types except M_DOTCODE, in which case, the default value is M_LOW. When using a code context with M_INITIALIZATION_MODE set to M_TYPICAL_RECOGNITION, the default value is M_LOW.	c		e	f	g	a		i		j	k	l	m	n	b
	M_HIGH	Specifies to report the positional results of the code operation with high accuracy.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOW	Specifies to report the positional results of the code operation with low accuracy.	c		e	f	g	a		i		j	k	l	m	n	b

M_SEARCH_ANGLE +	Sets the nominal search angle. The search is performed between the range of angles defined by: (M_SEARCH_ANGLE - M_SEARCH_ANGLE_DELTA_NEG) to (M_SEARCH_ANGLE + M_SEARCH_ANGLE_DELTA_POS), inclusively, starting with an angle closest to that of M_SEARCH_ANGLE. For the M_AZTEC, M_DATAMATRIX, M_DOTCODE, M_QRCODE, M_MICROQRCODE, M_MAXICODE, M_PDF417, and M_TRUNCATED_PDF417 code types, the angular range does not affect the speed of the operation. Note that M_SEARCH_ANGLE_DELTA_NEG and M_SEARCH_ANGLE_DELTA_POS must be set to M_DEFAULT when dealing with the following code types: M_PHARMACODE, M_COMPOSITECODE, and M_MICROPDF417. Their default angular search range will be used. When the search speed M_SPEED is set to M_VERY_LOW, a more exhaustive search algorithm is enabled that can exceed the specified angle range. In this case, the specified angle range is used as a starting reference for the search instead of as a search restriction. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0 degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_REGION	Specifies that the nominal angle is set to the angle of the target image's ROI (MbufSetRegion()); recall that for McodeGrade() and McodeRead(), the ROI must be rectangular. If this value is chosen but no ROI has been defined, the angle is set to 0.0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the nominal angle, in degrees, relative to the input coordinate system specified using M_SEARCH_ANGLE_INPUT_UNITS. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b

M_SEARCH_ANGLE_DELTA_NEG	Sets the negative angular range of the search. When M_SEARCH_ANGLE_MODE is enabled, codes are sought at the angular range defined by M_SEARCH_ANGLE - M_SEARCH_ANGLE_DELTA_NEG to M_SEARCH_ANGLE + M_SEARCH_ANGLE_DELTA_POS. When M_SEARCH_ANGLE_MODE is disabled, this setting has no effect. Note that M_SEARCH_ANGLE_DELTA_NEG and M_SEARCH_ANGLE_DELTA_POS must be set to M_DEFAULT when dealing with the following code types: M_PHARMACODE and M_COMPOSITECODE. Their default angular search range will be used: Code type: Maximum search angle M_PHARMACODE: ±5° M_COMPOSITECODE: ±5° When the search speed M_SPEED is set to M_VERY_LOW, a more exhaustive search algorithm is enabled that can exceed the specified angle range. In this case, the specified angle range is used as a starting reference for the search instead of as a search restriction. Note that this control type is automatically activated for training if you activate M_SEARCH_ANGLE for training. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type and the initialization mode. When using a code context with M_INITIALIZATION_MODE set to M_IMPROVED_RECOGNITION, the default value is 180.0°. Note that if your code type is M_PHARMACODE or M_COMPOSITECODE, the default value is 5.0°. When using a code context with M_INITIALIZATION_MODE set to M_TYPICAL_RECOGNITION, the default value is 5.0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 180.0	Specifies a negative angular range, in degrees, relative to the nominal angle set by M_SEARCH_ANGLE. Note that while the nominal angle is relative to the input coordinate system specified using M_SEARCH_ANGLE_INPUT_UNITS, M_SEARCH_ANGLE_DELTA_NEG is not affected by the coordinate system used. For 1D codes, the Code module will not use the search angular range algorithm if M_SEARCH_ANGLE_DELTA_NEG and M_SEARCH_ANGLE_DELTA_POS both have a value of less than 5°, regardless of M_SEARCH_ANGLE_MODE.	c		e	f	g	a		i		j	k	l	m	n	b

M_SEARCH_ANGLE_DELTA_POS	Sets the positive angular range of the search. When M_SEARCH_ANGLE_MODE is enabled, codes are sought at the angular range defined by M_SEARCH_ANGLE - M_SEARCH_ANGLE_DELTA_NEG to M_SEARCH_ANGLE + M_SEARCH_ANGLE_DELTA_POS. When M_SEARCH_ANGLE_MODE is disabled, this setting has no effect. Note that M_SEARCH_ANGLE_DELTA_NEG and M_SEARCH_ANGLE_DELTA_POS must be set to M_DEFAULT when dealing with the following code types: M_PHARMACODE and M_COMPOSITECODE. Their default angular search range will be used: Code type: Maximum search angle M_PHARMACODE: ±5° M_COMPOSITECODE: ±5° When the search speed M_SPEED is set to M_VERY_LOW, a more exhaustive search algorithm is enabled that can exceed the specified angle range. In this case, the specified angle range is used as a starting reference for the search instead of as a search restriction. Note that this control type is automatically activated for training if you activate M_SEARCH_ANGLE for training. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type and the initialization mode. When using a code context with M_INITIALIZATION_MODE set to M_IMPROVED_RECOGNITION, the default value is 180.0°. Note that if your code type is M_PHARMACODE or M_COMPOSITECODE, the default value is 5.0°. When using a code context with M_INITIALIZATION_MODE set to M_TYPICAL_RECOGNITION, the default value is 5.0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 180.0	Specifies a positive angular range, in degrees, relative to the nominal angle set by M_SEARCH_ANGLE. Note that while the nominal angle is relative to the input coordinate system specified using M_SEARCH_ANGLE_INPUT_UNITS, M_SEARCH_ANGLE_DELTA_POS is not affected by the coordinate system used. For 1D codes, the Code module will not use the search angular range algorithm if M_SEARCH_ANGLE_DELTA_NEG and M_SEARCH_ANGLE_DELTA_POS both have a value of less than 5°, regardless of M_SEARCH_ANGLE_MODE.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_INPUT_UNITS	Sets the units with which to interpret the M_SEARCH_ANGLE control type. This essentially sets the input coordinate system to use. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the value in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the value in world units, with respect to the relative coordinate system. If world units are specified, calling McodeRead() or McodeGrade() generates an error if the operation is not performed on a calibrated image. > If there is distortion in the target image, you should correct the image before performing an McodeRead() or McodeGrade() operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_SEARCH_ANGLE_STEP	Sets the angle increment/decrement used when searching for codes through an angular range. When M_SEARCH_ANGLE_STEP is enabled, the search angular range algorithm will first search at the specified nominal angle. It will then toggle between incrementing and decrementing from the nominal angle, with each iteration moving further away from the initial search angle by the value of M_SEARCH_ANGLE_STEP. The process will continue either until the code is found, or the search falls outside of the range defined by M_SEARCH_ANGLE - M_SEARCH_ANGLE_DELTA_NEG and M_SEARCH_ANGLE + M_SEARCH_ANGLE_DELTA_POS. Note that setting the M_SEARCH_ANGLE_STEP will affect both the speed and robustness of the search. Note that this control type is ignored if M_SPEED is set to M_VERY_LOW, or M_SEARCH_ANGLE_MODE is set to M_DISABLE. This control type is available for 1D code types (excluding M_4_STATE, M_PLANET, and M_POSTNET). It is also available for M_PDF417 and M_TRUNCATED_PDF417 code types when M_DECODE_ALGORITHM is set to M_SCANLINE_AT_ANGLE. Note that for M_PDF417 and M_TRUNCATED_PDF417 code types, this control type is automatically activated for training if you activate M_SEARCH_ANGLE for training. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that no explicit increment/decrement is used. The actual increment/decrement used during the search is determined by the search angular range algorithm.	c		e	f	g	a		i		j	k	l	m	n	b
0.1 <= *Value* <= 180.0	Specifies the explicit angle increment/decrement, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SIZE_MAX	Sets the maximum size (number of characters) of the string encoded in a code occurrence. For M_QRCODE, M_MICROQRCODE, M_COMPOSITECODE, and M_GS1_DATABAR code types, the maximum string size is ignored when performing McodeRead() and McodeGrade() operations. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type. For the M_IATA25 code type, the default value is 15. For all other code types, the default value is M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies that there is no maximum string size.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 65535	Specifies the maximum string size. For a 1D code (except M_POSTNET, M_PLANET and M_CODABAR codes), the range of possible values starts at 1. For an M_POSTNET code, the value must be either 5, 9, or 11. For an M_PLANET code, the value must be 11 or 13. For an M_CODABAR code, the value must be at least 3. For all 2D codes and cross-row codes (except M_AZTEC with M_ENCODING set to M_ENC_AZTEC_RUNE and M_MAXICODE with M_ENCODING set to either M_ENC_MODE2 or M_ENC_MODE3) the value can start at 0. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_SIZE_MIN	Sets the minimum size (number of characters) of the string encoded in a code occurrence. For M_QRCODE, M_MICROQRCODE, M_COMPOSITECODE, and M_GS1_DATABAR code types, the minimum string size is ignored when performing McodeRead() and McodeGrade() operations. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type. For the M_IATA25 code type, the default value is 15. For all other code types, the default value is M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies that there is no minimum string size.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 65535	Specifies the minimum string size. For a 1D code (except M_POSTNET, M_PLANET and M_CODABAR codes), the range of possible values starts at 1. For an M_POSTNET code, the value must be either 5, 9, or 11. For an M_PLANET code, the value must be 11 or 13. For an M_CODABAR code, the value must be at least 3. For all 2D codes and cross-row codes (except M_AZTEC with M_ENCODING set to M_ENC_AZTEC_RUNE and M_MAXICODE with M_ENCODING set to either M_ENC_MODE2 or M_ENC_MODE3) the value can start at 0. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_TYPE	Sets the particular code sub-types for which to search. Enabling fewer sub-types will help increase the speed of the operation. Some versions of M_GS1_DATABAR codes only differ by their bar height and not their structure. In these cases, the operation is successful even if only one of the sub-types is enabled. For example, GS1 Databar Omnidirectional and GS1 Databar Truncated can be decoded if either M_GS1_DATABAR_OMNI or M_GS1_DATABAR_TRUNCATED is enabled. The sub-types are additive. If you set the sub-type to M_UPC_A + M_GS1_DATABAR_OMNI, you can search for M_UPC_A and M_GS1_DATABAR codes. If you enabled other sub-types previously, they are no longer enabled. This value is available only for M_GS1_DATABAR and M_COMPOSITECODE code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to search for all of the code sub-types that can be specified for M_SUB_TYPE.	c		e	f	g	a		i		j	k	l	m	n	b
M_EAN8	Specifies that the EAN 8 code sub-type is enabled. M_COMPOSITECODE is the only code type that supports this setting.	c		e	f	g	a		i		j	k	l	m	n	b
M_EAN13	Specifies that the EAN 13 code sub-type is enabled. M_COMPOSITECODE is the only code type that supports this setting.	c		e	f	g	a		i		j	k	l	m	n	b
M_GS1_128	Specifies that the GS1-128 code sub-type is enabled. M_COMPOSITECODE is the only code type that supports this setting.	c		e	f	g	a		i		j	k	l	m	n	b

	M_GS1_DATABAR_EXPANDED	Specifies that the GS1 Databar Expanded code sub-type is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GS1_DATABAR_EXPANDED_STACKED	Specifies that the GS1 Databar Expanded Stacked code sub-type is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GS1_DATABAR_LIMITED	Specifies that the GS1 Databar Limited code sub-type is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GS1_DATABAR_OMNI	Specifies that the GS1 Databar Omnidirectional code sub-type is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GS1_DATABAR_STACKED	Specifies that the GS1 Databar Stacked code sub-type is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GS1_DATABAR_STACKED_OMNI	Specifies that the GS1 Databar Stacked Omnidirectional code sub-type is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GS1_DATABAR_TRUNCATED	Specifies that the GS1 Databar Truncated code sub-type is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPC_A	Specifies that the UPC-A code sub-type is enabled. M_COMPOSITECODE is the only code type that supports this setting.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPC_E	Specifies that the UPC-E code sub-type is enabled. M_COMPOSITECODE is the only code type that supports this setting.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_PRESEARCH +		Sets whether the localization operation is performed prior to the decoding step of an operation. Note that, when reading multiple occurrences of M_DATAMATRIX codes, the setting of this control type is ignored. This control type is available for 2D code types, excluding the M_DOTCODE code type. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value, which is dependent on the code type and the initialization mode. When using a code context with M_INITIALIZATION_MODE set to M_IMPROVED_RECOGNITION and the code type is M_DATAMATRIX, the default value is M_FINDER_PATTERN_BASE. For supported 2D code types other than M_DATAMATRIX, the default value is M_STAT_BASE. When using a code context with M_INITIALIZATION_MODE set to M_TYPICAL_RECOGNITION, the default value is M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the operation is not performed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FINDER_PATTERN_BASE	Specifies that the localization operation is only performed on the base pattern of the M_DATAMATRIX code (an "L" starting at the top-most left corner, and ending on the bottom-most right corner of the code). This localization operation is more robust than an M_STAT_BASE localization operation. Note that if the surrounding image is very complex, better results might be obtained by setting M_FINDER_PATTERN_EXHAUSTIVE_SEARCH to M_ENABLE. M_DATAMATRIX is the only code type that supports this setting.	c		e	f	g	a		i		j	k	l	m	n	b

			c		e	f	g	a		i		j	k	l	m	n	b
	M_STAT_BASE	Specifies to localize the code within the image with the statistical characteristics of a 2D bar code (for example, local variance and the presence of a lot of edges). This localization operation is performed faster than an M_FINDER_PATTERN_BASE localization operation. Note that this localization operation might be less efficient when a code occurrence's cell size is greater than a particular value, depending on the code type. To obtain the best results from this operation, you should specify the cell size as a range, using M_CELL_SIZE_MIN and M_CELL_SIZE_MAX, when the cell sizes of the codes in your images are greater than the cell sizes listed in the table below. If they are less than or equal to these values, you can leave M_CELL_SIZE_MIN and M_CELL_SIZE_MAX to their default values. Code type: Cell size Aztec: 9 MicroPDF417: 4 PDF417/Truncated PDF417: 7 QR code: 8 Note that unlisted code types have no known cell size limitations.															

The following ControlType and corresponding ControlValue parameter settings can be specified for a code model to control an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation. If you pass a code context to the CodeId parameter, the specified control type setting is applied to all the code models in the context. If none of the code models support the specified control type, an error occurs. If only some code models do not support the specified control type, the control type setting is ignored for these code models.

> Note that besides code type restrictions listed explicitly in the values below, McodeGrade() does not support M_4_STATE, M_PHARMACODE, M_POSTNET, and M_PLANET code types.

Although all these control types affect the results of McodeTrain(), see the description of the control types to determine if they can be activated for training; you can also use filters to limit the control types in the table to those that can be trained.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Code model settings for an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation			(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlType		Description															
	ControlValue																
M_CELL_NUMBER_X +		Sets the number of cells of a 2D code in the X-direction. Note that setting the number of cells in both the X- and Y-direction, increases the robustness of the operation. Note that **McodeControl**() generates an error if you set both a range of cells for which to search (using M_CELL_NUMBER_X_MAX, M_CELL_NUMBER_X_MIN, M_CELL_NUMBER_Y_MAX, and/or M_CELL_NUMBER_Y_MIN) and an explicit number of cells for which to search (using M_CELL_NUMBER_X and/or M_CELL_NUMBER_Y). This control type is only used for 2D code types. The number of cells is not required for M_MAXICODE because it is a fixed size. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. INQ															
	M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b

M_ANY	Specifies to search for a code with any number of cells, when performing an McodeRead() or McodeGrade() operation. Fits the string to be encoded into the smallest possible number of cells, when performing an McodeWrite() operation.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of cells. This can be any integer value. Note that for the M_PDF417 code type, the value that you pass must be equal to 17*c* + 35, where *c* is the required number of columns, and 35 represents the number of cells required for the start and stop characters. For the M_TRUNCATED_PDF417 code type, the value that you pass must be equal to 17*c* + 18, where *c* is the required number of columns, and 18 represents the number of cells required for the start and stop characters. For the M_MICROPDF417 code type, this setting should be set to 38 for a one-column code, 55 for a two-column code, 82 for a three-column code, and 99 for a four-column code. Only integer values are accepted. For the M_DOTCODE code type, this setting should be set to a value greater than 4. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_NUMBER_Y +	Sets the number of cells of a 2D code in the Y-direction. Note that setting the number of cells in both the X- and Y-direction, increases the robustness of the operation. Note that **McodeControl**() generates an error if you set both a range of cells for which to search (using M_CELL_NUMBER_X_MAX, M_CELL_NUMBER_X_MIN, M_CELL_NUMBER_Y_MAX, and/or M_CELL_NUMBER_Y_MIN) and an explicit number of cells for which to search (using M_CELL_NUMBER_X and/or M_CELL_NUMBER_Y). This control type is only used for 2D code types. The number of cells is not required for M_MAXICODE because it is a fixed size. When used with M_PDF417, M_TRUNCATED_PDF417, and M_MICROPDF417, this represents the number of rows. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to search for a code with any number of cells, when performing an McodeRead() or McodeGrade() operation. Fits the string to be encoded into the smallest possible number of cells, when performing an McodeWrite() operation.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of cells. For exact values, refer to the standard documentation of the code used. For the M_PDF417 and M_TRUNCATED_PDF417 code types, M_CELL_NUMBER_Y should have a value between 3 and 90. The value of M_CELL_NUMBER_Y is used when calculating the value of M_CELL_NUMBER_X. Only integer values are accepted. For the M_MICROPDF417 code type, M_CELL_NUMBER_Y should have a value between 4 and 44. Only integer values are accepted. For the M_DOTCODE code type, this setting should be set to a value greater than 4. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE_INPUT_UNITS	Sets the units with which to interpret the M_CELL_SIZE_MAX and M_CELL_SIZE_MIN control types. This essentially sets the input coordinate system to use. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the values in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

		c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the values in world units, with respect to the relative coordinate system. If world units are specified, calling McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() generates an error if the operation is not performed on a calibrated image.															
M_ENCODING +	Sets the type of encoding scheme. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. Note that training this control type (M_TRAIN) is not supported for the M_DOTCODE code type. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies to use the default encoding scheme, which is dependent on the code type and the initialization mode. The following table lists the default encoding scheme. Note that when performing an McodeWrite() operation, M_DEFAULT is not supported in all circumstances. If your specific code type corresponds to M_ANY, check the description of M_ANY to see if it can be used with an McodeWrite() operation.	c		e	f	g	a		i		j	k	l	m	n	b

Code type	M_IMPROVED_RECOGNITION	M_TYPICAL_RECOGNITION
4-state	M_ENC_US_MAIL	M_ENC_US_MAIL
Aztec	M_ANY	M_ANY
BC412	M_ENC_STANDARD	M_ENC_STANDARD
Codabar	M_ENC_STANDARD	M_ENC_STANDARD
Code 39	M_ENC_STANDARD	M_ENC_STANDARD
Code 93	M_ENC_ASCII	M_ENC_ASCII
Code 128	M_ENC_ASCII	M_ENC_ASCII
Composite	M_ENC_GS1_DATABAR_OMNI	M_ENC_GS1_DATABAR_OMNI
Data Matrix	M_ANY	M_ANY
DotCode	M_ENC_ISO8	M_ENC_ISO8
EAN 8	M_ANY	M_ENC_NUM
EAN 13	M_ANY	M_ENC_NUM
EAN 14	M_ENC_ASCII	M_ENC_ASCII
GS1-128	M_ENC_ASCII	M_ENC_ASCII
GS1 Databar	M_ENC_GS1_DATABAR_OMNI	M_ENC_GS1_DATABAR_OMNI
Industrial 2 of 5	M_ENC_NUM	M_ENC_NUM

		Interleaved 2 of 5	M_ENC_NUM	M_ENC_NUM
		Maxicode	M_ANY	M_ANY
		MicroPDF417	M_ENC_STANDARD	M_ENC_STANDARD
		Micro QR	M_ENC_STANDARD	M_ENC_STANDARD
		PDF417	M_ENC_STANDARD	M_ENC_STANDARD
		Pharmacode	M_ENC_NUM	M_ENC_NUM
		Planet	M_ENC_NUM	M_ENC_NUM
		Postnet	M_ENC_NUM	M_ENC_NUM
		QR	M_ANY	M_ANY
		Truncated PDF	M_ENC_STANDARD	M_ENC_STANDARD
		UPC-A	M_ANY	M_ENC_NUM
		UPC-E	M_ANY	M_ENC_NUM

	M_ANY	Specifies any type of encoding scheme. M_AZTEC, M_DATAMATRIX, M_MAXICODE and M_QRCODE are the code types that can use this encoding scheme for McodeWrite() operations. M_EAN8, M_EAN13, M_UPC_A, M_UPC_E, M_AZTEC, M_DATAMATRIX, M_MAXICODE, and M_QRCODE are the code types that can use this encoding scheme for McodeRead() and McodeGrade() operations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ALPHA	Specifies an encoding scheme that supports uppercase alphabetical characters, along with the space. M_DATAMATRIX is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ALPHANUM	Specifies an encoding scheme that supports alphanumeric characters, as well as the space. M_DATAMATRIX is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ALPHANUM_PUNC	Specifies a similar encoding scheme to M_ENC_ALPHANUM, except it also supports the following characters: (,), (-), (/) and (.). M_DATAMATRIX is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ASCII	Specifies an encoding scheme that supports ASCII characters. M_CODE39, M_CODE93, M_CODE128, M_EAN14, and M_DATAMATRIX are the code types that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_AUSTRALIA_MAIL_C	Specifies an encoding scheme for a 4-state format used with the C encoding table by the Australian Mail service. M_4_STATE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENC_AUSTRALIA_MAIL_N	Specifies an encoding scheme for a 4-state format used with the N encoding table by the Australian Mail service. M_4_STATE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_AUSTRALIA_MAIL_RAW	Specifies an encoding scheme for a 4-state format used by the Australian Mail service. M_4_STATE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_AZTEC_COMPACT	Specifies an encoding scheme for a compact Aztec code. M_AZTEC is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_AZTEC_FULL_RANGE	Specifies an encoding scheme for a full-range (not compact) Aztec code. M_AZTEC is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_AZTEC_RUNE	Specifies an encoding scheme for an Aztec rune (the smallest version of an Aztec code). M_AZTEC is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_EAN8	Specifies an encoding scheme for a composite code whose 1D portion uses an EAN 8 format and whose 2D portion uses a MicroPDF417 format. M_COMPOSITECODE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_EAN8_ADDON	Specifies an encoding scheme for an EAN 8 format with a supplemental 2 or 5 digit add-on. When a code occurrence is read, the character "|" is used as a separator between the string read from the EAN 8 code and the string read from the EAN 8 add-on. M_EAN8 is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_EAN13	Specifies an encoding scheme for a composite code whose 1D portion uses an EAN 13 format and whose 2D portion uses a MicroPDF417 format. M_COMPOSITECODE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_EAN13_ADDON	Specifies an encoding scheme for an EAN 13 format with a supplemental 2 or 5 digit add-on. When a code occurrence is read, the character "|" is used as a separator between the string read from the EAN 13 code and the string read from the EAN 13 add-on. M_EAN13 is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_GS1_128_MICROPDF417	Specifies an encoding scheme for a composite code whose 1D portion uses a GS1 128 format and whose 2D portion uses a MicroPDF417 format. M_COMPOSITECODE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_GS1_128_PDF417	Specifies an encoding scheme for a composite code whose 1D portion uses a GS1 128 format and whose 2D portion uses a PDF417 format. M_COMPOSITECODE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_GS1_DATABAR_EXPANDED	Specifies an encoding scheme that uses a GS1 Databar format. For a composite code, this encoding scheme specifies that the 1D portion uses a GS1 Databar Expanded format and the 2D portion uses a MicroPDF417 format. M_GS1_DATABAR and M_COMPOSITECODE are the code types that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENC_GS1_DATABAR_EXPANDED_STACKED	Specifies an encoding scheme that uses a GS1 Databar Expanded Stacked format. For a composite code, this encoding scheme specifies that the 1D portion uses a GS1 Databar Expanded Stacked format and the 2D portion uses a MicroPDF417 format. M_GS1_DATABAR and M_COMPOSITECODE are the code types that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_GS1_DATABAR_LIMITED	Specifies an encoding scheme that uses a GS1 Databar Limited format. For a composite code, this encoding scheme specifies that the 1D portion uses a GS1 Databar Limited format and the 2D portion uses a MicroPDF417 format. M_GS1_DATABAR and M_COMPOSITECODE are the code types that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_GS1_DATABAR_OMNI	Specifies an encoding scheme that uses a GS1 Databar format. For a composite code, this encoding scheme specifies that the 1D portion uses a GS1 Databar format and the 2D portion uses a MicroPDF417 format. M_GS1_DATABAR and M_COMPOSITECODE are the code types that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_GS1_DATABAR_STACKED	Specifies an encoding scheme that uses a GS1 Databar Stacked format. For a composite code, this encoding scheme specifies that the 1D portion uses a GS1 Databar Stacked format and the 2D portion uses a MicroPDF417 format. M_GS1_DATABAR and M_COMPOSITECODE are the code types that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_GS1_DATABAR_STACKED_OMNI	Specifies an encoding scheme that uses a GS1 Databar Stacked Omnidirectional format. For a composite code, this encoding scheme specifies that the 1D portion uses a GS1 Databar Stacked Omnidirectional format and the 2D portion uses a MicroPDF417 format. M_GS1_DATABAR and M_COMPOSITECODE are the code types that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_GS1_DATABAR_TRUNCATED	Specifies an encoding scheme that uses a GS1 Databar Truncated format. For a composite code, this encoding scheme specifies that the 1D portion uses a GS1 Databar Truncated format and the 2D portion uses a MicroPDF417 format. M_GS1_DATABAR and M_COMPOSITECODE are the code types that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ISO8	Specifies a similar encoding scheme as M_ENC_ASCII, but supports the extended ASCII character set. M_DATAMATRIX and M_DOTCODE are the code types that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_KOREA_MAIL	Specifies an encoding scheme for a 4-state format used by the Korean Mail service. M_4_STATE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_MODE2	Specifies an encoding scheme that requires a Structured Carrier Message. This encoding scheme can be used for a numeric postal code. M_MAXICODE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_MODE3	Specifies an encoding scheme that requires a Structured Carrier Message. This encoding scheme can be used for an alphanumeric postal code. M_MAXICODE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENC_MODE4	Specifies an encoding scheme that requires a Free Format Message. M_MAXICODE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_MODE5	Specifies an encoding scheme that requires a Free Format Message. This data type has a higher level of error correction than M_ENC_MODE4. M_MAXICODE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_MODE6	Specifies an encoding scheme that requires a Free Format Message. This encoding scheme indicates that the resulting code is used to program a code reader. M_MAXICODE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_NUM	Specifies an encoding scheme that only supports numbers. M_EAN8, M_EAN13, M_INDUSTRIAL25, M_INTERLEAVED25, M_PHARMACODE, M_PLANET, M_POSTNET, M_UPC_A, M_UPC_E, and M_DATAMATRIX are the code types that can use this encoding scheme. If the code type is M_DATAMATRIX, spaces are also allowed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_QRCODE_MODEL1	Specifies an encoding scheme that uses an older version of the QR code format. M_QRCODE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_QRCODE_MODEL2	Specifies an encoding scheme that uses a newer version of the QR code format. This version can handle more data and is more robust than model 1. M_QRCODE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_STANDARD	Specifies different types of encoding schemes, depending on what code type is used. If M_CODE39 or M_CODE93 is the code type, M_ENC_STANDARD supports the alphabet (capital letters), digits 0 - 9, and the (-), space, ($), (/), (+), (.), (%) characters. If M_BC412 is the code type, M_ENC_STANDARD supports numbers and the alphabet (capital letters), except for the letter O. If M_PDF417, M_TRUNCATED_PDF417, or M_MICROPDF417 is the code type, M_ENC_STANDARD supports ASCII and extended ASCII characters. If M_CODABAR is the code type, M_ENC_STANDARD supports digits 0 - 9, the (-), ($), (:), (/), (.), and (+) characters, and the a, b, c, and d characters. It uses the latter four characters as start and stop characters.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_UK_MAIL	Specifies an encoding scheme for a 4-state format used by the UK Mail service. M_4_STATE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_UPCA	Specifies an encoding scheme for a composite code whose 1D portion uses an UPC-A format and whose 2D portion uses a MicroPDF417 format. M_COMPOSITECODE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_UPCA_ADDON	Specifies an encoding scheme for an UPC-A format with a supplemental 2 or 5 digit add-on. When a code occurrence is read, the character "|" is used as a separator between the string read from the UPC-A code and the string read from the UPC-A add-on. M_UPC_A is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_UPCE	Specifies an encoding scheme for a composite code whose 1D portion uses an UPC-E format and whose 2D portion uses a MicroPDF417 format. M_COMPOSITECODE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b

M_ENC_UPCE_ADDON	Specifies an encoding scheme for an UPC-E format with a supplemental 2 or 5 digit add-on. When a code occurrence is read, the character "|" is used as a separator between the string read from the UPC-E code and the string read from the UPC-E add-on. M_UPC_E is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENC_US_MAIL	Specifies the Intelligent Mail Barcode encoding scheme for a 4-state format used by the US Mail service. M_4_STATE is the code type that can use this encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
5 <= *Value* <= 95	Specifies the minimum amount of the symbol that contains error correction information, as a percentage. M_4_STATE is the code type that can use this encoding scheme. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_ERROR_CORRECTION +	Sets the type of error correction. Note that if the size of an M_MICROQRCODE code is the smallest possible size, 11x11 (M1 version), M_ERROR_CORRECTION must be set to either M_ANY or M_DEFAULT when performing McodeRead() or McodeGrade() operations. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. Note that training this control type (M_TRAIN) is not supported for the M_DOTCODE code type. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies to use the default error correction scheme for the code type. The following table lists the default error correction scheme:	c		e	f	g	a		i		j	k	l	m	n	b

4-state	M_ECC_4STATE
Aztec	M_ANY
BC412	M_ECC_NONE
Codabar	M_ECC_NONE
Code39	M_ECC_NONE
Code93	M_ECC_CHECK_DIGIT
Code128	M_ECC_CHECK_DIGIT
Composite	M_ECC_COMPOSITE
Data Matrix	M_ANY
DotCode	M_ANY
Ean8	M_ECC_CHECK_DIGIT
Ean13	M_ECC_CHECK_DIGIT
GS1 Databar	M_ECC_CHECK_DIGIT

		Industrial 2 of 5 | M_ECC_NONE Interleaved 2 of 5 | M_ECC_NONE Maxicode | M_ECC_REED_SOLOMON MicroPDF417 | M_ECC_REED_SOLOMON Micro QR | M_ANY PDF417 | M_ANY Pharmacode | M_ECC_NONE Planet | M_ECC_CHECK_DIGIT Postnet | M_ECC_CHECK_DIGIT QR | M_ANY Truncated PDF417 | M_ANY UPC-A | M_ECC_CHECK_DIGIT UPC-E | M_ECC_CHECK_DIGIT														
	M_ANY	Specifies that the error correction type for McodeRead() and McodeGrade() operations is detected automatically. This error correction scheme is only supported for write operations (McodeWrite()) when dealing with M_AZTEC, M_QRCODE, M_MICROQRCODE, M_PDF417, or M_TRUNCATED_PDF417. When dealing with M_AZTEC, this value specifies that error correction will make up 23% of the symbol. M_AZTEC, M_DATAMATRIX, M_DOTCODE, M_QRCODE, M_MICROQRCODE, M_PDF417, and M_TRUNCATED_PDF417 are the code types that can use this error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_4STATE	Specifies to use the Reed-Solomon-based algorithm or a check digit type of error correction scheme, depending on the specification of the encoding. M_4_STATE is the code type that can use this error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_200	Specifies to use a Reed-Solomon-based algorithm as an error correction scheme. M_DATAMATRIX is the code type that can use this error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_CHECK_DIGIT	Specifies to use an additional digit to check whether there is an error or not. There is error detection but not correction. M_4_STATE M_BC412, M_EAN8, M_EAN13, M_CODE39, M_CODE93, M_CODE128, M_GS1_DATABAR, M_INDUSTRIAL25, M_INTERLEAVED25, M_POSTNET, M_PLANET, M_UPC_A, and M_UPC_E are the code types that can use this error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ECC_COMPOSITE	Specifies to use the default error correction scheme for the 1D and 2D portions of the composite code. M_COMPOSITECODE is the code type that can use this error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_H	Specifies to use the highest-level error correction scheme. M_QRCODE is the code type that can use this error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_L	Specifies to use the lowest-level error correction scheme. M_QRCODE and M_MICROQRCODE are the code types that can use this error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_M	Specifies to use a medium-low level error correction scheme. M_QRCODE and M_MICROQRCODE are the code types that can use this error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_NONE	Specifies no error correction. M_BC412, M_CODABAR, M_CODE39, M_INDUSTRIAL25, M_INTERLEAVED25, and M_PHARMACODE are the code types that can use this error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_Q	Specifies to use a medium-high level error correction scheme. M_QRCODE and M_MICROQRCODE are the code types that can use this error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_REED_SOLOMON	Specifies to use a Reed-Solomon type of error correction scheme. M_DOTCODE, M_MAXICODE and M_MICROPDF417 are the code types that can use this error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_REED_SOLOMON_*n*	Specifies to use a Reed-Solomon type of error correction scheme. *n* is an integer from 0 to 8. The higher the number, the higher the level of error correction. M_PDF417 and M_TRUNCATED_PDF417 are the code types that can use this error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	5 <= *Value* <= 95	Specifies the minimum percentage of the symbol that contains error correction information. M_AZTEC is the only code type that can use this error correction scheme. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_VALUE +		Sets the foreground color of the code. Note that this control type can be trained using McodeTrain(); activate it for training with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value, which is dependent on the code type and the initialization mode. When using a code context with M_INITIALIZATION_MODE set to M_IMPROVED_RECOGNITION, the default value is M_FOREGROUND_ANY. When using a code context with M_INITIALIZATION_MODE set to M_TYPICAL_RECOGNITION, the default value is M_FOREGROUND_BLACK for all code types except M_DOTCODE, in which case, the default value is M_FOREGROUND_WHITE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_FOREGROUND_ANY	Specifies the foreground color as black or white. Note that using this value will impact the performance of McodeRead() and McodeGrade() operations and should be used only when the foreground can change or cannot be known beforehand. When used for McodeWrite() operations, the default color (M_FOREGROUND_WHITE for M_DOTCODE and M_FOREGROUND_BLACK for all other code types) is used.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FOREGROUND_BLACK	Specifies that the foreground color is black.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FOREGROUND_WHITE	Specifies that the foreground color is white.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_MINIMUM_CONTRAST; M_SPEED; M_THRESHOLD_MODE; M_CELL_SIZE_MAX; M_CELL_SIZE_MIN; M_CODE_FLIP; M_CODE_SEARCH_MODE; M_DATAMATRIX_SHAPE; M_DECODE_ALGORITHM; M_DOT_SIZE_MAX; M_DOT_SIZE_MIN; M_DOT_SPACING_MAX; M_DOT_SPACING_MIN; M_NUMBER; M_SEARCH_ANGLE; M_USE_PRESEARCH; M_CELL_NUMBER_X; M_CELL_NUMBER_Y; M_ENCODING; M_ERROR_CORRECTION; M_FOREGROUND_VALUE.

You can add the following value to the above-mentioned values to set whether the control is to be trained.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For training a control type																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_TRAIN		Sets whether to activate or deactivate a specified control type for training. Note that training one control type might automatically train some other related control types that cannot be combined with M_TRAIN. For example, activating M_SEARCH_ANGLE for training, also automatically activates M_SEARCH_ANGLE_STEP, M_SEARCH_ANGLE_MODE, M_SEARCH_ANGLE_DELTA_POS, and M_SEARCH_ANGLE_DELTA_NEG for training.															
	M_DEFAULT	Specifies the default activation; the default activation depends on the control type and code model type. For a given trainable control type, if an unsupported code model is included in the training, the default value will be M_DISABLE, otherwise it will be M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Disables the training of the specified control type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Enables the training of the specified control type.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings can be specified when dealing with an McodeGrade() operation. If you pass a code context to the CodeId parameter, the specified control type setting is applied to all the code models in the context. If none of the code models support the specified control type, an error will occur. If only some code models do not support the specified control type, the control type setting is ignored for these code models.

> Note that besides code type restrictions listed explicitly in the values below, McodeGrade() does not support M_4_STATE, M_PHARMACODE, M_POSTNET, and M_PLANET code types.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Code model settings for an McodeGrade() operation																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_GRADING_STANDARD_EDITION		Sets the grading standard edition to use when performing an McodeGrade() operation. Note that the grading standard edition must correspond to the grading standard specified using M_GRADING_STANDARD. This control type is available for all code types except M_MAXICODE. INQ															

	M_DEFAULT	Specifies to use the most recent grading standard edition that is supported, which is dependent on the code type. When M_GRADING_STANDARD is set to M_ISO_GRADING, the default value for 1D code types supported by McodeGrade() is the same as M_ISO_15416_2016. For supported 2D code types and M_COMPOSITECODE code types, the default value is the same as M_ISO_15415_2011_15416_2016. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING, the default value is the same as M_ISO_29158_2020. When M_GRADING_STANDARD is set to M_SEMI_T10_GRADING, the default value is the same as M_SEMI_T10_0701.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ISO_15415_2011_15416_2000	Specifies to use the ISO/IEC 15415:2011 and ISO/IEC 15416:2000 specifications. This control value is available for 2D and M_COMPOSITECODE code types, excluding M_MAXICODE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ISO_15415_2011_15416_2016	Specifies to use the ISO/IEC 15415:2011 and ISO/IEC 15416:2016 specifications. This control value is available for 2D and M_COMPOSITECODE code types, excluding M_MAXICODE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ISO_15416_2000	Specifies to use the ISO/IEC 15416:2000 specification. This control value is available for 1D code types.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ISO_15416_2016	Specifies to use the ISO/IEC 15416:2016 specification. This control value is available for 1D code types.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ISO_29158_2011	Specifies to use the ISO/IEC 29158:2011 specification. This control value is available for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_MICROQRCODE code types.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ISO_29158_2020	Specifies to use the ISO/IEC 29158:2020 specification. This control value is available for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_MICROQRCODE code types.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SEMI_T10_0701	Specifies to use the Semi T10-0701 specification. This control value is available for the M_DATAMATRIX code type.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings can be specified when dealing with an McodeWrite() operation. If you pass a code context to the CodeId parameter, the specified control type setting is applied to all the code models in the context. If none of the code models support the specified control type, an error will occur. If only some code models do not support the specified control type, the control type setting is ignored for these code models.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Code model settings for an McodeWrite() operation																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_CELL_SIZE_MODE		Sets how to establish the cell size to use when performing an McodeWrite() operation. INQ															
	M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUTO	Specifies to use the largest possible cell size such that the code will be resized as to just fit into the target image of the operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies that the cell size corresponds to the value of M_CELL_SIZE_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE_VALUE	Sets the cell size to use when performing an McodeWrite() operation and M_CELL_SIZE_MODE is set to M_USER_DEFINED. If M_CELL_SIZE_INPUT_UNITS is set to M_WORLD, specify the cell size in world units. The specified cell size is transformed to pixel units according to the calibration of the target image used for writing. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type. This value is equivalent to 7 pixels for M_DOTCODE, 10 pixels for M_MAXICODE, or 4 pixels for all other codes types.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the cell size. Note that if the cell size is specified in pixel units, the minimum value that this control type can take is 1. When entering a value in world units, the number that results from the internal transformation of world units to pixel units must still respect the value's pixel unit constraint. To find the range of allowed values in world units, according to your associated camera calibration, use McalTransformResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SHAPE	Sets the shape of the dots used to draw foreground cells when performing an McodeWrite() operation. This control type is only available for M_AZTEC, M_DATAMATRIX, M_DOTCODE, M_QRCODE, and M_MICROQRCODE code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type. The default value for M_DOTCODE is M_CIRCLE. For all other code types that use M_DOT_SHAPE, the default is M_SQUARE.	c		e	f	g	a		i		j	k	l	m	n	b
M_CIRCLE	Specifies that the foreground cells will be drawn with circular dots.	c		e	f	g	a		i		j	k	l	m	n	b
M_SQUARE	Specifies that the foreground cells will be drawn with square dots.	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SIZE	Sets the size of the dot (or square) when performing an McodeWrite() operation. The dot size corresponds to the circle diameter if M_DOT_SHAPE is set to M_CIRCLE or to the length of the square if it is set to M_SQUARE. If M_CELL_SIZE_INPUT_UNITS is set to M_WORLD, specify the dot size in world units. The specified dot size is transformed to pixel units according to the calibration of the target image used for writing. This control type is only available for M_AZTEC, M_DATAMATRIX, M_DOTCODE, M_QRCODE, and M_MICROQRCODE code types. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type. The default value for M_DOTCODE is 7 pixels. For all other code types that use M_DOT_SIZE, the default value is equivalent to the size of the cell set by M_CELL_SIZE_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the size of the dot or square. When entering a value in world units, the number that results from the internal transformation of world units to pixel units must still respect the value's pixel unit constraint. To find the range of allowed values in world units, according to your associated camera calibration, use McalTransformResult().	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings can be specified when dealing with an McodeRead(), McodeGrade(), McodeDetect(), McodeTrain(), or McodeWrite() operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Result buffer settings dealing with an McodeRead(), McodeGrade(), McodeDetect(), McodeTrain(), or McodeWrite() operation

ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_RESULT_OUTPUT_UNITS		Sets whether to return results in pixels or world units. This essentially sets the output coordinate system to use. The setting of this control type will only affect functions within this module which return positional results. This control type can be changed at any time to return results in the required output units. INQ															
	M_DEFAULT	Same as M_ACCORDING_TO_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. If world units are specified, calling McodeGetResult() generates an error if the result was not calculated on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings can be specified when dealing with an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Result buffer settings dealing with an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_STRING_FORMAT		Sets the format in which to return the string, retrieved using McodeGetResult() with M_STRING. INQ															
	M_DEFAULT	Same as M_AUTO_FORMAT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO_FORMAT	Specifies that the string is returned in the string format associated with the code type. Graphical interpretations are assigned depending on corresponding character sets, allowing for the use of character set ECIs.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GS1_HUMAN_READABLE	Specifies that the string is returned in a format that is human-readable. Note that this is the default format for both an M_EAN14 and M_GS1_128 code type. This format is only available when reading the following code types: M_CODE128, M_EAN14, M_GS1_128, M_GS1_DATABAR, M_AZTEC, M_DATAMATRIX, M_DOTCODE, M_QRCODE, and M_COMPOSITECODE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_JAPANESE	[This is only applicable to Windows] Specifies that the string is returned in the Japanese (Windows-932) encoding. This format is only available when reading the following code types: M_AZTEC, M_PDF417, M_MICROPDF417, M_TRUNCATED_PDF417, M_QRCODE, M_MICROQRCODE, M_DOTCODE, and M_DATAMATRIX. This format is not available with Windows Embedded Compact.	c		e	f	g	a		i		j	k	l	m	n	b

	M_KOREAN	[This is only applicable to Windows] Specifies that the string is returned in the Korean (Windows-949) encoding. This format is only available when reading the following code types: M_AZTEC, M_PDF417, M_MICROPDF417, M_TRUNCATED_PDF417, M_QRCODE, M_MICROQRCODE, M_DOTCODE, and M_DATAMATRIX. This format is not available with Windows Embedded Compact.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LATIN	[This is only applicable to Windows] Specifies that the returned string uses Latin (Windows-1252) encoding. This format is only available when reading the following code types: M_AZTEC, M_PDF417, M_MICROPDF417, M_TRUNCATED_PDF417, M_QRCODE, M_MICROQRCODE, M_DOTCODE, and M_DATAMATRIX. This format is not available with Windows Embedded Compact.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RAW_DATA	Specifies that the string is returned in a raw data format. This format will have the FNC1 separator at the beginning of the string (represented by the è character) and the group separators (GS) will be included in the string (represented by the `ascii=29` character or `\x1D`) (for example, è01034531200000111708050810ABCD1234\x1D4109501101020917). This format is only available when reading the following code types: M_CODE128, M_EAN14, M_GS1_128, M_GS1_DATABAR, all 2D code types (except M_MICROQRCODE), and M_COMPOSITECODE code types. When decoding character set ECIs for M_AZTEC, M_DATAMATRIX, M_MAXICODE, M_MICROPDF417, M_QRCODE, M_PDF417, or M_TRUNCATED_PDF417 code types, encodable ECIs are inserted at the appropriate locations. All back slashes (\) in the string are doubled (\\); while typically unprintable characters, such as Arabic characters, are put in a \xnnnnnn format, where nnnnnn is the hexadecimal value representing the unprintable character.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIMPLIFIED_CHINESE	[This is only applicable to Windows] Specifies that the returned string uses Simplified Chinese (Windows-936) encoding. This format is only available when reading the following code types: M_AZTEC, M_PDF417, M_MICROPDF417, M_TRUNCATED_PDF417, M_QRCODE, M_MICROQRCODE, M_DOTCODE, and M_DATAMATRIX. This format is not available with Windows Embedded Compact.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPCE_COMPRESSED	Specifies that the returned string is in the UPCE compressed string format. Note that this is only available when reading an M_UPC_E code. Otherwise, an error will occur.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UTF8	Specifies that the returned string is in the UTF-8 string format. This format is only available when reading the following code types: M_AZTEC, M_PDF417, M_MICROPDF417, M_TRUNCATED_PDF417, M_QRCODE, M_MICROQRCODE, M_DOTCODE, and M_DATAMATRIX. This format is not available with Windows Embedded Compact.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings can be specified when dealing with an McodeDetect() operation. In this case, an M_CODE_DETECT_RESULT result buffer must be passed to the CodeId parameter.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Result buffer settings dealing with McodeDetect() operation																	
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																

M_STOP_DETECT	Stops the current McodeDetect() operation being performed on the code result buffer passed to CodeId. This must be called from another thread, typically of higher priority. After making this call, the results returned to the result buffer of the McodeDetect() operation are invalid and discarded.																
M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b	

Type-specific versions of the function when using a C compiler under 64-bit

```
void McodeControlInt64 (AIL_ID CodeId, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

CodeId

See CodeId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void McodeControlDouble (AIL_ID CodeId, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

CodeId

See CodeId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcode11.lib.
DLL	Requires ail11.dll; ailcode11.dll.

McodeDetect

Synopsis

Detect 1D code occurrences in an image.

Syntax

```
void McodeDetect(
    AIL_ID ImageBufId,               //in
    AIL_INT SizeOfCodeTypeArray,     //in
    const AIL_INT *CodeTypeArrayPtr, //in
    AIL_INT NumCodesToDetect,        //in
    AIL_INT64 TimeoutValue,          //in
    AIL_INT64 ControlFlag,           //in
    AIL_ID DetectResultCodeId        //out
)
```

Description

This function detects code occurrences of most 1D code types in a specified image. It detects their code type and encoding scheme; it also establishes their position with high accuracy. You can automatically add a code model for each detected code type to a code context, using McodeModel() with M_RESET_FROM_DETECTED_RESULTS. Alternatively, you can use the detected code type and encoding scheme information to manually set up your code models with an appropriate code type and encoding scheme for the code occurrences in your images.

You can retrieve results using McodeGetResult(). Retrieve the code type and encoding scheme with M_CODE_TYPE and M_ENCODING, respectively. Prior to retrieving other results, you should ensure that **McodeDetect**() was successful by verifying that M_STATUS returns M_STATUS_DETECT_OK.

Some code types are a subset of another. If a code occurrence has only the common features of these code types, **McodeDetect**() cannot distinguish if the code occurrence is of one type or the other at decode-time. In these cases, **McodeDetect**() reports the following:

When the following code types cannot be differentiated	Code type reported
M_UPC_A and M_EAN13	M_UPC_A
M_EAN14, M_GS1_128, and M_CODE128	M_EAN14
M_GS1_128 and M_CODE128	M_GS1_128

You can remove this ambiguity by restricting the possible code types with CodeTypeArrayPtr. Another reason to restrict the possible code types is if you are not interested in detecting certain code types; this will accelerate the **McodeDetect**() operation.

> Note that this function does not detect postal, GS1 Databar, Pharmacode, 2D, or composite code types. For the list of code types supported by **McodeDetect**(), use McodeInquire() with M_SUPPORTED_CODE_TYPES_DETECT.

This function tries to detect the specified number of code occurrences. If it detects more than the specified number of code occurrences in the specified image, the function only stores the results of those with the best quality. If it detects less than the specified number, McodeGetResult() with M_STATUS returns M_STATUS_DETECT_FAILED.

Parameters

ImageBufId

Specifies the identifier of an image buffer that contains the code occurrences to detect. The image buffer must be an 8-bit unsigned 1-band buffer.

This image buffer can only have a rectangular region of interest (ROI) defined using MbufSetRegion() (with or without rotation). The region must be defined in vector format from a 2D graphics list (M_VECTOR or M_VECTOR_AND_RASTER). An error will be generated if the ROI is only in raster format (M_RASTER) or a non-rectangular shape.

SizeOfCodeTypeArray

Specifies the size of the array passed to CodeTypeArrayPtr.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the CodeTypeArrayPtr parameter.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the size of the code type array																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value = 0	Specifies that an array is not passed to CodeTypeArrayPtr. **McodeDetect**() will search for all code types supported by it. This value is only applicable when CodeTypeArrayPtr is set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the size of the array.	c		e	f	g	a		i		j	k	l	m	n	b

CodeTypeArrayPtr
Accepts the address of one of the following:

- array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>] (and a maximum array rank for C# of 1) Required array size:(SizeOfCodeTypeArray)

Specifies the address of an array listing the possible code types to detect in the image. Instead of passing an array with M_SUPPORTED_CODE_TYPES_DETECT, you can set this parameter to **M_NULL** and set SizeOfCodeTypeArray to 0.

If an array is passed to CodeTypeArrayPtr, it can contain one or more of the following values to specify which code types to look for:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the code types to detect																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BC412	Specifies a BC412 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODABAR	Specifies a Codabar code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE39	Specifies a Code 39 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE93	Specifies a Code 93 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE128	Specifies a Code 128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_EAN8	Specifies an EAN 8 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_EAN13	Specifies an EAN 13 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_EAN14	Specifies an EAN 14 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_GS1_128	Specifies a GS1-128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_IATA25	Specifies an IATA 2 of 5 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_INDUSTRIAL25	Specifies an Industrial 2 of 5 (standard 2 of 5) code type.	c		e	f	g	a		i		j	k	l	m	n	b

M_INTERLEAVED25	Specifies an Interleaved 2 of 5 (ITF-14) code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED_CODE_TYPES_DETECT	Specifies all 1D code types supported by this function.	c		e	f	g	a		i		j	k	l	m	n	b
M_UPC_A	Specifies a UPC-A code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_UPC_E	Specifies a UPC-E code type.	c		e	f	g	a		i		j	k	l	m	n	b

NumCodesToDetect

Specifies the total number of code occurrences to detect in the image, regardless of their code type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of code occurrences to detect																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the number of code occurrences to detect.	c		e	f	g	a		i		j	k	l	m	n	b

TimeoutValue

Specifies the timeout period, in msec.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

DetectResultCodeId

Specifies the code detect result buffer in which to write the results. The result buffer must have been allocated using McodeAllocResult() with M_CODE_DETECT_RESULT.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcode11.lib.
DLL	Requires ail11.dll; ailcode11.dll.

McodeDraw

Synopsis

Draw specific features of results obtained from an McodeRead(), McodeGrade(), McodeWrite(), or McodeDetect() operation.

Syntax

```
void McodeDraw(
    AIL_ID ContextGraId,           //in
    AIL_ID ResultCodeId,           //in
    AIL_ID DstImageBufOrListGraId, //out
    AIL_INT64 Operation,           //in
    AIL_INT ResultIndex,           //in
    AIL_INT ScanIndex,             //in
    AIL_INT64 ControlFlag          //in
)
```

Description

This function draws specific features of results, obtained from an McodeRead(), McodeGrade(), McodeWrite(), or McodeDetect() operation, in the specified destination image buffer or 2D graphics list.

Parameters

ContextGraId

Specifies the identifier of the 2D graphics context to use when drawing. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 2D graphics context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the default 2D graphics context of the current Aurora Imaging Library application is used. Note that there is a different default 2D graphics context for each thread.	c		e	f	g	a		i		j	k	l	m	n	b
2D graphics context identifier	Specifies a valid 2D graphics context identifier, which you have allocated using MgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ResultCodeId

Specifies the identifier of the code read, grade, or detect result buffer from which to extract features to draw. The code result buffer must have been previously allocated on the required system using McodeAllocResult() with M_CODE_READ_RESULT, M_CODE_GRADE_RESULT, M_CODE_WRITE_RESULT, or M_CODE_DETECT_RESULT.

DstImageBufOrListGraId

Specifies the identifier of the destination image buffer or 2D graphics list in which to draw specific features of results.

If specifying an image buffer, it must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. The image buffer must be an 8-bit, unsigned, 1-band image buffer, allocated using MbufAlloc...().

In general, the destination image buffer should be the same size as the image that you just read. For the M_DRAW_REFLECTANCE_PROFILE and M_DRAW_SCAN_PROFILES drawing operations, determine the required image buffer width using McodeGetResult() with M_SCAN_REFLECTANCE_PROFILE_LENGTH. For best results for an M_DRAW_REFLECTANCE_PROFILE drawing operation, the image buffer must have a height of at least 256.

If you specify a 2D graphics list, the 2D graphics list must have been previously allocated using MgraAllocList().

By drawing into the display's overlay buffer or associating the 2D graphics list with the display, you can also annotate an image non-destructively.

Operation

Specifies the type of drawing operation to perform. Drawing operations can be added together to draw multiple features at a time. For example, to draw both the result occurrence's position and code symbol as it was read using McodeRead(), set the Operation parameter to

M_DRAW_POSITION + M_DRAW_CODE.

The following drawing operations are only available after performing an McodeRead() or McodeGrade() operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of drawing operation after an McodeRead()or McodeGrade() operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_CODE +	Draws the specified code occurrence as it was read by McodeRead() or McodeGrade(). This drawing operation is not supported for GS1 Databar, MicroPDF417, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_DECODED_SCANS +	Draws the specified decoded scanline(s) that an McodeRead() or McodeGrade() operation used to read the specified code occurrence(s). The scanlines are drawn using their start and end points (McodeGetResult() with M_DECODED_SCANS_START... and M_DECODED_SCANS_END...). You can specify the scanline for which to draw results using ScanIndex. This drawing operation is available for linear and composite code types. For composite code types, only the scanlines in the 1D portion of the code occurrence(s) are drawn.	c		e	f	g	a		i		j	k	l	m	n	b

The following types of drawing operations are only available after performing an McodeRead(), McodeGrade(), McodeWrite() operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of drawing operation after an McodeRead(), McodeGrade(), McodeWrite() operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_POSITION +	Draws a cross-like symbol at the mid-point of the code occurrence. Note that for a Data Matrix code occurrence, after an McodeRead() or McodeGrade() operation, the cross symbol is drawn in the middle of the top-left cell. For a composite code occurrence, the cross is drawn at the midpoint of the 1D part of the code. For a Maxicode code occurrence, the cross appears in the center of the bull's eye pattern. Regardless of the code type, the cross is drawn maintaining the angle of the occurrence.	c		e	f	g	a		i		j	k	l	m	n	b

The following types of drawing operations are only available after performing an McodeGrade() operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of drawing operation after an McodeGrade() operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_EXTENDED_AREA +	Draws a box around the specified code occurrence, including its extended area, as analyzed by the McodeGrade() operation. Note that this drawing operation is only available if M_EXTENDED_AREA_REFLECTANCE_CHECK was enabled for the McodeGrade() operation. To establish the dimensions of occurrence with its extended area, use McodeGetResult() with M_EXTENDED_AREA_TOP... and M_EXTENDED_AREA_BOTTOM.... This drawing operation is available for Aztec, Data Matrix, QR code, and Micro QR code types.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_REFLECTANCE_PROFILE +	Draws the scan reflectance profile of the specified scanline(s) of the code occurrence, as analyzed by the McodeGrade() operation. You can specify the scanline for which to draw results using ScanIndex. The result is not scaled to the size of the destination image buffer. For best results, the image buffer must have a height of at least 256.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_SCAN_PROFILES +	Draws the specified scanline(s) analyzed in the code occurrence by the McodeGrade() operation. You can specify the scanline for which to draw results using ScanIndex. The drawn scanline(s) represents only the region(s) from which the results were successfully located.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values that can be used alone or as combination values for

Tables:

- For specifying the type of drawing operation after an McodeRead()or McodeGrade() operation
- For specifying the type of drawing operation after an McodeRead(), McodeGrade(), McodeWrite() operation
- For specifying the type of drawing operation after an McodeGrade() operation

the values listed in all the values group tables.

You can use one or more of the following values in combination with each other or with the above-mentioned values, to specify a drawing operation that is available after an McodeRead(), McodeGrade(), McodeWrite(), or McodeDetect() operation.

Refer to individual values for usage details after specific code operations.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a drawing operation that is available after either an McodeRead(), McodeGrade(), McodeWrite(), or McodeDetect() operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_BOX +	Draws a box around the code occurrence. This box is drawn according to the corner coordinates of the code occurrence. You can retrieve these coordinates using McodeGetResult() with M_BOTTOM... and M_TOP.... The box only contains the part of the code occurrence that was processed; for occurrences of 1D code types from an McodeRead() operation performed in low accuracy (McodeControl() with M_POSITION_ACCURACY set to M_LOW), this means that the box might not encompass the entire code occurrence. The box is drawn maintaining the angle and scale of the occurrence. When dealing with composite code occurrences (only supported by McodeRead(), McodeGrade(), and McodeWrite() operations), the box is drawn around the 1D component of the code occurrence. To draw the box around the 2D component of a code occurrence, combine M_DRAW_BOX with M_2D_COMPONENT. > Note that after an McodeDetect() operation, you must use this value alone; it cannot be used in combination with any other value.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_QUIET_ZONE +	Draws a box around the specified code occurrence, including its quiet zone, as analyzed by the McodeRead(), McodeGrade(), or McodeWrite() operation. The box is drawn maintaining the angle and scale of the occurrence. To establish the dimensions of occurrence with its quiet zone, use McodeGetResult() with M_QUIET_ZONE_TOP... and M_QUIET_ZONE_BOTTOM.... The box only contains the part of the code occurrence that was processed; for occurrences of 1D code types from an McodeRead() operation performed in low accuracy (McodeControl() with M_POSITION_ACCURACY set to M_LOW), this means that the box might not encompass the entire code occurrence or might include white space. > Note that this drawing operation is not available after an McodeDetect() operation.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_DRAW_POSITION; M_DRAW_REFLECTANCE_PROFILE; M_DRAW_SCAN_PROFILES; M_DRAW_BOX; M_DRAW_QUIET_ZONE.

You can add one of the following values to the above-mentioned values to specify the component of a composite code occurrence for which to draw McodeGrade() results.

These values are only available when the ResultCodeId parameter is set to the identifier of a code grade result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the component of a composite code occurrence for which to perform the drawing after an McodeGrade() operation																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_LINEAR_COMPONENT	Draws the grading result for the 1D component (linear component) of a composite code occurrence.	c		e	f	g	a		i		j	k	l	m	n	b
M_2D_COMPONENT	Draws the grading result for the 2D component of a composite code occurrence.	c		e	f	g	a		i		j	k	l	m	n	b

ResultIndex

Specifies the code occurrence for which to draw results.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the occurrence index of the code result																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Performs the specified operation for all code occurrences in the code result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *OccurrenceIndex* < M_NUMBER	Specifies the index of the code occurrence in the code result buffer. See the Retrieving results section of Chapter 17: Codes for more information on code occurrence indices.	c		e	f	g	a		i		j	k	l	m	n	b

ScanIndex

Specifies the scanline(s) for which to draw results.

> Note that specifying a scanline is only applicable to drawing operations that draw results for specified scanlines (for example, M_DRAW_REFLECTANCE_PROFILE or M_DRAW_SCAN_PROFILES). For all other drawing operations, set this parameter to M_GENERAL. If specifying both a scanline-specific drawing operation and a general drawing operation (for example, M_DRAW_REFLECTANCE_PROFILE + M_DRAW_CODE), set this parameter to the scanline(s) for which to draw results instead of passing M_GENERAL.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the scanline																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value. For a scanline-specific drawing operation, this value is the same as M_ALL. For a general drawing operation, this value is the same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to perform the scanline-specific drawing operation for all scanlines of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to perform the drawing operation without specifying a scanline. > Note that this value is not supported for the M_DRAW_REFLECTANCE_PROFILE and M_DRAW_SCAN_PROFILES drawing operations.	c		e	f	g	a		i		j	k	l	m	n	b

0 <= *Value* < M_NUMBER_OF_DECODED_SCANS	Specifies the index of the decoded scanline for which to perform the drawing operation, for the specified code occurrence. > Note that this value cannot be used if ResultIndex is set to M_ALL or M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_NUMBER_OF_SCANS	Specifies the index of the scanline for which to perform the drawing operation, for the specified code occurrence. > Note that this value cannot be used if ResultIndex is set to M_ALL or M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies the function's control flag. Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcode11.lib.
DLL	Requires ail11.dll; ailcode11.dll.

McodeFree

Synopsis

Free a code context or a code result buffer.

Syntax

```
void McodeFree(
    AIL_ID ContextOrResultCodeId //in
)
```

Description

This function deletes the specified code context (and all its models) or code result buffer identifier, and releases any memory associated with it.

To only delete individual models in the context, use McodeModel() with M_DELETE.

All code contexts and all code result buffers allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

ContextOrResultCodeId

Specifies the identifier of the code context or code result buffer to free. These must have been successfully allocated (with McodeAlloc() or McodeAllocResult()) prior to calling this function.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcode11.lib.
DLL	Requires ail11.dll; ailcode11.dll.

McodeGetResult

Synopsis

Get the specified type of result for the specified code occurrence(s), or in the case of an McodeTrain() operation, for the specified code model(s), from a code result buffer.

Syntax

```
void McodeGetResult(
    AIL_ID ResultCodeId,    //in
    AIL_INT ResultIndex,    //in
    AIL_INT RowOrScanIndex, //in
    AIL_INT64 ResultType,   //in
    void *UserVarArrayPtr   //out
)
```

Description

This function retrieves the result(s) of the specified type for the specified code occurrence(s), or in the case of an McodeTrain() operation, for the specified code model(s), from a code result buffer. For results from an McodeGrade() operation, you can also retrieve a result for a specific scanline or row of a specified code occurrence.

In some cases, you can also retrieve the specified result for all occurrences, or in the case of McodeTrain(), for all code models. The order in which these results are returned depends on the code operation that returned them. Use M_CODE_MODEL_ID to establish the corresponding code model of each returned result (or set of results for result types that return multiple values such as M_DATA_CODEWORDS). For more information on the order in which results are returned, see the Retrieving results section of Chapter 15: Codes.

The code operation must have been performed prior to calling **McodeGetResult**(); otherwise, an error will occur.

Besides code type restrictions listed explicitly in the result types below, keep in mind that McodeGrade() does not support M_4_STATE, M_PHARMACODE, M_POSTNET, and M_PLANET code types.

The results from an McodeTrain() operation are the recommended control type settings for a code context and its code models, based on the occurrences found in all the training images. To retrieve the results of the read operation that McodeTrain() internally performed on each training image, use M_CODE_RESULT_ID. This retrieves the identifiers of internal code read result buffers; there is one per training image. You can then pass **McodeGetResult**() one of these result buffer identifiers and retrieve any type of result available for an McodeRead() operation.

From an McodeGrade() operation, a resulting quality grade (such as, M_CONTRAST_UNIFORMITY_GRADE) is typically based on a calculation described in the standard selected using McodeControl() with M_GRADING_STANDARD. The calculation returns one result per codeword or one result per code occurrence, depending on what was being graded. In most cases, there is a non-grade result which has more information about the test performed (such as, M_CONTRAST_UNIFORMITY). The grade is based on a scale from 4.0 to 0.0 that is converted to an alphabetic grade (A through F).

When dealing with results that are derived from other results, retrieve the results for the associated results first. For example, before retrieving M_SCAN_DEFECTS, retrieve M_SCAN_ERN_MAXIMUM and M_SCAN_SYMBOL_CONTRAST. This assists in narrowing down where your code occurrence (or potentially the configuration of your code context) is lacking.

If your target image was associated with a camera calibration context, positional and dimensional results are, by default, returned with respect to the relative coordinate system of the image. Otherwise, these results are returned in pixels, relative to the top-left pixel in the target image.

If your target image was associated with a camera calibration context but you want to retrieve positional and dimensional results in pixel units, use McodeControl() with the M_RESULT_OUTPUT_UNITS control type set to M_PIXEL. However, note that if you set M_RESULT_OUTPUT_UNITS to M_WORLD without specifying a calibrated target image in which to calculate the results, **McodeGetResult**() will generate an error.

Note that **McodeGetResult**() in combination with McodeWrite() can be used to determine the required image buffer size to encode a given string. First, call McodeWrite() with ImageBufId set to M_NULL and then call **McodeGetResult**() to retrieve the minimum width (M_WRITE_SIZE_X) and height (M_WRITE_SIZE_Y) required to draw the code. For simplicity, the codes drawn using McodeWrite() are referred to as "code occurrences" in the ResultType tables below.

Parameters

ResultCodeId

Specifies the identifier of the code result buffer from which to retrieve results.

See the Parameter associations section for possible values that can be specified.

ResultIndex

Specifies the code occurrence(s) or code model(s) for which to retrieve results.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the occurrence or model index																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value. For code occurrence-specific or code model-specific result types, this value is the same as M_ALL. For other result types, this value is the same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to get results related to all code occurrences. > Note that when this value is used, results from an McodeDetect(), McodeRead(), McodeGrade(), or McodeWrite() operation are returned in the order in which the code occurrences were found, regardless of the code model/code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to get general results related to all code occurrences, or in the case of McodeTrain(), the recommended control type settings for the code context. > Note that the order in which these results are returned depends on the code operation that returned them.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *CodeModelIndex* < M_NUMBER_OF_CODE_MODELS	Specifies the index of the code model in the McodeTrain() result buffer from which to retrieve results. Code models are indexed in the result buffer in the same order as they were added to the training context. Use M_CODE_MODEL_ID to establish the corresponding code model identifier. See the Retrieving results section of Chapter 17: Codes for more information on result indices.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *CodeOccurrenceIndex* < M_NUMBER	Specifies the index of the code occurrence in the McodeRead(), McodeGrade(), or McodeDetect() result buffer from which to retrieve results. Code occurrences are indexed in the result buffer in the order in which they were found, regardless of the code model/code type. Use M_CODE_MODEL_ID and M_CODE_TYPE to establish the corresponding code model and code type, respectively. See the Retrieving results section of Chapter 17: Codes for more information on result indices.	c		e	f	g	a		i		j	k	l	m	n	b

RowOrScanIndex

Specifies the scanline(s) or row(s) of the code occurrence in the McodeGrade() result buffer for which to retrieve results. For result types that are not scanline-specific or row-specific, set this parameter to M_GENERAL or M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the scanline result index or row result index																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Specifies the default value. For a scanline-specific or row-specific result type, this value is the same as M_ALL. For other result types, this value is the same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to retrieve the results related to all scanlines or rows of the specified code occurrence.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to retrieve general results.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_NUMBER_OF_ROWS	Specifies the row for which to retrieve results of the specified code occurrence. 2D cross-row code types, composite code types, and the stacked sub-types of the GS1 Databar code type have multiple rows; the remaining 1D code types have one row. > Note that this value cannot be used if ResultIndex is set to M_ALL or M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_NUMBER_OF_SCANS	Specifies the scanline for which to retrieve results of the specified code occurrence. > Note that this value cannot be used if ResultIndex is set to M_ALL or M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result to retrieve.

Not all result types are available for all operations; the availability of results depends on which operation has been performed. In the tables below, the result types are grouped according to the operations after which they are available.

See the Parameter associations section for possible values that can be specified.

UserVarArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64
- AIL_TEXT_CHAR

Specifies the address in which to place the specified result.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ResultCodeId** and **ResultType** parameters and possible values returned to the **UserVarArrayPtr** parameter.

- For retrieving the number of code occurrences from an McodeDetect(), McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation
- For retrieving general results from an McodeDetect(), McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation
- For retrieving code model-specific results from an McodeTrain() operation
- For retrieving code occurrence-specific results from an McodeDetect(), McodeRead(), McodeGrade(), or McodeWrite() operation
- For retrieving row-specific results from an McodeRead() or McodeGrade() operation
- For retrieving scanline-specific results from an McodeRead() or McodeGrade() operation
- For retrieving information about the code context or code model control types that were enabled for training using the McodeTrain()

operation

To retrieve the number of code occurrences, the ResultCodeId and ResultType parameters can be set to the following values. To retrieve the result obtained from an McodeTrain() operation, ResultIndex should be set to the index of a specific code model or M_ALL (or M_DEFAULT); whereas, to retrieve the result obtained from an McodeDetect(), McodeRead(), McodeGrade(), or McodeWrite() operation, ResultIndex should be set to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the number of code occurrences from an McodeDetect(), McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation																
ResultCodeId ResultType UserVarArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Code result buffer ID	Specifies a code detect, read, grade, train, or write result buffer, allocated using McodeAllocResult() with M_CODE_..._RESULT.															
M_NUMBER +	Retrieves the total number of code occurrences. Note that multiple occurrences of the same model are counted separately. Note that only 1D code types (excluding M_GS1_DATABAR, M_4_STATE, M_PLANET, and M_POSTNET code types), the M_DOTCODE code type, and the M_DATAMATRIX code type support multiple occurrences. Since training the corresponding control type is only supported for the M_DOTCODE code type, this result type is only available for the M_DOTCODE code type after an McodeTrain() operation. For the remaining operations, this result type is available for all code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the maximum number of code occurrences for an McodeTrain() operation; for all other operations, specifies the number of code occurrences.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve a general result that is returned from an McodeDetect(), McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation, the ResultCodeId and ResultType parameters can be set to the following values.

> Note that these result types require that ResultIndex be set to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving general results from an McodeDetect(), McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation																
ResultCodeId ResultType UserVarArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Code detect result buffer ID for general results	Specifies a code detect result buffer, allocated using McodeAllocResult() with M_CODE_DETECT_RESULT, and used to store McodeDetect() results.															

M_STATUS +	Retrieves the status of the McodeDetect() operation. It can be helpful to retrieve the number of code occurrences by calling M_NUMBER, along with the status. For example, if you specify that the McodeDetect() operation should detect two code occurrences (NumCodesToDetect) and only one was detected, the status returned is M_STATUS_DETECT_FAILED. Calling M_NUMBER will return 1.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_DETECT_FAILED	Specifies that the McodeDetect() operation failed.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_DETECT_OK	Specifies that the McodeDetect() operation was successful.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_TIMEOUT_END	Specifies that the McodeDetect() operation timed out.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT_END +	Retrieves whether the timeout limit was reached. You can set the timeout limit using McodeControl() with M_TIMEOUT.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the timeout limit was not reached.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the timeout limit was reached.	c		e	f	g	a		i		j	k	l	m	n	b
Code grade result buffer ID for general results	Specifies a code grade result buffer, allocated using McodeAllocResult() with M_CODE_GRADE_RESULT, and used to store McodeGrade() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS +	Retrieves the status of the McodeGrade() operation. It can be helpful to retrieve the number of code occurrences by calling M_NUMBER, along with the status. For example, if you set an McodeGrade() operation to grade two code occurrences (McodeControl() with M_NUMBER) and only one was graded, the status returned is M_STATUS_GRADE_FAILED. Calling M_NUMBER will return 1. To retrieve the confidence score of an McodeGrade() operation as a grade, use M_OVERALL_SYMBOL_GRADE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_GRADE_FAILED	Specifies that the McodeGrade() operation failed.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_GRADE_OK	Specifies that the McodeGrade() operation was successful.	c		e	f	g	a		i		j	k	l	m	n	b

	M_STATUS_NO_RESULT_AVAILABLE	Specifies that there are no results available. The McodeGrade() operation has not been performed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_STATUS_TIMEOUT_END	Specifies that the McodeGrade() operation timed out.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT_END +		Retrieves whether the timeout limit was reached. You can set the timeout limit using McodeControl() with M_TIMEOUT.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the timeout limit was not reached.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the timeout limit was reached.	c		e	f	g	a		i		j	k	l	m	n	b
Code read result buffer ID for general results		Specifies a code read result buffer, allocated using McodeAllocResult() with M_CODE_READ_RESULT, and used to store McodeRead() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS +		Retrieves the status of the McodeRead() operation. It can be helpful to retrieve the number of code occurrences by calling M_NUMBER, along with the status. For example, if you set an McodeRead() operation to read two code occurrences (McodeControl() with M_NUMBER) and only one was found, the status returned is M_STATUS_NOT_FOUND. Calling M_NUMBER will return 1. If the operation that produced the results expected to operate on multiple code occurrences (McodeControl() with M_NUMBER) and it incurred an error for more than one, the least-common error is returned. For example, if you set an McodeRead() operation to read two code occurrences and one fails because the code was not found (M_STATUS_NOT_FOUND) and the other fails because the encoding type was unknown (M_STATUS_ENC_UNKNOWN), the latter is returned. To retrieve the confidence score of an McodeRead() operation, use M_SCORE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_STATUS_CRC_FAILED	Specifies that the McodeRead() operation failed when validating the CRC.	c		e	f	g	a		i		j	k	l	m	n	b
	M_STATUS_ECC_UNKNOWN	Specifies an unknown error correction type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_STATUS_ENC_UNKNOWN	Specifies an unknown encoding type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_STATUS_ENCODING_ERROR	Specifies that an error occurred when decoding the code.	c		e	f	g	a		i		j	k	l	m	n	b

M_STATUS_NO_RESULT_AVAILABLE	Specifies that there are no results available. The McodeRead() operation has not been performed.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_NOT_FOUND	Specifies that the code was not found.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_READ_OK	Specifies that the McodeRead() operation was successful.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_TIMEOUT_END	Specifies that the McodeRead() operation timed out.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT_END +	Retrieves whether the timeout limit was reached. You can set the timeout limit using McodeControl() with M_TIMEOUT.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the timeout limit was not reached.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the timeout limit was reached.	c		e	f	g	a		i		j	k	l	m	n	b
Code train result buffer ID for general results	Specifies a code train result buffer, allocated using McodeAllocResult() with M_CODE_TRAIN_RESULT, and used to store McodeTrain() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_MODEL_ID +	Retrieves the identifier of the trained code model. Note that this result is not valid when the result buffer is on a remote system.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_RESULT_ID +	Retrieves the identifiers of the internal code read result buffers associated with the training images; there is one result buffer per training image. Each identifier can be used to retrieve the results of the read operation that McodeTrain() internally performed on the corresponding training image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAILED_IMAGES_ID +	Retrieves the identifiers of the training images that have been unsuccessfully read.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAILED_IMAGES_INDEX +	Retrieves the indices of the training images that have been unsuccessfully read.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAILED_NUMBER_OF_IMAGES +	Retrieves the number of images that have been unsuccessfully read.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_MINIMUM_CONTRAST +		Retrieves the recommended setting for the minimum possible contrast between the foreground and background in the target image (McodeControl() with M_MINIMUM_CONTRAST). This result type is available for the M_DOTCODE code type.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 255	Specifies the minimum contrast value.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_CODE_MODELS +		Retrieves the number of code models added to the training context.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_TRAINING_IMAGES +		Retrieves the number of images used in the current train operation.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_PASSED_IMAGES_ID +		Retrieves the identifiers of the training images that have been successfully read.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_PASSED_IMAGES_INDEX +		Retrieves the indices of the training images that have been successfully read.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_PASSED_NUMBER_OF_IMAGES +		Retrieves the number of images that have been successfully read.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_MODE +		Retrieves the recommended setting for whether to enable the search angular range algorithm (McodeControl() with M_SEARCH_ANGLE_MODE) for the code context.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the search angular range algorithm is used.	c		e	f	g	a		i		j	k	l	m	n	b

	M_TRAIN	Specifies that the corresponding control type was not trained since M_SEARCH_ANGLE was not selected for training; alternatively, it means that the corresponding control type does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_SPEED +		Retrieves the recommended setting for the search speed (McodeControl() with M_SPEED) for the code context. This result type is available for all code types except the M_DOTCODE code type.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value, which is dependent on the initialization mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOW	Specifies a low search speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEDIUM	Specifies a medium search speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
	M_VERY_LOW	Specifies a very low search speed.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS +		Retrieves the status of the McodeTrain() operation.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_STATUS_TIMEOUT_END	Specifies that the McodeTrain() operation timed out.	c		e	f	g	a		i		j	k	l	m	n	b
	M_STATUS_TRAIN_FAILED	Specifies that the McodeTrain() operation failed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_STATUS_TRAIN_OK	Specifies that the McodeTrain() operation was successful.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_MODE +		Retrieves the recommended threshold mode setting (McodeControl() with M_THRESHOLD_MODE) for the code context.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ADAPTIVE	Specifies the use of a fast dynamic local threshold. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_GLOBAL_SEGMENTATION	Specifies the use of a global threshold value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_GLOBAL_WITH_LOCAL_RESEGMENTATION	Specifies that the source image was globally thresholded and then the edges in the binarized image were resegmented according to the intensities of the surrounding bars and spaces in the original source image. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code context.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_VALUE +		Retrieves the recommended threshold value setting (McodeControl() with M_THRESHOLD_VALUE) for the code context. Note that, this result type is only available when M_THRESHOLD_MODE is not set to M_ADAPTIVE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO_COMPUTE	Specifies to calculate the threshold value automatically.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRAIN	Specifies that the corresponding control type was not trained since M_THRESHOLD_MODE was not selected for training; alternatively, it means that the corresponding control type does not apply to the code context.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 255	Specifies the threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT_END +		Retrieves whether the timeout limit was reached. You can set the timeout limit using McodeControl() with M_TRAIN_TIMEOUT.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the timeout limit was not reached.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the timeout limit was reached.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_ENABLED_NUMBER_OF_MODEL_CONTROL_TYPES +		Retrieves, for each code model, the number of code model control types that were enabled for training.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINED_NUMBER_OF_MODEL_CONTROL_TYPES +		Retrieves, for each code model, the number of code model control types that can be modified by the training results; that is, the code model control types that will be changed after calling McodeControl() with M_RESET_FROM_TRAINED_RESULTS.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_SCORE +		Retrieves the percentage of successful internal McodeRead() operations. The percentage is the number of successfully-read images out of the total number of images used.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

0.0 <= *Value* <= 100.0	Specifies the percentage of successful internal McodeRead() operations.	c		e	f	g	a		i		j	k	l	m	n	b
Code write result buffer ID for general results	Specifies a code write result buffer, allocated using McodeAllocResult() with M_CODE_WRITE_RESULT, and used to store McodeWrite() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS +	Retrieves the status of the McodeWrite() operation.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_NO_RESULT_AVAILABLE	Specifies that there are no results available. The McodeWrite() operation has not been performed.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_WRITE_FAILED	Specifies that the McodeWrite() operation failed.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_WRITE_OK	Specifies that the McodeWrite() operation was successful.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve a code model-specific result that is returned from an McodeTrain() operation, the ResultCodeId and ResultType parameters can be set to the following values. In this case, the ResultIndex parameter must be set to the index of a specific code model or M_ALL (or M_DEFAULT).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving code model-specific results from an McodeTrain() operation																
ResultCodeId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultType																
UserVarArrayPtr - Possible values returned																
Code train result buffer ID for model-specific results	Specifies a code train result buffer, allocated using McodeAllocResult() with M_CODE_TRAIN_RESULT, and used to store McodeTrain() results.															
M_CELL_NUMBER_X +	Retrieves the recommended number of cells setting in the X-direction (McodeControl() with M_CELL_NUMBER_X) for the trained code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies that the code can have any number of cells in the X-direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of cells in the X-direction.	c		e	f	g	a		i		j	k	l	m	n	b

M_CELL_NUMBER_X_MAX +	Retrieves the recommended setting for the maximum number of cells for which to search, in the X-direction of a 2D code (McodeControl() with M_CELL_NUMBER_X_MAX) for the trained code model. This result type is available for 2D code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to search for code occurrences with any number of cells.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN	Specifies that the corresponding control type was not trained since M_CELL_NUMBER_X was not selected for training; alternatively, it means that the corresponding control type does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the maximum number of cells for which to search. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_NUMBER_X_MIN +	Retrieves the recommended setting for the minimum number of cells for which to search, in the X-direction of a 2D code (McodeControl() with M_CELL_NUMBER_X_MIN) for the trained code model. This result type is available for 2D code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to search for code occurrences with any number of cells.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN	Specifies that the corresponding control type was not trained since M_CELL_NUMBER_X was not selected for training; alternatively, it means that the corresponding control type does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the minimum number of cells for which to search. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_NUMBER_Y +	Retrieves the recommended number of cells setting in the Y direction (McodeControl() with M_CELL_NUMBER_Y) for the trained code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies that the code can have any number of cells in the Y-direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of cells in the Y-direction.	c		e	f	g	a		i		j	k	l	m	n	b

M_CELL_NUMBER_Y_MAX +	Retrieves the recommended setting for the maximum number of cells for which to search, in the Y-direction of a 2D code (McodeControl() with M_CELL_NUMBER_Y_MAX) for the trained code model. This result type is available for 2D code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to search for code occurrences with any number of cells.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN	Specifies that the corresponding control type was not trained since M_CELL_NUMBER_Y was not selected for training; alternatively, it means that the corresponding control type does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the maximum number of cells for which to search. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_NUMBER_Y_MIN +	Retrieves the recommended setting for the minimum number of cells for which to search, in the Y-direction of a 2D code (McodeControl() with M_CELL_NUMBER_Y_MIN) for the trained code model. This result type is available for 2D code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to search for code occurrences with any number of cells.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN	Specifies that the corresponding control type was not trained since M_CELL_NUMBER_Y was not selected for training; alternatively, it means that the corresponding control type does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the minimum number of cells for which to search. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE_MAX +	Retrieves the recommended setting for the maximum cell size (McodeControl() with M_CELL_SIZE_MAX) for the trained code model(s). For 2D matrix code types, this corresponds to the nominal maximum module size, whereas for other code types, this corresponds to the size of the cell in X.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum cell size, relative to the input coordinate system specified using M_CELL_SIZE_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_CELL_SIZE_MIN +	Retrieves the recommended setting for the minimum cell size (McodeControl() with M_CELL_SIZE_MIN) for the trained code model(s). For 2D matrix code types, this corresponds to the nominal minimum module size, whereas for other code types, this corresponds to the size of the cell in X.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the minimum cell size, relative to the input coordinate system specified using M_CELL_SIZE_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_FLIP +	Retrieves the recommended code flip setting (McodeControl() with M_CODE_FLIP) for the trained code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to allow Aurora Imaging Library to decide whether the code needs to be flipped or read in the opposite direction to be read properly.	c		e	f	g	a		i		j	k	l	m	n	b
M_FLIP	Specifies that the code occurrence was reversed.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_FLIP	Specifies that the code occurrence was not reversed.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_MODEL_NUMBER_OF_OCCURRENCES +	Retrieves the total number of occurrences found of the code model(s) in all the images used for training.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_SEARCH_MODE +	Retrieves the recommended setting for the code search mode (McodeControl() with M_CODE_SEARCH_MODE) for the trained code model(s). This result type is available for 1D code types (excluding M_4_STATE, M_GS1_DATABAR, M_PLANET, and M_POSTNET) and the M_DOTCODE code type.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BEST	Specifies to search and return the best quality code occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAST	Specifies to search and return the first code occurrences decoded.	c		e	f	g	a		i		j	k	l	m	n	b

		M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_TYPE +		Retrieves the code type of the specified code model(s). The code type is retrieved as a numeric that can be passed to McodeModel().	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_4_STATE	Specifies a 4-state code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_AZTEC	Specifies an Aztec code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_BC412	Specifies a BC412 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODABAR	Specifies a Codabar code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE39	Specifies a Code 39 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE93	Specifies a Code 93 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE128	Specifies a Code 128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COMPOSITECODE	Specifies a composite code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_DATAMATRIX	Specifies a Data Matrix code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_DOTCODE	Specifies a DotCode code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_EAN8	Specifies an EAN 8 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_EAN13	Specifies an EAN 13 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_EAN14	Specifies an EAN 14 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_GS1_128	Specifies a GS1-128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_GS1_DATABAR	Specifies a GS1 Databar code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IATA25	Specifies an IATA 2 of 5 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INDUSTRIAL25	Specifies an Industrial 2 of 5 (standard 2 of 5) code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INTERLEAVED25	Specifies an Interleaved 2 of 5 (ITF-14) code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_MAXICODE	Specifies a Maxicode code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_MICROPDF417	Specifies a MicroPDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_MICROQRCODE	Specifies a Micro QR code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_PDF417	Specifies a PDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PHARMACODE	Specifies a Pharmacode code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PLANET	Specifies a Planet code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_POSTNET	Specifies a Postnet code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_QRCODE	Specifies a QR code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUNCATED_PDF417	Specifies a Truncated PDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPC_A	Specifies a UPC-A code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPC_E	Specifies a UPC-E code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_DATAMATRIX_SHAPE +		Retrieves the recommended setting for the shape of the Data Matrix code type (McodeControl() with M_DATAMATRIX_SHAPE) for the trained code model. This result type is available for the M_DATAMATRIX code type.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ANY	Specifies that the M_DATAMATRIX code type can be any shape.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RECTANGLE	Specifies that the M_DATAMATRIX code has a rectangular shape. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SQUARE	Specifies that the M_DATAMATRIX code has a square shape. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_DECODE_ALGORITHM +		Retrieves the recommended setting for the decoding algorithm (McodeControl() with M_DECODE_ALGORITHM) for the trained code model. This result type is available for M_AZTEC, M_DATAMATRIX, M_QRCODE, M_PDF417, and M_TRUNCATED_PDF417 code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_DEFORMED	Specifies to use the algorithm to decode deformed code occurrences. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_NOT_DEFORMED	Specifies to use the algorithm to decode non-deformed code occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SCANLINE_AT_ANGLE	Specifies to use the algorithm to decode M_PDF417 or M_TRUNCATED_PDF417 code occurrences using rotated scanlines, without any localization. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SIZE_MAX +		Retrieves the recommended setting for the maximum dot size (McodeControl() with M_DOT_SIZE_MAX) for the M_DOTCODE model. This result type is available for the M_DOTCODE code type.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the maximum dot size, relative to the input coordinate system specified using M_DOT_SIZE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SIZE_MIN +		Retrieves the recommended setting for the minimum dot size (McodeControl() with M_DOT_SIZE_MIN) for the M_DOTCODE model. This result type is available for the M_DOTCODE code type.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the minimum dot size, relative to the input coordinate system specified using M_DOT_SIZE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SPACING_MAX +		Retrieves the recommended setting for the maximum distance between 2 dots in a matrix code type composed of dots (McodeControl() with M_DOT_SPACING_MAX) for the trained code model. This result type is available for M_AZTEC, M_DATAMATRIX, M_MAXICODE, M_QRCODE, and M_MICROQRCODE code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
	-2 <= *Value* <= 3	Specifies the distance.	c		e	f	g	a		i		j	k	l	m	n	b

M_DOT_SPACING_MIN +	Retrieves the recommended setting for the minimum distance between 2 dots in a matrix code type composed of dots (McodeControl() with M_DOT_SPACING_MIN) for the trained code model. This result type is available for M_AZTEC, M_DATAMATRIX, M_MAXICODE, M_QRCODE, and M_MICROQRCODE code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
-2 <= *Value* <= 3	Specifies the distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENCODING +	Retrieves the recommended encoding scheme (McodeControl() with M_ENCODING) for the trained code model(s). Note that since some versions of the GS1 Databar code types only differ by the bar height (and not the structure), the same result is returned for these similar code types. GS1 Databar Omnidirectional and GS1 Databar Truncated will return M_ENC_GS1_DATABAR_OMNI. GS1 Databar Stacked and GS1 Databar Stacked Omnidirectional will return M_ENC_GS1_DATABAR_STACKED. It is possible to obtain a more accurate result to distinguish between these structurally similar code types, using McodeControl() with M_POSITION_ACCURACY set to M_HIGH. Since training the corresponding control type is not supported for the M_DOTCODE code type, this result type is not available for the M_DOTCODE code type after an McodeTrain() operation.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies any type of encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENC_ALPHA	Specifies an encoding scheme that supports uppercase alphabetical characters, along with the space. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENC_ALPHANUM	Specifies an encoding scheme that supports alphanumeric characters, as well as the space. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENC_ALPHANUM_PUNC	Specifies a similar encoding scheme to M_ENC_ALPHANUM, except it also supports the following characters: (,), (-), (/) and (. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENC_ASCII	Specifies an encoding scheme that supports ASCII characters. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENC_AUSTRALIA_MAIL_C	Specifies an encoding scheme for a 4-state format used with the C encoding table by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENC_AUSTRALIA_MAIL_N	Specifies an encoding scheme for a 4-state format used with the N encoding table by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b

		M_ENC_AUSTRALIA_MAIL_RAW	Specifies an encoding scheme for a 4-state format used by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AZTEC_COMPACT	Specifies an encoding scheme for a compact Aztec code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AZTEC_FULL_RANGE	Specifies an encoding scheme for a full-range (not compact) Aztec code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AZTEC_RUNE	Specifies an encoding scheme for an Aztec rune (the smallest version of an Aztec code). INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN8	Specifies an encoding scheme for a composite code whose 1D portion uses an EAN 8 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN8_ADDON	Specifies an encoding scheme for an EAN 8 format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN13	Specifies an encoding scheme for a composite code whose 1D portion uses an EAN 13 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN13_ADDON	Specifies an encoding scheme for an EAN 13 format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_128_MICROPDF417	Specifies an encoding scheme for a composite code whose 1D portion uses a GS1 128 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_128_PDF417	Specifies an encoding scheme for a composite code whose 1D portion uses a GS1 128 format and whose 2D portion uses a PDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_EXPANDED	Specifies an encoding scheme that uses a GS1 Databar format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_EXPANDED_STACKED	Specifies an encoding scheme that uses a GS1 Databar Expanded Stacked format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_LIMITED	Specifies an encoding scheme that uses a GS1 Databar Limited format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_OMNI	Specifies an encoding scheme that uses a GS1 Databar format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_STACKED	Specifies an encoding scheme that uses a GS1 Databar Stacked format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_STACKED_OMNI	Specifies an encoding scheme that uses a GS1 Databar Stacked Omnidirectional format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_TRUNCATED	Specifies an encoding scheme that uses a GS1 Databar Truncated format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_ISO8	Specifies a similar encoding scheme as M_ENC_ASCII, but supports the extended ASCII character set. INFO	c		e	f	g	a		i		j	k	l	m	n	b

		M_ENC_KOREA_MAIL	Specifies an encoding scheme for a 4-state format used by the Korean Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE2	Specifies an encoding scheme that requires a Structured Carrier Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE3	Specifies an encoding scheme that requires a Structured Carrier Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE4	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE5	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE6	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_NUM	Specifies an encoding scheme that only supports numbers. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_QRCODE_MODEL1	Specifies an encoding scheme that uses an older version of the QR code format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_QRCODE_MODEL2	Specifies an encoding scheme that uses a newer version of the QR code format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_STANDARD	Specifies different types of encoding schemes, depending on what code type is used. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UK_MAIL	Specifies an encoding scheme for a 4-state format used by the UK Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UPCA	Specifies an encoding scheme for a composite code whose 1D portion uses an UPC-A format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UPCA_ADDON	Specifies an encoding scheme for an UPC-A format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UPCE	Specifies an encoding scheme for a composite code whose 1D portion uses an UPC-E format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UPCE_ADDON	Specifies an encoding scheme for an UPC-E format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_US_MAIL	Specifies the Intelligent Mail Barcode encoding scheme for a 4-state format used by the US Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
		5 <= *Value* <= 95	Specifies the minimum amount of the symbol that contains error correction information, as a percentage. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_ERROR_CORRECTION +	Retrieves the recommended error correction scheme (McodeControl() with M_ERROR_CORRECTION) for the trained code model(s). Since training the corresponding control type is not supported for the M_DOTCODE code type, this result type is not available for the M_DOTCODE code type after an McodeTrain() operation.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies that the error correction type is detected automatically.	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_4STATE	Specifies the Reed-Solomon-based algorithm or a check digit type of error correction scheme, depending on the specification of the encoding. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_200	Specifies a Reed-Solomon-based algorithm as an error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_CHECK_DIGIT	Specifies an additional digit to check whether there is an error or not. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_COMPOSITE	Specifies the default error correction scheme for the 1D and 2D portions of the composite code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_H	Specifies the highest-level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_L	Specifies the lowest-level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_M	Specifies a medium-low level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_NONE	Specifies no error correction. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_Q	Specifies a medium-high level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_REED_SOLOMON	Specifies a Reed-Solomon type of error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_REED_SOLOMON_*n*	Specifies a Reed-Solomon type of error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_UNKNOWN	Specifies an unknown error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
5 <= *Value* <= 95	Specifies the minimum percentage of the symbol that contains error correction information. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_VALUE +	Retrieves the recommended foreground color setting (McodeControl() with M_FOREGROUND_VALUE) for the trained code model(s).	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FOREGROUND_ANY	Specifies the foreground color as black or white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FOREGROUND_BLACK	Specifies that the foreground color is black.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FOREGROUND_WHITE	Specifies that the foreground color is white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE +		Retrieves the recommended setting for the nominal search angle (McodeControl() with M_SEARCH_ANGLE) for the trained code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the nominal angle, in degrees, relative to the input coordinate system specified using M_SEARCH_ANGLE_INPUT_UNITS. The default value is 0.0 degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_DELTA_NEG +		Retrieves the recommended setting for the negative angular range of the search (McodeControl() with M_SEARCH_ANGLE_DELTA_NEG) for the trained code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRAIN	Specifies that the corresponding control type was not trained since M_SEARCH_ANGLE was not selected for training; alternatively, it means that the corresponding control type does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
	5.0 <= *Value* <= 180.0	Specifies a negative angular range, in degrees, relative to the nominal angle set by M_SEARCH_ANGLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_DELTA_POS +		Retrieves the recommended setting for the positive angular range of the search (McodeControl() with M_SEARCH_ANGLE_DELTA_POS) for the trained code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRAIN	Specifies that the corresponding control type was not trained since M_SEARCH_ANGLE was not selected for training; alternatively, it means that the corresponding control type does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b

5.0 <= *Value* <= 180.0	Specifies a positive angular range, in degrees, relative to the nominal angle set by M_SEARCH_ANGLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_STEP +	Retrieves the recommended setting for the angle increment/decrement (McodeControl() with M_SEARCH_ANGLE_STEP) used when searching for PDF417 or TruncatedPDF417 codes through an angular range.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that no explicit increment/decrement is used. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.1 <= *Value* <= 180.0	Specifies the explicit angle increment/decrement, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_PRESEARCH +	Retrieves the recommended presearch setting (McodeControl() with M_USE_PRESEARCH) for the trained code model. The presearch setting determines whether the localization operation is performed prior to the decoding step of an operation. This result type is available for 2D matrix code types excluding the M_DOTCODE code type.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the operation is not performed.	c		e	f	g	a		i		j	k	l	m	n	b
M_FINDER_PATTERN_BASE	Specifies that the localization operation is only performed on the base pattern of the M_DATAMATRIX code (an "L" starting at the top-most left corner, and ending on the bottom-most right corner of the code). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_BASE	Specifies to localize the code within the image with the statistical characteristics of a 2D bar code (for example, local variance and the presence of a lot of edges). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN	Specifies that the corresponding control type was not selected for training or does not apply to the code model(s).	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve a code occurrence-specific result that is returned from an McodeDetect(), McodeRead(), McodeGrade(), or McodeWrite() operation, the ResultCodeId and ResultType parameters can be set to the following values. In this case, the ResultIndex parameter must be set to the index of a specific occurrence or M_ALL (or M_DEFAULT).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving code occurrence-specific results from an McodeDetect(), McodeRead(), McodeGrade(), or McodeWrite() operation																	
ResultCodeId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultType																	
UserVarArrayPtr - Possible values returned																	

Code detect result buffer ID for occurrence-specific results	Specifies a code detect result buffer, allocated using McodeAllocResult() with M_CODE_DETECT_RESULT, and used to store McodeDetect() results.													
M_BOTTOM_LEFT_X +	Retrieves the X-coordinate of the bottom-left corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTTOM_LEFT_Y +	Retrieves the Y-coordinate of the bottom-left corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTTOM_RIGHT_X +	Retrieves the X-coordinate of the bottom-right corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTTOM_RIGHT_Y +	Retrieves the Y-coordinate of the bottom-right corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_TYPE +	Retrieves the code type of the specified code occurrence(s). The code type is retrieved as a numeric that can be passed to McodeModel(). After an McodeDetect() operation, you can retrieve the code type's corresponding name using **McodeGetResult**() with M_CODE_TYPE_NAME.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_4_STATE	Specifies a 4-state code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_AZTEC	Specifies an Aztec code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_BC412	Specifies a BC412 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODABAR	Specifies a Codabar code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE39	Specifies a Code 39 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE93	Specifies a Code 93 code type.	c		e	f	g	a		i		j	k	l	m	n	b

		M_CODE128	Specifies a Code 128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COMPOSITECODE	Specifies a composite code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_DATAMATRIX	Specifies a Data Matrix code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_DOTCODE	Specifies a DotCode code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_EAN8	Specifies an EAN 8 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_EAN13	Specifies an EAN 13 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_EAN14	Specifies an EAN 14 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_GS1_128	Specifies a GS1-128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_GS1_DATABAR	Specifies a GS1 Databar code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IATA25	Specifies an IATA 2 of 5 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INDUSTRIAL25	Specifies an Industrial 2 of 5 (standard 2 of 5) code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INTERLEAVED25	Specifies an Interleaved 2 of 5 (ITF-14) code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_MAXICODE	Specifies a Maxicode code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_MICROPDF417	Specifies a MicroPDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_MICROQRCODE	Specifies a Micro QR code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_PDF417	Specifies a PDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_PHARMACODE	Specifies a Pharmacode code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PLANET	Specifies a Planet code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_POSTNET	Specifies a Postnet code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_QRCODE	Specifies a QR code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_TRUNCATED_PDF417	Specifies a Truncated PDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_UPC_A	Specifies a UPC-A code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_UPC_E	Specifies a UPC-E code type.	c		e	f	g	a		i		j	k	l	m	n	b

M_CODE_TYPE_NAME +	Retrieves the name of the code type of the specified code occurrence(s) detected. You can use this name, for example, to annotate the display of the image used with McodeDetect(). To retrieve the code type as a numeric that can be passed to McodeModel(), use M_CODE_TYPE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BC412_NAME	Specifies a BC412 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODABAR_NAME	Specifies a Codabar code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE39_NAME	Specifies a Code 39 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE93_NAME	Specifies a Code 93 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE128_NAME	Specifies a Code 128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_EAN8_NAME	Specifies an EAN 8 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_EAN13_NAME	Specifies an EAN 13 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_EAN14_NAME	Specifies an EAN 14 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_GS1_128_NAME	Specifies a GS1-128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_IATA25_NAME	Specifies an IATA 2 of 5 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_INDUSTRIAL25_NAME	Specifies an Industrial 2 of 5 (standard 2 of 5) code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERLEAVED25_NAME	Specifies an Interleaved 2 of 5 (ITF-14) code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_UPC_A_NAME	Specifies a UPC-A code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_UPC_E_NAME	Specifies a UPC-E code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENCODING +	Retrieves the type of encoding of the specified code occurrence(s). Note that since some versions of the GS1 Databar code types only differ by the bar height (and not the structure), the same result is returned for these similar code types. GS1 Databar Omnidirectional and GS1 Databar Truncated will return M_ENC_GS1_DATABAR_OMNI. GS1 Databar Stacked and GS1 Databar Stacked Omnidirectional will return M_ENC_GS1_DATABAR_STACKED. It is possible to obtain a more accurate result to distinguish between these structurally similar code types, using McodeControl() with M_POSITION_ACCURACY set to M_HIGH.	c		e	f	g	a		i		j	k	l	m	n	b

		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_ALPHA	Specifies an encoding scheme that supports uppercase alphabetical characters, along with the space. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_ALPHANUM	Specifies an encoding scheme that supports alphanumeric characters, as well as the space. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_ALPHANUM_PUNC	Specifies a similar encoding scheme to M_ENC_ALPHANUM, except it also supports the following characters: (,), (-), (/) and (. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_ASCII	Specifies an encoding scheme that supports ASCII characters. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AUSTRALIA_MAIL_C	Specifies an encoding scheme for a 4-state format used with the C encoding table by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AUSTRALIA_MAIL_N	Specifies an encoding scheme for a 4-state format used with the N encoding table by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AUSTRALIA_MAIL_RAW	Specifies an encoding scheme for a 4-state format used by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AZTEC_COMPACT	Specifies an encoding scheme for a compact Aztec code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AZTEC_FULL_RANGE	Specifies an encoding scheme for a full-range (not compact) Aztec code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AZTEC_RUNE	Specifies an encoding scheme for an Aztec rune (the smallest version of an Aztec code). INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN8	Specifies an encoding scheme for a composite code whose 1D portion uses an EAN 8 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN8_ADDON	Specifies an encoding scheme for an EAN 8 format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN13	Specifies an encoding scheme for a composite code whose 1D portion uses an EAN 13 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN13_ADDON	Specifies an encoding scheme for an EAN 13 format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b

		M_ENC_GS1_128_MICROPDF417	Specifies an encoding scheme for a composite code whose 1D portion uses a GS1 128 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_128_PDF417	Specifies an encoding scheme for a composite code whose 1D portion uses a GS1 128 format and whose 2D portion uses a PDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_EXPANDED	Specifies an encoding scheme that uses a GS1 Databar format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_EXPANDED_STACKED	Specifies an encoding scheme that uses a GS1 Databar Expanded Stacked format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_LIMITED	Specifies an encoding scheme that uses a GS1 Databar Limited format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_OMNI	Specifies an encoding scheme that uses a GS1 Databar format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_STACKED	Specifies an encoding scheme that uses a GS1 Databar Stacked format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_STACKED_OMNI	Specifies an encoding scheme that uses a GS1 Databar Stacked Omnidirectional format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_TRUNCATED	Specifies an encoding scheme that uses a GS1 Databar Truncated format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_ISO8	Specifies a similar encoding scheme as M_ENC_ASCII, but supports the extended ASCII character set. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_KOREA_MAIL	Specifies an encoding scheme for a 4-state format used by the Korean Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE2	Specifies an encoding scheme that requires a Structured Carrier Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE3	Specifies an encoding scheme that requires a Structured Carrier Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE4	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE5	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE6	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b

		M_ENC_NUM	Specifies an encoding scheme that only supports numbers. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_QRCODE_MODEL1	Specifies an encoding scheme that uses an older version of the QR code format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_QRCODE_MODEL2	Specifies an encoding scheme that uses a newer version of the QR code format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_STANDARD	Specifies different types of encoding schemes, depending on what code type is used. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UK_MAIL	Specifies an encoding scheme for a 4-state format used by the UK Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UPCA	Specifies an encoding scheme for a composite code whose 1D portion uses an UPC-A format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UPCA_ADDON	Specifies an encoding scheme for an UPC-A format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UPCE	Specifies an encoding scheme for a composite code whose 1D portion uses an UPC-E format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UPCE_ADDON	Specifies an encoding scheme for an UPC-E format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_US_MAIL	Specifies the Intelligent Mail Barcode encoding scheme for a 4-state format used by the US Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		5 <= *Value* <= 95	Specifies the minimum amount of the symbol that contains error correction information, as a percentage. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_TOP_LEFT_X +		Retrieves the X-coordinate of the top-left corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_TOP_LEFT_Y +		Retrieves the Y-coordinate of the top-left corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_TOP_RIGHT_X +	Retrieves the X-coordinate of the top-right corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TOP_RIGHT_Y +	Retrieves the Y-coordinate of the top-right corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Code grade result buffer ID for occurrence-specific results	Specifies a code grade result buffer, allocated using McodeAllocResult() with M_CODE_GRADE_RESULT, and used to store McodeGrade() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE +	Retrieves the angle, in degrees, of the specified code occurrence(s), relative to the output coordinate system specified using McodeControl() with M_RESULT_OUTPUT_UNITS. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_APERTURE_SIZE_USED +	Retrieves the aperture size used for the specified code occurrence(s). This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the aperture size used.	c		e	f	g	a		i		j	k	l	m	n	b

M_ASTERISK +	Retrieves whether the you should append an asterisk to the overall symbol grade, for the specified code occurrence(s). Typically, an asterisk is appended to the overall symbol grade when the region containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides) has a symbol contrast value (reflectance) that might interfere with the reading of the code. This is the case, for example, if the region is too bright or too dark. This result type requires that the specified code be graded using proper code verification techniques. For more information, refer to the ISO/IEC 15415 specification. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 2D matrix code types, except Maxicode.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that no asterisk needs to be appended to the overall symbol grade.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that an asterisk needs to be appended to the overall symbol grade.	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIAL_NONUNIFORMITY +	Retrieves the axial non-uniformity of the specified code occurrence(s). The axial non-uniformity is calculated using the following formula: $\frac{abs(X_{avg}-Y_{avg})}{0.5 \times (X_{avg}+Y_{avg})}$ Where X_{avg} is the average cell size in the X-dimension and Y_{avg} is the average measurement cell size in the Y-dimension. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is available for 2D matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the axial non-uniformity measure.	c		e	f	g	a		i		j	k	l	m	n	b

M_AXIAL_NONUNIFORMITY_GRADE +	Retrieves the axial non-uniformity of the specified code occurrence(s), as a grade. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...). This result type is available for 2D matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTTOM_LEFT_X +	Retrieves the X-coordinate of the bottom-left corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTTOM_LEFT_Y +	Retrieves the Y-coordinate of the bottom-left corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTTOM_RIGHT_X +	Retrieves the X-coordinate of the bottom-right corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_BOTTOM_RIGHT_Y +	Retrieves the Y-coordinate of the bottom-right corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_CONTRAST +	Retrieves the cell contrast value (CC) of the specified code occurrence(s). The cell contrast value is calculated as the difference between the average intensity of lighter cells minus the average intensity of darker cells, divided by the average intensity of lighter cells. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_DPM_GRADING before McodeGrade() is called. This result type is available for Aztec, Data Matrix, QR code, and Micro QR code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the cell contrast value.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_CONTRAST_GRADE +	Retrieves the cell contrast value of the specified code occurrence(s), as a grade. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_DPM_GRADING before McodeGrade() is called. This result type is available for Aztec, Data Matrix, QR code, and Micro QR code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b

		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CELL_DEFECTS +		Retrieves the cell defects value of the specified code occurrence(s). The cell defects value is the ratio between the number of incorrect pixels and the total number of pixels that fall within the bounds of the Data Matrix grid, expressed as a percentage. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_COMPUTABLE	Specifies that the result was not computable for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 100.0	Specifies the cell defects value, expressed as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CELL_HEIGHT +		Retrieves the Data Matrix cell height (DMCH) of the specified code occurrence(s). This is the average height of the cells in the Data Matrix. The Data Matrix cell height is calculated from the 4 Data Matrix corner points and the number of rows. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		Value > 0.0	Specifies the cell height.	c		e	f	g	a		i		j	k	l	m	n	b

M_CELL_MODULATION_GRADE +		Retrieves the modulation of the specified code occurrence(s), as a grade calculated using the ISO/IEC 29158 standard. Modulation is a measure of the uniformity of reflectance in the dark and light areas of the code. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...). To retrieve the modulation of a code occurrence calculated using the ISO/IEC 15415 standard, use M_MODULATION_GRADE and M_REFLECTANCE_MARGIN_GRADE. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_DPM_GRADING before McodeGrade() is called. This result type is available for Aztec, Data Matrix, QR code, and Micro QR code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_NUMBER_X +		Retrieves the number of cells in the X-direction of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the number of cells in the X-direction.	c		e	f	g	a		i		j	k	l	m	n	b

M_CELL_NUMBER_Y +	Retrieves the number of cells in the Y-direction of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of cells in the Y-direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE +	Retrieves the size of the cell in X (module size (Z)) of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE_MAX +	Retrieves the maximum cell size of the code occurrence. For 2D matrix code types, this corresponds to the nominal maximum module size, whereas for other code types, this corresponds to the same result as M_CELL_SIZE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum cell size, relative to the input coordinate system specified using M_CELL_SIZE_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE_MIN +	Retrieves the minimum cell size of the code occurrence. For 2D matrix code types, this corresponds to the nominal minimum module size, whereas for other code types, this corresponds to the same result as M_CELL_SIZE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the minimum cell size, relative to the input coordinate system specified using M_CELL_SIZE_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_CELL_WIDTH +	Retrieves the Data Matrix cell width (DMCW) of the specified code occurrence(s). This is the average width of the cells in the Data Matrix. The Data Matrix cell width is calculated from the 4 Data Matrix corner points and the number of columns. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the cell width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_FLIP +	Retrieves for the specified code occurrence(s) whether it was necessary to flip it/read it in the opposite direction.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FLIP	Specifies that the code occurrence was reversed.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_FLIP	Specifies that the code occurrence was not reversed.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_MODEL_ID +	Retrieves the identifier of the code model of the code occurrence. Note that this result is not valid when the result buffer is on a remote system. To find out which code model was used when the result buffer is on a remote system, use M_CODE_MODEL_INDEX instead.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_MODEL_INDEX +	Retrieves the index of the code model of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_TYPE +	Retrieves the code type of the specified code occurrence(s). The code type is retrieved as a numeric that can be passed to McodeModel().	c		e	f	g	a		i		j	k	l	m	n	b

		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_4_STATE	Specifies a 4-state code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_AZTEC	Specifies an Aztec code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_BC412	Specifies a BC412 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODABAR	Specifies a Codabar code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE39	Specifies a Code 39 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE93	Specifies a Code 93 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE128	Specifies a Code 128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COMPOSITECODE	Specifies a composite code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_DATAMATRIX	Specifies a Data Matrix code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_DOTCODE	Specifies a DotCode code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_EAN8	Specifies an EAN 8 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_EAN13	Specifies an EAN 13 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_EAN14	Specifies an EAN 14 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_GS1_128	Specifies a GS1-128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_GS1_DATABAR	Specifies a GS1 Databar code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IATA25	Specifies an IATA 2 of 5 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INDUSTRIAL25	Specifies an Industrial 2 of 5 (standard 2 of 5) code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INTERLEAVED25	Specifies an Interleaved 2 of 5 (ITF-14) code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_MAXICODE	Specifies a Maxicode code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_MICROPDF417	Specifies a MicroPDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_MICROQRCODE	Specifies a Micro QR code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_PDF417	Specifies a PDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_PHARMACODE	Specifies a Pharmacode code type.	c		e	f	g	a		i		j	k	l	m	n	b

		M_PLANET	Specifies a Planet code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_POSTNET	Specifies a Postnet code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_QRCODE	Specifies a QR code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_TRUNCATED_PDF417	Specifies a Truncated PDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_UPC_A	Specifies a UPC-A code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_UPC_E	Specifies a UPC-E code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODEWORD_DECODABILITY +		Retrieves a measure of the print quality of the codeword relative to the decoding algorithm, for the specified code occurrence(s). Note that this excludes any start/stop patterns. To retrieve the decodability of the start/stop patterns of a code occurrence, use M_SCAN_DECODABILITY. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is used in the grade overlay procedure for measuring codeword print quality, as described in the ISO/IEC 15415 specification. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. The following result types are only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 2D cross-row and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 1.0	Specifies the measure of the print quality of each codeword relative to the decoding algorithm.	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODEWORD_DECODABILITY_GRADE +	Retrieves the decodability of the codeword in the specified code occurrence(s), as a grade. Note that this excludes any start/stop patterns. To retrieve the decodability of the start/stop patterns of a code occurrence as a grade, use M_SCAN_DECODABILITY_GRADE. This result type is used in the grade overlay procedure for measuring codeword print quality, as described in the ISO/IEC 15415 specification. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 2D cross-row and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

M_CODEWORD_DEFECTS +	Retrieves a measure of the defects of the codeword in the specified code occurrence(s), as a ratio between the deviations in the expected signal. The larger the result, the greater the defects of the codeword and the less likely that the codeword can be decoded without error. Note that this excludes any start/stop patterns. To retrieve a measure of the defects of the specified code occurrence(s) as a single grade, use M_DEFECTS_GRADE. To retrieve the defects of the start/stop patterns, use M_SCAN_DEFECTS. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is used in the grade overlay procedure for measuring codeword print quality, as described in the ISO/IEC 15415 specification. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 2D cross-row and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the deviation in the expected signal that denotes a codeword in your code occurrence.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODEWORD_DEFECTS_GRADE +	Retrieves a measure of the defects of the codeword in the specified code occurrence(s), as a grade. Note that this excludes any start/stop patterns. To retrieve a measure of the defects of the specified code occurrence(s) as a single grade, use M_DEFECTS_GRADE. To retrieve the defects of the start/stop patterns, use M_SCAN_DEFECTS_GRADE. This result type is used in the grade overlay procedure for measuring codeword print quality, as described in the ISO/IEC 15415 specification. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 2D cross-row and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_CODEWORD_MODULATION +		Retrieves the modulation (MOD) of the codeword in the specified code occurrence(s). Modulation is a measure of the uniformity of reflectance in the dark and light areas of the code. For 2D cross-row and composite code types, modulation is calculated as the ratio of the minimum edge contrast value (EC_{min}) to symbol contrast value (M_SYMBOL_CONTRAST) within the code. To retrieve the modulation of the start/stop patterns of 2D cross-row and composite code types, use M_SCAN_MODULATION. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is used in the grade overlay procedure for measuring codeword print quality, as described in the ISO/IEC 15415 specification. This result type is available for Aztec, Data Matrix, DotCode, QR code, Micro QR, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the modulation.	c		e	f	g	a		i		j	k	l	m	n	b

M_CODEWORD_MODULATION_GRADE +	Retrieves the modulation (MOD) of the codeword in the specified code occurrence(s), as a grade. To retrieve the modulation of the specified code occurrence(s) as a single grade, use M_MODULATION_GRADE. To retrieve the modulation of the start/stop patterns of 2D cross-row and composite code types, as a grade, use M_SCAN_MODULATION_GRADE. This result type is used in the grade overlay procedure for measuring codeword print quality, as described in the ISO/IEC 15415 specification. This result type is available for Aztec, Data Matrix, DotCode, QR code, Micro QR, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_CODEWORD_REFLECTANCE_MARGIN +	Retrieves the reflectance margin of the codeword in the specified code occurrence(s). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 2D matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the reflectance margin.	c		e	f	g	a		i		j	k	l	m	n	b

M_CODEWORD_REFLECTANCE_MARGIN_GRADE +	Retrieves the reflectance margin of the codeword in the specified code occurrence(s), as a grade. To retrieve the modulation of the specified code occurrence(s) as a single grade, use M_REFLECTANCE_MARGIN_GRADE. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 2D matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_CODEWORD_YIELD +	Retrieves the codeword yield result of the specified code occurrence(s). The codeword yield result determines how well the code can be read at an angle relative to the horizontal and vertical axis of the code. When all other results are good, a poor codeword yield result can indicate a problem along the Y-axis of your code. For more information, refer to the ISO/IEC 15415 specification. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 2D cross-row and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the codeword yield.	c		e	f	g	a		i		j	k	l	m	n	b

M_CODEWORD_YIELD_GRADE +	Retrieves the codeword yield, as a single grade for the specified code occurrence(s). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 2D cross-row and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTRAST_UNIFORMITY +	Retrieves the contrast uniformity of the specified code occurrence(s). The contrast uniformity is the minimum (MOD) value for the code. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the contrast uniformity.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTRAST_UNIFORMITY_GRADE +	Retrieves the contrast uniformity of the specified code occurrence(s), as a grade. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_CORNER_P1_X +		Retrieves the X-coordinate of the first corner (top-left) of the specified code occurrence(s). This result type returns the same value as M_TOP_LEFT_X. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CORNER_P1_Y +		Retrieves the Y-coordinate of the first corner (top-left) of the specified code occurrence(s). This result type returns the same value as M_TOP_LEFT_Y. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CORNER_P2_X +		Retrieves the X-coordinate of the second corner (bottom-left) of the specified code occurrence(s). This result type returns the same value as M_BOTTOM_LEFT_X. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_CORNER_P2_Y +	Retrieves the Y-coordinate of the second corner (bottom-left) of the specified code occurrence(s). This result type returns the same value as M_BOTTOM_LEFT_Y. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CORNER_P3_X +	Retrieves the X-coordinate of the third corner (bottom-right) of the specified code occurrence(s). This result type returns the same value as M_BOTTOM_RIGHT_X. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CORNER_P3_Y +	Retrieves the Y-coordinate of the third corner (bottom-right) of the specified code occurrence(s). This result type returns the same value as M_BOTTOM_RIGHT_Y. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CORNER_P4_X +	Retrieves the X-coordinate of the fourth corner (top-right) of the specified code occurrence(s). This result type returns the same value as M_TOP_RIGHT_X. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_CORNER_P4_Y +	Retrieves the Y-coordinate of the fourth corner (top-right) of the specified code occurrence(s). This result type returns the same value as M_TOP_RIGHT_Y. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DATA_CODEWORDS +	Retrieves the data codewords from the graphical representation of the specified code occurrence(s). Note that each codeword is returned as a numerical value, rather than as one or more alpha-numeric characters. The returned data codewords could be used to validate the code, as with PDF417, or to allow interpretation of the data codewords using a character set other than the default set supported by Aurora Imaging Library. For more information, refer to the Extended Channel Interpretation (ECI) protocol in ISO/IEC 15438:2006. To retrieve the decoded characters from the specified code occurrence(s), use M_STRING. This result type is available for 2D and composite code types, with the exception of Data Matrix code types that do not use M_ERROR_CORRECTION with M_ECC_200.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DECODABILITY_GRADE +	Retrieves a measure of the print quality of the specified code occurrence(s) relative to the decoding algorithm, as a grade. To determine the decodability for each codeword in the specified code occurrence(s), use M_CODEWORD_DECODABILITY. To retrieve the decodability of the start/stop patterns of the specified code occurrence(s) as a grade, use M_SCAN_DECODABILITY_GRADE. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 2D cross-row and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_DECODE_GRADE +		Retrieves the code decodability of the specified code occurrence(s), as a grade. This result type is available for 2D matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies that the code can be read.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies that the code cannot be read.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_DECODING_MASK_SCORE +		Retrieves the score of each decoded mask of the specified code occurrence(s). This result type is available for the M_DOTCODE code type.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the score of each decoded mask of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_DECODING_MASK_SCORE_THRESHOLD +		Retrieves the threshold score of each decoded mask of the specified code occurrence(s). This result type is available for the M_DOTCODE code type.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the threshold score of each decoded mask of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_DECODING_MASK_TYPE +		Retrieves the decoding mask type of the specified code occurrence(s). This result type is available for the M_DOTCODE code type.	c		e	f	g	a		i		j	k	l	m	n	b

		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_INVALID	Specifies that no mask was applied to the code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		M_MASK_0	Specifies that the mask used for decoding is 0.	c		e	f	g	a		i		j	k	l	m	n	b
		M_MASK_0_PRIME	Specifies that the mask used for decoding is 0 prime (0').	c		e	f	g	a		i		j	k	l	m	n	b
		M_MASK_1	Specifies that the mask used for decoding is 1.	c		e	f	g	a		i		j	k	l	m	n	b
		M_MASK_1_PRIME	Specifies that the mask used for decoding is 1 prime (1').	c		e	f	g	a		i		j	k	l	m	n	b
		M_MASK_2	Specifies that the mask used for decoding is 2.	c		e	f	g	a		i		j	k	l	m	n	b
		M_MASK_2_PRIME	Specifies that the mask used for decoding is 2 prime (2').	c		e	f	g	a		i		j	k	l	m	n	b
		M_MASK_3	Specifies that the mask used for decoding is 3.	c		e	f	g	a		i		j	k	l	m	n	b
		M_MASK_3_PRIME	Specifies that the mask used for decoding is 3 prime (3').	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFECTS_GRADE +		Retrieves a measure of the defects in the specified code occurrence(s), as a grade. To retrieve the measure of defects for each codeword as a grade, use M_CODEWORD_DEFECTS_GRADE. To retrieve the measure of defects in the start/stop patterns as a grade, use M_SCAN_DEFECTS_GRADE. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 2D cross-row and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SPACING_USED +		Retrieves the value between M_DOT_SPACING_MIN and M_DOT_SPACING_MAX used to decode the specified code occurrence(s). The expected dot spacing is set using McodeControl() with M_DOT_SPACING_MIN and M_DOT_SPACING_MAX. This result type is available for Aztec, Data Matrix, Maxicode, QR code, and Micro QR code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ELEMENT_NUMBER_X +		Retrieves the number of elements in the X-direction. For 1D code types, this result retrieves the number of bars and spaces. For GS1 Databar Stacked code types and composite code types whose 1D portion uses a GS1 Databar Stacked code type, this result retrieves the number of bars and spaces in all rows. For composite code types, this result retrieves the number of bars and spaces in the 1D portion of the code. For 2D cross-row and 2D matrix code types, this result retrieves the same value as M_CELL_NUMBER_X.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ELEMENT_NUMBER_Y +		Retrieves the number of elements in the Y-direction. For composite code types, this result retrieves the number of rows in the 1D portion of the code. For 2D matrix code types, this result retrieves the number of cells. For 1D and 2D cross-row code types, this result retrieves the same value as M_CELL_NUMBER_Y. Note that for 1D code types, excluding GS1 Databar, this result will always return 1.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENCODING +	Retrieves the type of encoding of the specified code occurrence(s). Note that since some versions of the GS1 Databar code types only differ by the bar height (and not the structure), the same result is returned for these similar code types. GS1 Databar Omnidirectional and GS1 Databar Truncated will return M_ENC_GS1_DATABAR_OMNI. GS1 Databar Stacked and GS1 Databar Stacked Omnidirectional will return M_ENC_GS1_DATABAR_STACKED. It is possible to obtain a more accurate result to distinguish between these structurally similar code types, using McodeControl() with M_POSITION_ACCURACY set to M_HIGH.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_ALPHA	Specifies an encoding scheme that supports uppercase alphabetical characters, along with the space. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_ALPHANUM	Specifies an encoding scheme that supports alphanumeric characters, as well as the space. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_ALPHANUM_PUNC	Specifies a similar encoding scheme to M_ENC_ALPHANUM, except it also supports the following characters: (,), (-), (/) and (. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_ASCII	Specifies an encoding scheme that supports ASCII characters. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AUSTRALIA_MAIL_C	Specifies an encoding scheme for a 4-state format used with the C encoding table by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AUSTRALIA_MAIL_N	Specifies an encoding scheme for a 4-state format used with the N encoding table by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AUSTRALIA_MAIL_RAW	Specifies an encoding scheme for a 4-state format used by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AZTEC_COMPACT	Specifies an encoding scheme for a compact Aztec code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AZTEC_FULL_RANGE	Specifies an encoding scheme for a full-range (not compact) Aztec code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AZTEC_RUNE	Specifies an encoding scheme for an Aztec rune (the smallest version of an Aztec code). INFO	c		e	f	g	a		i		j	k	l	m	n	b

		M_ENC_EAN8	Specifies an encoding scheme for a composite code whose 1D portion uses an EAN 8 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN8_ADDON	Specifies an encoding scheme for an EAN 8 format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN13	Specifies an encoding scheme for a composite code whose 1D portion uses an EAN 13 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN13_ADDON	Specifies an encoding scheme for an EAN 13 format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_128_MICROPDF417	Specifies an encoding scheme for a composite code whose 1D portion uses a GS1 128 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_128_PDF417	Specifies an encoding scheme for a composite code whose 1D portion uses a GS1 128 format and whose 2D portion uses a PDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_EXPANDED	Specifies an encoding scheme that uses a GS1 Databar format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_EXPANDED_STACKED	Specifies an encoding scheme that uses a GS1 Databar Expanded Stacked format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_LIMITED	Specifies an encoding scheme that uses a GS1 Databar Limited format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_OMNI	Specifies an encoding scheme that uses a GS1 Databar format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_STACKED	Specifies an encoding scheme that uses a GS1 Databar Stacked format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_STACKED_OMNI	Specifies an encoding scheme that uses a GS1 Databar Stacked Omnidirectional format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_TRUNCATED	Specifies an encoding scheme that uses a GS1 Databar Truncated format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_ISO8	Specifies a similar encoding scheme as M_ENC_ASCII, but supports the extended ASCII character set. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_KOREA_MAIL	Specifies an encoding scheme for a 4-state format used by the Korean Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b

		M_ENC_MODE2	Specifies an encoding scheme that requires a Structured Carrier Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE3	Specifies an encoding scheme that requires a Structured Carrier Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE4	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE5	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE6	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_NUM	Specifies an encoding scheme that only supports numbers. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_QRCODE_MODEL1	Specifies an encoding scheme that uses an older version of the QR code format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_QRCODE_MODEL2	Specifies an encoding scheme that uses a newer version of the QR code format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_STANDARD	Specifies different types of encoding schemes, depending on what code type is used. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UK_MAIL	Specifies an encoding scheme for a 4-state format used by the UK Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UPCA	Specifies an encoding scheme for a composite code whose 1D portion uses an UPC-A format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UPCA_ADDON	Specifies an encoding scheme for an UPC-A format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UPCE	Specifies an encoding scheme for a composite code whose 1D portion uses an UPC-E format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UPCE_ADDON	Specifies an encoding scheme for an UPC-E format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_US_MAIL	Specifies the Intelligent Mail Barcode encoding scheme for a 4-state format used by the US Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		5 <= *Value* <= 95	Specifies the minimum amount of the symbol that contains error correction information, as a percentage. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_ERROR_CORRECTION +	Retrieves the type of error correction scheme of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_4STATE	Specifies the Reed-Solomon-based algorithm or a check digit type of error correction scheme, depending on the specification of the encoding. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_200	Specifies a Reed-Solomon-based algorithm as an error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_CHECK_DIGIT	Specifies an additional digit to check whether there is an error or not. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_COMPOSITE	Specifies the default error correction scheme for the 1D and 2D portions of the composite code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_H	Specifies the highest-level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_L	Specifies the lowest-level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_M	Specifies a medium-low level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_NONE	Specifies no error correction. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_Q	Specifies a medium-high level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_REED_SOLOMON	Specifies a Reed-Solomon type of error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_REED_SOLOMON_*n*	Specifies a Reed-Solomon type of error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_UNKNOWN	Specifies an unknown error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b
5 <= *Value* <= 95	Specifies the minimum percentage of the symbol that contains error correction information. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_EXTENDED_AREA_BOTTOM_LEFT_X +	Retrieves the X-coordinate of the bottom-left corner of the specified code occurrence(s), including the extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_BOTTOM_LEFT_Y +	Retrieves the Y-coordinate of the bottom-left corner of the specified code occurrence(s), including the extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_EXTENDED_AREA_BOTTOM_RIGHT_X +	Retrieves the X-coordinate of the bottom-right corner of the specified code occurrence(s), including the extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_BOTTOM_RIGHT_Y +	Retrieves the Y-coordinate of the bottom-right corner of the specified code occurrence(s), including the extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_EXTENDED_AREA_CODEWORD_MODULATION +	Retrieves the modulation (MOD) of the codeword in the specified code occurrence(s) of any 2D matrix code type, except Maxicode, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type requires that the specified code be graded using proper code verification techniques. For more information, refer to the ISO/IEC 15415 specification. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_CODEWORD_MODULATION_GRADE +	Retrieves the modulation (MOD) of the codeword in the specified code occurrence(s) of any 2D matrix code type, except Maxicode, as a grade, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type requires that the specified code be graded using proper code verification techniques. For more information, refer to the ISO/IEC 15415 specification. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_A1_GRADE +	Retrieves the fixed pattern damage in the A1 segment of the specified code occurrence(s) of a QR code or Micro QR code type, as a grade, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_A2_GRADE +	Retrieves the fixed pattern damage in the A2 segment of the specified code occurrence(s) of a QR code type, as a grade, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_A3_GRADE +		Retrieves the fixed pattern damage in the A3 segment of the specified code occurrence(s) of a QR code type, as a grade, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_A_GRADE +		Retrieves the fixed pattern damage in the A segment of the specified code occurrence(s) of an Aztec code type, as a grade, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_AVERAGE_GRADE +		Retrieves the average grade of the fixed pattern damage (FDP) in the specified code occurrence(s) of a Data Matrix code type, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_B1_GRADE +	Retrieves the fixed pattern damage in the B1 segment of the specified code occurrence(s) of a QR code or Micro QR code type, as a grade, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_B2_GRADE +	Retrieves the fixed pattern damage in the B2 segment of the specified code occurrence(s) of a QR code or Micro QR code type, as a grade, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_B_GRADE +		Retrieves the fixed pattern damage in each B segment of the specified code occurrence(s) of an Aztec code type with a full-range encoding scheme (M_ENC_AZTEC_FULL_RANGE), as a grade; the greatest and lowest reflectance are measured over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_C_GRADE +		Retrieves the fixed pattern damage in the C segment of the specified code occurrence(s) of a QR code type, as a grade, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_CLOCKTRACK_SOLID_GRADE +		Retrieves the fixed pattern damage in the clock pattern and adjacent solid area segments of the specified code occurrence(s) of a Data Matrix code type, as a grade; the greatest and lowest reflectance are measured over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_GRADE +	Retrieves the fixed pattern damage in the specified code occurrence(s) of an Aztec, Data Matrix, DotCode, QR, or Micro QR code type, as a grade, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_INTERSTITIAL_DOTS_GRADE +	Retrieves the fixed pattern damage of the interstitial dot positions, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is available for the M_DOTCODE code type. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b

		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_L1_GRADE +		Retrieves the fixed pattern damage in the L1 segment of the specified code occurrence(s) of a Data Matrix code type, as a grade, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_L2_GRADE +		Retrieves the fixed pattern damage in the L2 segment of the specified code occurrence(s) of a Data Matrix code type, as a grade, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_QUIET_ZONES_GRADE +		Retrieves the fixed pattern damage of the 3-dot wide quiet zones on all four sides, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is available for the M_DOTCODE code type. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_QZL1_GRADE +		Retrieves the fixed pattern damage in the QZL1 segment of the specified code occurrence(s) of a Data Matrix code type, as a grade, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_FIXED_PATTERN_DAMAGE_QZL2_GRADE +		Retrieves the fixed pattern damage in the QZL2 segment of the specified code occurrence(s) of a Data Matrix code type, as a grade, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b

		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	M_EXTENDED_AREA_MODULATION_GRADE +		Retrieves the modulation (MOD) of the specified code occurrence(s) of any 2D matrix code type, except Maxicode, as a grade, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

M_EXTENDED_AREA_QUIET_ZONE_INCLUDED +	Retrieves whether the quiet zone and the extended area (that is, 20 times the cell size beyond the quiet zone on all sides) were included in the source image of the code operation, for the specified code occurrence(s). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the quiet zone and extended area were not included.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the quiet zone and extended area were included. This result type requires that the specified code be graded using proper code verification techniques. For more information, refer to the ISO/IEC 15415 specification.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_REFLECTANCE_MAXIMUM +	Retrieves the highest reflectance (Rmax) of the specified code occurrence(s) of any 2D matrix code type, except Maxicode, over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type requires that the specified code be graded using proper code verification techniques. For more information, refer to the ISO/IEC 15415 specification. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0 <= *Value* <= 255	Specifies the greatest reflectance value within the extended area.	c		e	f	g	a		i		j	k	l	m	n	b
	M_EXTENDED_AREA_REFLECTANCE_MINIMUM +		Retrieves the lowest reflectance (Rmin) of the specified code occurrence(s) of any 2D matrix code type, except Maxicode, over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type requires that the specified code be graded using proper code verification techniques. For more information, refer to the ISO/IEC 15415 specification. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0 <= *Value* <= 255	Specifies the lowest reflectance value within the extended area.	c		e	f	g	a		i		j	k	l	m	n	b
	M_EXTENDED_AREA_SYMBOL_CONTRAST +		Retrieves the symbol contrast value (SC) of the specified code occurrence(s), after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 1.0	Specifies the symbol contrast value.	c		e	f	g	a		i		j	k	l	m	n	b

M_EXTENDED_AREA_SYMBOL_CONTRAST_GRADE +	Retrieves the symbol contrast value (SC) of the specified code occurrence(s), as a grade, after measuring the greatest and lowest reflectance over an area containing the code and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_TOP_LEFT_X +	Retrieves the X-coordinate of the top-left corner of the specified code occurrence(s), including the extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

	M_EXTENDED_AREA_TOP_LEFT_Y +	Retrieves the Y-coordinate of the top-left corner of the specified code occurrence(s), including the extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_EXTENDED_AREA_TOP_RIGHT_X +	Retrieves the X-coordinate of the top-right corner of the specified code occurrence(s), including the extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_EXTENDED_AREA_TOP_RIGHT_Y +	Retrieves the Y-coordinate of the top-right corner of the specified code occurrence(s), including the extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FINDER_PATTERN_DEFECTS +	Retrieves the finder pattern defects value of the specified code occurrence(s). The finder pattern defects value is the ratio between the number of incorrect pixels and the total number of pixels that fall within the bounds of the L finder pattern of the Data Matrix grid, expressed as a percentage. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_COMPUTABLE	Specifies that the result was not computable for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the finder pattern defects value, expressed as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b

M_FIXED_PATTERN_DAMAGE_A1_GRADE +	Retrieves the fixed pattern damage in the A1 segment for the specified code occurrence(s) of a QR code or Micro QR code type, as a grade. The fixed pattern damage (FPD) is a measure of the amount of damage present in the finder pattern, clock pattern, and quiet zone of the code. For a QR code, the alignment pattern is also analyzed. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIXED_PATTERN_DAMAGE_A2_GRADE +	Retrieves the fixed pattern damage in the A2 segment for the specified code occurrence(s) of a QR code type as a grade. The fixed pattern damage (FPD) is a measure of the amount of damage present in the finder pattern, clock pattern, and quiet zone of the code. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIXED_PATTERN_DAMAGE_A3_GRADE +		Retrieves the fixed pattern damage in the A3 segment for the specified code occurrence(s) of a QR code type, as a grade. The fixed pattern damage (FPD) is a measure of the amount of damage present in the finder pattern, clock pattern, and quiet zone of the code. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...).	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b

M_FIXED_PATTERN_DAMAGE_A_GRADE +	Retrieves the fixed pattern damage in the A segment for the specified code occurrence(s) of an Aztec code type, as a grade. The fixed pattern damage (FPD) is a measure of the amount of damage present in the finder pattern and orientation patterns of the code. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIXED_PATTERN_DAMAGE_AVERAGE_GRADE +	Retrieves the average grade of the fixed pattern damage (FDP) in the specified code occurrence(s) of a Data Matrix code type. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b

		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIXED_PATTERN_DAMAGE_B1_GRADE +		Retrieves the fixed pattern damage in the B1 segment for the specified code occurrence(s) of a QR code or Micro QR code type, as a grade. The fixed pattern damage (FPD) is a measure of the amount of damage present in the finder pattern, clock pattern, and quiet zone of the code. For a QR code, the alignment pattern is also analyzed. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...).	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b

M_FIXED_PATTERN_DAMAGE_B2_GRADE +	Retrieves the fixed pattern damage in the B2 segment for the specified code occurrence(s) of a QR code or Micro QR code type, as a grade. The fixed pattern damage (FPD) is a measure of the amount of damage present in the finder pattern, clock pattern, and quiet zone of the code. For a QR code, the alignment pattern is also analyzed. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIXED_PATTERN_DAMAGE_B_GRADE +	Retrieves the fixed pattern damage in each B segment of the specified code occurrence(s) of an Aztec code type with a full-range encoding scheme (M_ENC_AZTEC_FULL_RANGE), as a grade. The fixed pattern damage (FPD) is a measure of the amount of damage present in the finder pattern, and orientation patterns of the code. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIXED_PATTERN_DAMAGE_C_GRADE +		Retrieves the fixed pattern damage in the C segment for the specified code occurrence(s) of a QR code type, as a grade. The fixed pattern damage (FPD) is a measure of the amount of damage present in the finder pattern, clock pattern, and quiet zone of the code. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...).	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b

M_FIXED_PATTERN_DAMAGE_CLOCKTRACK_SOLID_GRADE +		Retrieves the grade of the clock pattern and adjacent solid area segments for the specified code occurrence(s) of a Data Matrix code type. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIXED_PATTERN_DAMAGE_GRADE +		Retrieves the fixed pattern damage for the specified code occurrence(s), as a grade. The fixed pattern damage (FPD) is a measure of the amount of damage present in the finder pattern, clock pattern, and quiet zone of the code. For a QR code, the alignment pattern is also analyzed. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...). This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIXED_PATTERN_DAMAGE_INTERSTITIAL_DOTS_GRADE +		Retrieves the fixed pattern damage grade of the interstitial dot positions of the code occurrence(s). This result type is available for the M_DOTCODE code type.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_FIXED_PATTERN_DAMAGE_L1_GRADE +		Retrieves the grade of the L1 segment for the specified code occurrence(s) of a Data Matrix code type. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIXED_PATTERN_DAMAGE_L2_GRADE +		Retrieves the grade of the L2 segment for the specified code occurrence(s) of a Data Matrix code type. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIXED_PATTERN_DAMAGE_QUIET_ZONES_GRADE +		Retrieves the fixed pattern damage grade of the 3-dot wide quiet zones on all four sides of code occurrence. This result type is available for the M_DOTCODE code type.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIXED_PATTERN_DAMAGE_QZL1_GRADE +		Retrieves the grade of the QZL1 segment for the specified code occurrence(s) of a Data Matrix code type. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...).	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b

M_FIXED_PATTERN_DAMAGE_QZL2_GRADE +	Retrieves the grade of the QZL2 segment for the specified code occurrence(s) of a Data Matrix code type. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_VALUE +	Retrieves the foreground color of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_BLACK	Specifies that the foreground color is black.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_WHITE	Specifies that the foreground color is white.	c		e	f	g	a		i		j	k	l	m	n	b

M_FORMAT_INFORMATION_1_GRADE +	Retrieves the readability of the format information in segment 1 of the specified code occurrence(s), as a grade. The format information is an embedded pattern that contains information about the characteristics of the code that are essential for further decoding. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...). This result type is available for the QR code type.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
M_FORMAT_INFORMATION_2_GRADE +	Retrieves the readability of the format information in segment 2 of the specified code occurrence(s), as a grade. The format information is an embedded pattern that contains information about the characteristics of the code that are essential for further decoding. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...). This result type is available for the QR code type.	c		e	f	g	a		i		j	k	l	m	n	b

		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FORMAT_INFORMATION_GRADE +		Retrieves the readability of the format information of the specified code occurrence(s), as a grade. The format information is an embedded pattern that contains information about the characteristics of the code that are essential for further decoding. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...). This result type is available for QR code and Micro QR code types.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

	0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRADING_STANDARD_EDITION_USED +		Retrieves the grading standard edition used to grade the code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ISO_15415_2011_15416_2000	Specifies that the ISO/IEC 15415:2011 and ISO/IEC 15416:2000 specifications are used. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ISO_15415_2011_15416_2016	Specifies that the ISO/IEC 15415:2011 and ISO/IEC 15416:2016 specifications are used. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ISO_15416_2000	Specifies that the ISO/IEC 15416:2000 specification is used. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ISO_15416_2016	Specifies that the ISO/IEC 15416:2016 specification is used. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ISO_29158_2011	Specifies that the ISO/IEC 29158:2011 specification is used. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ISO_29158_2020	Specifies that the ISO/IEC 29158:2020 specification is used. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SEMI_T10_0701	Specifies that the Semi T10-0701 specification is used. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GRID_NONUNIFORMITY +		Retrieves the grid non-uniformity value of the specified code occurrence(s). Grid non-uniformity is calculated as the amount of deviation of the code's actual grid of an ideal grid. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the grid non-uniformity value.	c		e	f	g	a		i		j	k	l	m	n	b

M_GRID_NONUNIFORMITY_GRADE +	Retrieves the grid non-uniformity of the specified code occurrence(s), as a grade. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...). This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
M_HORIZONTAL_MARK_GROWTH +	Retrieves the horizontal mark growth (HMG) value of the specified code occurrence(s). The horizontal mark growth value is the ratio between the median mark cell width (MCW) and the sum of this width and the median space cell width (SCW) in the horizontal alternating pattern, expressed as a percentage. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODE_GRADE_NOT_COMPUTABLE	Specifies that the result was not computable for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the horizontal mark growth value, expressed as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_HORIZONTAL_MARK_MISPLACEMENT +		Retrieves the horizontal mark misplacement (HMM) value of the specified code occurrence(s). The horizontal mark misplacement value is the average horizontal displacement of the marks' centers from the their ideal cell center point, expressed as a percentage. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_COMPUTABLE	Specifies that the result was not computable for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the horizontal mark misplacement value, expressed as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_IS_ECI +		Retrieves whether the decoded string of the specified code occurrence(s) contains character set ECIs. This result type is available for Aztec, Data Matrix, DotCode, Maxicode, Micro PDF417, PDF417, QR code, and truncated PDF417 code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the decoded string does not contains character set ECIs.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the decoded string contains character set ECIs.	c		e	f	g	a		i		j	k	l	m	n	b
M_IS_GS1 +		Retrieves whether the specified code occurrence(s) follows the industry standard for a GS1 code. This result type is available for Aztec, Data Matrix, Code 128, EAN 14, GS1-128, GS1-Databar, QR code, DotCode, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the specified code occurrence(s) does not follow the industry standard for a GS1 code.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the specified code occurrence(s) follows the industry standard for a GS1 code.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN_LIGHT_CALIBRATION +		Retrieves the mean intensity of the centers of the white elements of the reference code occurrence (MLcal). This result is determined during the reflectance calibration phase of the ISO/IEC 29158 standard. To start the target grading phase, you must retrieve this result and load it into the code context of the target code. Use McodeControl() with M_MEAN_LIGHT_CALIBRATION to load this result into the target code context; alternatively, you can call McodeControl() with M_DPM_CALIBRATION_RESULTS. This result type is available for 1D (except for 4-state, Pharmacode, Postnet, and Planet), Aztec, Data Matrix, DotCode, QR code, and Micro QR code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 255	Specifies the mean intensity of the centers of the white elements.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN_LIGHT_TARGET +		Retrieves the mean intensity of the centers of the white elements of the specified code occurrence(s) (MLtarget). This result is determined during the target grading phase of the ISO/IEC 29158 standard. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_DPM_GRADING before McodeGrade() is called. This result type is available for Aztec, Data Matrix, QR code, and Micro QR code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 255	Specifies the mean intensity of the centers of the white elements.	c		e	f	g	a		i		j	k	l	m	n	b

M_MINIMUM_REFLECTANCE +	Retrieves the amount of reflectance in the white elements of the specified code occurrence(s). This result type will return M_CODE_GRADE_NOT_AVAILABLE if McodeControl() with M_LIGHTING_CONFIGURATION is set to M_90_DEGREE and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_DPM_GRADING before McodeGrade() is called. This result type is available for Aztec, Data Matrix, QR code, and Micro QR code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the amount of reflectance in the white elements, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_MINIMUM_REFLECTANCE_GRADE +	Retrieves the minimum reflectance of the specified code occurrence(s), as a grade. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...). This result type will return M_CODE_GRADE_NOT_AVAILABLE if McodeControl() with M_LIGHTING_CONFIGURATION is set to M_90_DEGREE and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_DPM_GRADING before McodeGrade() is called. This result type is available for Aztec, Data Matrix, QR code, and Micro QR code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b

		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MODULATION_GRADE +		Retrieves the modulation (MOD) of the specified code occurrence(s), as a grade calculated using the ISO standard. Modulation is a measure of the uniformity of reflectance in the dark and light areas of the code. For more information, refer to the ISO/IEC 15415 specification. To retrieve the modulation of each codeword as a grade, use M_CODEWORD_MODULATION_GRADE. To retrieve the modulation of the start/stop patterns of 2D cross-row and composite code types, as a grade, use M_SCAN_MODULATION_GRADE. To retrieve the modulation of a code calculated using the ISO/IEC 29158 standard, use M_CELL_MODULATION_GRADE. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for Aztec, Data Matrix, QR code, Micro QR, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

M_NUMBER_OF_CODEWORDS +	Retrieves the number of codewords in the specified code occurrence(s); this includes data, overhead, and error correction codewords. This result type is available for 2D and composite code types, with the exception of Data Matrix code types that do not use M_ERROR_CORRECTION with M_ECC_200.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_DATA_CODEWORDS +	Retrieves the number of data codewords in the specified code occurrence(s). This result type is available for 2D and composite code types, with the exception of Data Matrix code types that do not use M_ERROR_CORRECTION with M_ECC_200.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_DECODED_ROWS +	Retrieves the number of successfully decoded rows of the specified code occurrence(s). This result type is available for all 1D and composite code types. For GS1 Databar code types, the number of rows is the number of stacks in the specified code occurrence(s). For composite code types, this is the number of rows in the 1D portion of the code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_DECODED_SCANS +	Retrieves the total number of successfully decoded scanlines of the specified code occurrence(s). This is the sum of the number of scanlines per row, for every row within the code occurrence. For composite code types, this is the number of scanlines in the 1D portion of the code occurrence(s). This result type is available for all 1D and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_ERASURES +	Retrieves the number of erasures in the specified code occurrence(s). Erasures are missing or unreadable codewords at known positions. This result type is available for 2D and composite code types, with the exception of Data Matrix code types that do not use M_ERROR_CORRECTION with M_ECC_200.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_ERROR_CORRECTION_CODEWORDS +		Retrieves the number of error correction codewords in the specified code occurrence(s). Error correction can be used to compensate for defects found during the decoding process. This result type is available for 2D and composite code types, with the exception of Data Matrix code types that do not use M_ERROR_CORRECTION with M_ECC_200.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_ERRORS +		Retrieves the number of errors found in the specified code occurrence(s). Note that the number of errors does not include the number of erasures. This result type is available for 2D and composite code types, with the exception of Data Matrix code types that do not use M_ERROR_CORRECTION with M_ECC_200.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_FIXED_PATTERN_DAMAGE_B_SEGMENT +		Retrieves the number of B segments in the specified code occurrence(s) of an Aztec code type with a full-range encoding scheme (M_ENC_AZTEC_FULL_RANGE); you can use this result type to establish the required array size to retrieve the fixed pattern damage in each B segment of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 18	Specifies the number of B segment damage errors reported within a full Aztec code occurrence.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_INTERLEAVED_BLOCKS +		Retrieves the number of interleaved Reed-Solomon blocks in the specified code occurrence(s). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

	Value > 0	Specifies the number of interleaved Reed-Solomon blocks.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_ROWS +		Retrieves the number of rows graded in the specified code occurrence(s). For GS1 Databar code types, the number of rows is the number of stacks in the specified code occurrence(s). For cross-row code types, the number of rows is the number of rows graded in the start/stop pattern grading. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_SCANS +		Retrieves the total number of scanlines analyzed in the specified code occurrence(s). This is the sum of the number of scanlines per row, for every row within the code occurrence. The number of scanlines determines the number of results within an array for all the M_SCAN... values described in this function. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERALL_SYMBOL_GRADE +		Retrieves the overall symbol grade of the specified code occurrence(s). The code type determines the calculation used to obtain the grade. For 1D code types, the overall symbol grade is the average grade of the scan reflectance profile (using M_SCAN_REFLECTANCE_PROFILE_GRADE). For composite code types, the overall symbol grade is the worst grade overall, obtained when analyzing either the 1D or the 2D portions. For 2D matrix code types, the overall symbol grade is the worst of all the returned grades. For cross-row codes, the overall symbol grade is the worst of all possible grades (with the exception of M_SCAN...GRADE). To retrieve the overall symbol grade for each row in the code, use M_ROW_OVERALL_GRADE.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 4.0	Specifies the overall symbol grade.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_X +		Retrieves the X-coordinate of the specified code occurrence(s). For all types of codes, except the Data Matrix and composite code types, this result is the center of the code in the X-direction. For Data Matrix codes, after an McodeRead() or McodeGrade() operation this result is the center, in the X-direction, of the top-left cell. For composite codes, this result is the center, in the X-direction, of the 1D component of the code. Note that to retrieve the top-left and bottom-right coordinates, use M_BOTTOM... and M_TOP....	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_Y +		Retrieves the Y-coordinate of the specified code occurrence(s). For all types of codes, except the Data Matrix and composite code types, this result is the center of the code in the Y-direction. For Data Matrix codes, after an McodeRead() or McodeGrade() operation this result is the center, in the Y-direction, of the top-left cell. For composite codes, this result is the center, in the Y-direction, of the 1D component of the code. Note that to retrieve the top-left and bottom-right coordinates, use M_BOTTOM... and M_TOP....	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINT_GROWTH +		Retrieves the print growth value of the specified code occurrence(s). The print growth is a value determining whether the graphical features of the code have either grown or shrunk, relative to the theoretical model, sufficiently to hinder readability. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is available for 2D matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

	-1.0 <= *Value* <= 1.0	Specifies the print growth value.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINT_GROWTH_GRADE +		Retrieves the print growth value of the specified code occurrence(s), as a grade. This result type is available for Aztec, Maxicode, and DotCode code types, and Data Matrix code types that do not use M_ERROR_CORRECTION with M_ECC_200.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINT_GROWTH_HORIZONTAL +		Retrieves the horizontal print growth value of the code occurrence. The print growth is a value determining whether the graphical features of the code have either grown or shrunk, relative to the theoretical model, sufficiently to hinder readability. This result type is available for the M_DOTCODE code type. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	-1.0 <= *Value* <= 1.0	Specifies the print growth value.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINT_GROWTH_HORIZONTAL_GRADE +		Retrieves the horizontal print growth value of the code occurrence, as a grade. This result type is available for the M_DOTCODE code type.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINT_GROWTH_VERTICAL +		Retrieves the vertical print growth value of the code occurrence. The print growth is a value determining whether the graphical features of the code have either grown or shrunk, relative to the theoretical model, sufficiently to hinder readability. This result type is available for the M_DOTCODE code type. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	-1.0 <= *Value* <= 1.0	Specifies the print growth value.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINT_GROWTH_VERTICAL_GRADE +		Retrieves the vertical print growth value of the code occurrence, as a grade. This result type is available for the M_DOTCODE code type.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

M_QUIET_ZONE_BOTTOM_LEFT_X +	Retrieves the bottom-left position along the X-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_BOTTOM_LEFT_Y +	Retrieves the bottom-left position along the Y-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_BOTTOM_RIGHT_X +	Retrieves the bottom-right position along the X-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_BOTTOM_RIGHT_Y +	Retrieves the bottom-right position along the Y-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_INCLUDED +	Retrieves whether the quiet zone was included in the source image of the code operation, for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Indicates the quiet zone was not included.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Indicates the quiet zone was included.	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_TOP_LEFT_X +	Retrieves the top-left position along the X-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_TOP_LEFT_Y +	Retrieves the top-left position along the Y-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_QUIET_ZONE_TOP_RIGHT_X +	Retrieves the top-right position along the X-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_TOP_RIGHT_Y +	Retrieves the top-right position along the Y-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_R_MAX +	Retrieves the highest reflectance (Rmax) of the specified code occurrence(s). This result type requires that the specified code be graded using proper code verification techniques. For more information on 1D code types, refer to the ISO/IEC 15416 specification. For more information on 2D matrix code types, refer to the ISO/IEC 15415 specification. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 255	Specifies the highest reflectance (Rmax).	c		e	f	g	a		i		j	k	l	m	n	b
M_R_MIN +	Retrieves the lowest reflectance (Rmin) of the specified code occurrence(s). This result type requires that the specified code be graded using proper code verification techniques. For more information on 1D code types, refer to the ISO/IEC 15416 specification. For more information on 2D matrix code types, refer to the ISO/IEC 15415 specification. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 255	Specifies the lowest reflectance (Rmin).	c		e	f	g	a		i		j	k	l	m	n	b

	M_RECOMMENDED_APERTURE_SIZE +	Retrieves the recommended aperture size for the specified code occurrence(s), following the ISO/IEC 15415 specification for a 2D matrix code type and the ISO/IEC 15416 specification for all other available code types. Note that, for 1D, 2D cross-row, and composite code types, this value depends on McodeControl() with M_PIXEL_SIZE_IN_MM. If M_PIXEL_SIZE_IN_MM is not set (or is set to M_UNKNOWN), the value returned is (0.5 x *cell size*). If the M_PIXEL_SIZE_IN_MM is set to a value, the recommended aperture size is computed using the ISO/IEC 15416 specification. For 2D matrix code types, the value returned is (0.8 x *cell size*), regardless of the value of M_PIXEL_SIZE_IN_MM.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_REFLECTANCE_CALIBRATION +	Retrieves the maximum reflectance value derived during the reflectance calibration phase (Rcal), for the specified code occurrence(s). This result is determined during the reflectance calibration phase of the ISO/IEC 29158 standard. To start the target grading phase, you must retrieve this result and load it into the code context of the target code. Use McodeControl() with M_REFLECTANCE_CALIBRATION to load this result into the target code context; alternatively, you can call McodeControl() with M_DPM_CALIBRATION_RESULTS. This result type is available for 1D (except for 4-state, Pharmacode, Postnet, and Planet), Aztec, Data Matrix, DotCode, QR code, and Micro QR code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 255	Specifies the maximum reflectance value derived during the reflectance calibration phase (Rcal).	c		e	f	g	a		i		j	k	l	m	n	b

	M_REFLECTANCE_MARGIN_GRADE +	Retrieves the assessment of the codeword print quality for the specified code occurrence(s), based on the reflectance margin, as a grade. For more information, refer to the ISO/IEC 15415 specification. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 2D matrix code types, except Maxicode.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

M_SCORE +		Retrieves the confidence score of the code operation for the specified code occurrence(s). The code type determines how this value is calculated. • For linear 1D and GS1 Databar code types, the score is based on: ◦ The number of redundant scanlines. Redundant scanlines can be omitted from a code, while still yielding the same result. ◦ The difference between the actual code and the theoretical code model. • For 1D Planet, Postnet, and 4-state code types, the score is 1 if the code was read and 0 if it was not. • For 2D code types, the score is calculated using: 1 - (*Number of errors*)/(*Number of error correction codewords*). If the code could not be decoded successfully, the score is 0. • For 2D cross-row types, the score is calculated using: 1 - (*Number of errors + Number of erasures*)/(*Number of codewords*). If the code could not be decoded successfully, the score is 0. • For the M_DATAMATRIX code type, the score is calculated using: 1 - (*Number of errors + Number of erasures*)/(*Number of error correction codewords*). If the code could not be decoded successfully, the score is 0. • For composite codes, the score is the score of either the 1D part or the 2D part; whichever was lower. To retrieve the error status of the McodeGrade() operation, use M_STATUS. To retrieve the confidence score of an McodeGrade() operation as a grade, use M_OVERALL_SYMBOL_GRADE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the confidence score. 0 indicates 0% confidence and 1 indicates 100% confidence.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_X +		Retrieves the size of the specified code occurrence(s) in the X-direction. For composite code types, this result is the width of the 1D component of the code.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_SIZE_Y +	Retrieves the size of the specified code occurrence(s) in the Y-direction. For composite code types, this result is the height of the 1D component of the code.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_START_STOP_PATTERN_GRADE +	Retrieves the average of each row's worst scan reflectance profile grade for the specified code occurrence(s), as a grade. A start/stop pattern is a set pattern that marks the end points of the code. For each scan reflectance profile, the start/stop pattern grade is computed for each graded row, then the average start/stop pattern grade for all the scan reflectance profiles is computed. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 2D cross-row and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 4.0	Specifies the average of each row's worst scan reflectance profile grade.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING +	Retrieves the decoded string of the specified code occurrence(s). If the string is decoded from a composite code model, the string is returned in the following format: 1D string|2D string, where | is used as the separator. Note that for this result type, ResultIndex cannot be set to M_ALL or M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR*	c		e	f	g	a		i		j	k	l	m	n	b

M_SYMBOL_CONTRAST +	Retrieves the symbol contrast value (SC) of the specified code occurrence(s). In the case of matrix code types, the symbol contrast value is calculated as the average 10% of the lighter pixels minus the average 10% of the darker pixels. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is available for 2D matrix code types. If using a Data Matrix code type, this result type is also available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. In this case, this result is expressed as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_COMPUTABLE	Specifies that the result could not be computed for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the symbol contrast value.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the symbol contrast value, expressed as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_SYMBOL_CONTRAST_GRADE +	Retrieves the symbol contrast value (SC) of the specified code occurrence(s), as a grade. The higher the contrast, the better it is for scanning purposes and the better the grade. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 2D matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_SYMBOL_CONTRAST_SNR +		Retrieves the symbol contrast signal to noise ratio (SNR) of the specified code occurrence(s). It is a relative measure of the symbol contrast (signal) to the maximum deviation in the light or dark grayscale levels in the symbol (noise). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_COMPUTABLE	Specifies that the result was not computable for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the symbol contrast value.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_MODE +		Retrieves the threshold mode used to internally binarize the source image for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ADAPTIVE	Specifies the use of a fast dynamic local threshold. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_GLOBAL_SEGMENTATION	Specifies the use of a global threshold value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_GLOBAL_WITH_LOCAL_RESEGMENTATION	Specifies that the source image was globally thresholded and then the edges in the binarized image were resegmented according to the intensities of the surrounding bars and spaces in the original source image. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_VALUE +		Retrieves the threshold value used to internally binarize the source image for the specified code occurrence(s). Note that, this result type is only available when M_THRESHOLD_MODE is not set to M_ADAPTIVE.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
	0 <= *Value* <= 255	Specifies the threshold value.	c	e	f	g	a	i	j	k	l	m	n	b
M_TOP_LEFT_X +		Retrieves the X-coordinate of the top-left corner of the specified code occurrence(s).	c	e	f	g	a	i	j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
M_TOP_LEFT_Y +		Retrieves the Y-coordinate of the top-left corner of the specified code occurrence(s).	c	e	f	g	a	i	j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
M_TOP_RIGHT_X +		Retrieves the X-coordinate of the top-right corner of the specified code occurrence(s).	c	e	f	g	a	i	j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
M_TOP_RIGHT_Y +		Retrieves the Y-coordinate of the top-right corner of the specified code occurrence(s).	c	e	f	g	a	i	j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
M_UNUSED_ERROR_CORRECTION +		Retrieves the ratio of unused error correction to the total amount of error correction available for the specified code occurrence(s). This tests the extent to which regional or spot damage in the code has eroded the reading safety margin that error correction provides. Note that when retrieving this result from an McodeGrade() operation that used the SEMI standard (McodeControl() with M_GRADING_STANDARD set to M_SEMI_T10_GRADING), this result is expressed as a percentage. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is available for all 2D and composite code types.	c	e	f	g	a	i	j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b

		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 1.0	Specifies the ratio of unused error correction within the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 100.0	Specifies the ratio of unused error correction within the specified code occurrence(s), expressed as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNUSED_ERROR_CORRECTION_GRADE +		Retrieves the ratio of unused error correction to the total amount of error correction available for the specified code occurrence(s), as a grade. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING or M_ISO_DPM_GRADING before McodeGrade() is called. This result type is available for all 2D and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b

M_VERSION_INFORMATION_1_GRADE +	Retrieves the readability of the version information in segment 1 of the specified code occurrence(s), as a grade. The version information is an embedded pattern used to identify the location of the error correction bits in the code. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...). This result type is only available for the QR code type.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
M_VERSION_INFORMATION_2_GRADE +	Retrieves the readability of the version information in segment 2 of the specified code occurrence(s), as a grade. The version information is an embedded pattern used to identify the location of the error correction bits in the code. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...). This result type is only available for the QR code type.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
M_VERSION_INFORMATION_GRADE +		Retrieves the readability of the version information for the specified code occurrence(s), as a grade. The version information is an embedded pattern used to identify the location of the error correction bits in the code. When M_GRADING_STANDARD is set to M_ISO_DPM_GRADING and M_GRADING_STANDARD_EDITION is set to M_ISO_29158_2020 or M_DEFAULT, this grade is retrieved as an interpolated value to the nearest 0.1 between grade levels. For M_ISO_29158_2011, this result type is retrieved as an interpolated integer value between 0 and 4. For all other settings, this result type is retrieved as a letter grade (M_CODE_GRADE_...). For QR codes version 1 through version 7, M_CODE_GRADE_NOT_AVAILABLE is returned. This result type is only available for the QR code type.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 4.0	Specifies an interpolation value between grading values rounded to the nearest 0.1.	c		e	f	g	a		i		j	k	l	m	n	b

M_VERTICAL_MARK_GROWTH +	Retrieves the vertical mark growth (VMG) value of the specified code occurrence(s). The vertical mark growth value is the ratio between the median mark cell height (MCH) and the sum of this height and the median space cell height (SCH) in the vertical alternating pattern, expressed as a percentage. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_COMPUTABLE	Specifies that the result was not computable for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the vertical mark growth value, expressed as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_VERTICAL_MARK_MISPLACEMENT +	Retrieves the vertical mark misplacement (VMM) value of the specified code occurrence(s). The vertical mark misplacement value is the average vertical displacement of the marks' centers from the their ideal cell center point, expressed as a percentage. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_SEMI_T10_GRADING before McodeGrade() is called. This result type is available for Data Matrix code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_COMPUTABLE	Specifies that the result was not computable for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the vertical mark misplacement value, expressed as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
Code read result buffer ID for occurrence-specific results	Specifies a code read result buffer, allocated using McodeAllocResult() with M_CODE_READ_RESULT, and used to store McodeRead() results.	c		e	f	g	a		i		j	k	l	m	n	b

M_ANGLE +	Retrieves the angle, in degrees, of the specified code occurrence(s), relative to the output coordinate system specified using McodeControl() with M_RESULT_OUTPUT_UNITS. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup. For an occurrence of a 1D code type from an McodeRead() operation, this value is dependent on McodeControl() with M_POSITION_ACCURACY.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTTOM_LEFT_X +	Retrieves the X-coordinate of the bottom-left corner of the specified code occurrence(s). For an occurrence of a 1D code type from an McodeRead() operation, this value is dependent on McodeControl() with M_POSITION_ACCURACY.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTTOM_LEFT_Y +	Retrieves the Y-coordinate of the bottom-left corner of the specified code occurrence(s). For an occurrence of a 1D code type from an McodeRead() operation, this value is dependent on McodeControl() with M_POSITION_ACCURACY.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTTOM_RIGHT_X +	Retrieves the X-coordinate of the bottom-right corner of the specified code occurrence(s). For an occurrence of a 1D code type from an McodeRead() operation, this value is dependent on McodeControl() with M_POSITION_ACCURACY.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_BOTTOM_RIGHT_Y +	Retrieves the Y-coordinate of the bottom-right corner of the specified code occurrence(s). For an occurrence of a 1D code type from an McodeRead() operation, this value is dependent on McodeControl() with M_POSITION_ACCURACY.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_NUMBER_X +	Retrieves the number of cells in the X-direction of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of cells in the X-direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_NUMBER_Y +	Retrieves the number of cells in the Y-direction of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of cells in the Y-direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE +	Retrieves the size of the cell in X (module size (Z)) of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE_MAX +	Retrieves the maximum cell size of the code occurrence. For 2D matrix code types, this corresponds to the nominal maximum module size, whereas for other code types, this corresponds to the same result as M_CELL_SIZE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum cell size, relative to the input coordinate system specified using M_CELL_SIZE_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_CELL_SIZE_MIN +	Retrieves the minimum cell size of the code occurrence. For 2D matrix code types, this corresponds to the nominal minimum module size, whereas for other code types, this corresponds to the same result as M_CELL_SIZE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the minimum cell size, relative to the input coordinate system specified using M_CELL_SIZE_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_FLIP +	Retrieves for the specified code occurrence(s) whether it was necessary to flip it/read it in the opposite direction.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FLIP	Specifies that the code occurrence was reversed.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_FLIP	Specifies that the code occurrence was not reversed.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_MODEL_ID +	Retrieves the identifier of the code model of the code occurrence. Note that this result is not valid when the result buffer is on a remote system. To find out which code model was used when the result buffer is on a remote system, use M_CODE_MODEL_INDEX instead.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_MODEL_INDEX +	Retrieves the index of the code model of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_TYPE +	Retrieves the code type of the specified code occurrence(s). The code type is retrieved as a numeric that can be passed to McodeModel().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_4_STATE	Specifies a 4-state code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_AZTEC	Specifies an Aztec code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b

		M_BC412	Specifies a BC412 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODABAR	Specifies a Codabar code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE39	Specifies a Code 39 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE93	Specifies a Code 93 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE128	Specifies a Code 128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COMPOSITECODE	Specifies a composite code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_DATAMATRIX	Specifies a Data Matrix code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_DOTCODE	Specifies a DotCode code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_EAN8	Specifies an EAN 8 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_EAN13	Specifies an EAN 13 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_EAN14	Specifies an EAN 14 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_GS1_128	Specifies a GS1-128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_GS1_DATABAR	Specifies a GS1 Databar code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IATA25	Specifies an IATA 2 of 5 code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INDUSTRIAL25	Specifies an Industrial 2 of 5 (standard 2 of 5) code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INTERLEAVED25	Specifies an Interleaved 2 of 5 (ITF-14) code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_MAXICODE	Specifies a Maxicode code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_MICROPDF417	Specifies a MicroPDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_MICROQRCODE	Specifies a Micro QR code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_PDF417	Specifies a PDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_PHARMACODE	Specifies a Pharmacode code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PLANET	Specifies a Planet code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_POSTNET	Specifies a Postnet code type.	c		e	f	g	a		i		j	k	l	m	n	b
		M_QRCODE	Specifies a QR code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_TRUNCATED_PDF417	Specifies a Truncated PDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPC_A	Specifies a UPC-A code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPC_E	Specifies a UPC-E code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_DATA_CODEWORDS +		Retrieves the data codewords from the graphical representation of the specified code occurrence(s). Note that each codeword is returned as a numerical value, rather than as one or more alpha-numeric characters. The returned data codewords could be used to validate the code, as with PDF417, or to allow interpretation of the data codewords using a character set other than the default set supported by Aurora Imaging Library. For more information, refer to the Extended Channel Interpretation (ECI) protocol in ISO/IEC 15438:2006. To retrieve the decoded characters from the specified code occurrence(s), use M_STRING. This result type is available for 2D and composite code types, with the exception of Data Matrix code types that do not use M_ERROR_CORRECTION with M_ECC_200.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SPACING_USED +		Retrieves the value between M_DOT_SPACING_MIN and M_DOT_SPACING_MAX used to decode the specified code occurrence(s). The expected dot spacing is set using McodeControl() with M_DOT_SPACING_MIN and M_DOT_SPACING_MAX. This result type is available for Aztec, Data Matrix, Maxicode, QR code, and Micro QR code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ELEMENT_NUMBER_X +		Retrieves the number of elements in the X-direction. For 1D code types, this result retrieves the number of bars and spaces. For GS1 Databar Stacked code types and composite code types whose 1D portion uses a GS1 Databar Stacked code type, this result retrieves the number of bars and spaces in all rows. For composite code types, this result retrieves the number of bars and spaces in the 1D portion of the code. For 2D cross-row and 2D matrix code types, this result retrieves the same value as M_CELL_NUMBER_X.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ELEMENT_NUMBER_Y +		Retrieves the number of elements in the Y-direction. For composite code types, this result retrieves the number of rows in the 1D portion of the code. For 2D matrix code types, this result retrieves the number of cells. For 1D and 2D cross-row code types, this result retrieves the same value as M_CELL_NUMBER_Y. Note that for 1D code types, excluding GS1 Databar, this result will always return 1.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ENCODING +		Retrieves the type of encoding of the specified code occurrence(s). Note that since some versions of the GS1 Databar code types only differ by the bar height (and not the structure), the same result is returned for these similar code types. GS1 Databar Omnidirectional and GS1 Databar Truncated will return M_ENC_GS1_DATABAR_OMNI. GS1 Databar Stacked and GS1 Databar Stacked Omnidirectional will return M_ENC_GS1_DATABAR_STACKED. It is possible to obtain a more accurate result to distinguish between these structurally similar code types, using McodeControl() with M_POSITION_ACCURACY set to M_HIGH.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ALPHA	Specifies an encoding scheme that supports uppercase alphabetical characters, along with the space. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ALPHANUM	Specifies an encoding scheme that supports alphanumeric characters, as well as the space. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ALPHANUM_PUNC	Specifies a similar encoding scheme to M_ENC_ALPHANUM, except it also supports the following characters: (,), (-), (/) and (. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ASCII	Specifies an encoding scheme that supports ASCII characters. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_AUSTRALIA_MAIL_C	Specifies an encoding scheme for a 4-state format used with the C encoding table by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b

		M_ENC_AUSTRALIA_MAIL_N	Specifies an encoding scheme for a 4-state format used with the N encoding table by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AUSTRALIA_MAIL_RAW	Specifies an encoding scheme for a 4-state format used by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AZTEC_COMPACT	Specifies an encoding scheme for a compact Aztec code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AZTEC_FULL_RANGE	Specifies an encoding scheme for a full-range (not compact) Aztec code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AZTEC_RUNE	Specifies an encoding scheme for an Aztec rune (the smallest version of an Aztec code). INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN8	Specifies an encoding scheme for a composite code whose 1D portion uses an EAN 8 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN8_ADDON	Specifies an encoding scheme for an EAN 8 format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN13	Specifies an encoding scheme for a composite code whose 1D portion uses an EAN 13 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN13_ADDON	Specifies an encoding scheme for an EAN 13 format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_128_MICROPDF417	Specifies an encoding scheme for a composite code whose 1D portion uses a GS1 128 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_128_PDF417	Specifies an encoding scheme for a composite code whose 1D portion uses a GS1 128 format and whose 2D portion uses a PDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_EXPANDED	Specifies an encoding scheme that uses a GS1 Databar format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_EXPANDED_STACKED	Specifies an encoding scheme that uses a GS1 Databar Expanded Stacked format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_LIMITED	Specifies an encoding scheme that uses a GS1 Databar Limited format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_OMNI	Specifies an encoding scheme that uses a GS1 Databar format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_STACKED	Specifies an encoding scheme that uses a GS1 Databar Stacked format. INFO	c		e	f	g	a		i		j	k	l	m	n	b

		M_ENC_GS1_DATABAR_STACKED_OMNI	Specifies an encoding scheme that uses a GS1 Databar Stacked Omnidirectional format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_TRUNCATED	Specifies an encoding scheme that uses a GS1 Databar Truncated format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_ISO8	Specifies a similar encoding scheme as M_ENC_ASCII, but supports the extended ASCII character set. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_KOREA_MAIL	Specifies an encoding scheme for a 4-state format used by the Korean Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE2	Specifies an encoding scheme that requires a Structured Carrier Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE3	Specifies an encoding scheme that requires a Structured Carrier Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE4	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE5	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE6	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_NUM	Specifies an encoding scheme that only supports numbers. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_QRCODE_MODEL1	Specifies an encoding scheme that uses an older version of the QR code format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_QRCODE_MODEL2	Specifies an encoding scheme that uses a newer version of the QR code format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_STANDARD	Specifies different types of encoding schemes, depending on what code type is used. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UK_MAIL	Specifies an encoding scheme for a 4-state format used by the UK Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UPCA	Specifies an encoding scheme for a composite code whose 1D portion uses an UPC-A format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b

		M_ENC_UPCA_ADDON	Specifies an encoding scheme for an UPC-A format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UPCE	Specifies an encoding scheme for a composite code whose 1D portion uses an UPC-E format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UPCE_ADDON	Specifies an encoding scheme for an UPC-E format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_US_MAIL	Specifies the Intelligent Mail Barcode encoding scheme for a 4-state format used by the US Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		5 <= *Value* <= 95	Specifies the minimum amount of the symbol that contains error correction information, as a percentage. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ERROR_CORRECTION +		Retrieves the type of error correction scheme of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_ECC_4STATE	Specifies the Reed-Solomon-based algorithm or a check digit type of error correction scheme, depending on the specification of the encoding. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ECC_200	Specifies a Reed-Solomon-based algorithm as an error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ECC_CHECK_DIGIT	Specifies an additional digit to check whether there is an error or not. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ECC_COMPOSITE	Specifies the default error correction scheme for the 1D and 2D portions of the composite code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ECC_H	Specifies the highest-level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ECC_L	Specifies the lowest-level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ECC_M	Specifies a medium-low level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ECC_NONE	Specifies no error correction. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ECC_Q	Specifies a medium-high level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_ECC_REED_SOLOMON	Specifies a Reed-Solomon type of error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_REED_SOLOMON_*n*	Specifies a Reed-Solomon type of error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_UNKNOWN	Specifies an unknown error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	5 <= *Value* <= 95	Specifies the minimum percentage of the symbol that contains error correction information. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_BOTTOM_LEFT_X +		Retrieves the X-coordinate of the bottom-left corner of the specified code occurrence(s), including the extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_BOTTOM_LEFT_Y +		Retrieves the Y-coordinate of the bottom-left corner of the specified code occurrence(s), including the extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

	M_EXTENDED_AREA_BOTTOM_RIGHT_X +	Retrieves the X-coordinate of the bottom-right corner of the specified code occurrence(s), including the extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_EXTENDED_AREA_BOTTOM_RIGHT_Y +	Retrieves the Y-coordinate of the bottom-right corner of the specified code occurrence(s), including the extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_EXTENDED_AREA_QUIET_ZONE_INCLUDED +	Retrieves whether the quiet zone and the extended area (that is, 20 times the cell size beyond the quiet zone on all sides) were included in the source image of the code operation, for the specified code occurrence(s). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the quiet zone and extended area were not included.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the quiet zone and extended area were included. This result type requires that the specified code be graded using proper code verification techniques. For more information, refer to the ISO/IEC 15415 specification.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_TOP_LEFT_X +	Retrieves the X-coordinate of the top-left corner of the specified code occurrence(s), including the extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_EXTENDED_AREA_TOP_LEFT_Y +	Retrieves the Y-coordinate of the top-left corner of the specified code occurrence(s), including the extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_TOP_RIGHT_X +	Retrieves the X-coordinate of the top-right corner of the specified code occurrence(s), including the extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_EXTENDED_AREA_TOP_RIGHT_Y +	Retrieves the Y-coordinate of the top-right corner of the specified code occurrence(s), including the extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. When performing an McodeGrade() operation from the result of an McodeRead() operation, the required controls must be set before the McodeRead() operation. This result type is available for Aztec, Data Matrix, DotCode, QR code, and Micro QR code types. This result type is only available when M_EXTENDED_AREA_REFLECTANCE_CHECK is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_VALUE +	Retrieves the foreground color of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_BLACK	Specifies that the foreground color is black.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_WHITE	Specifies that the foreground color is white.	c		e	f	g	a		i		j	k	l	m	n	b
M_IS_ECI +	Retrieves whether the decoded string of the specified code occurrence(s) contains character set ECIs. This result type is available for Aztec, Data Matrix, DotCode, Maxicode, Micro PDF417, PDF417, QR code, and truncated PDF417 code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the decoded string does not contains character set ECIs.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the decoded string contains character set ECIs.	c		e	f	g	a		i		j	k	l	m	n	b

M_IS_GS1 +	Retrieves whether the specified code occurrence(s) follows the industry standard for a GS1 code. This result type is available for Aztec, Data Matrix, Code 128, EAN 14, GS1-128, GS1-Databar, QR code, DotCode, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the specified code occurrence(s) does not follow the industry standard for a GS1 code.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the specified code occurrence(s) follows the industry standard for a GS1 code.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_CODEWORDS +	Retrieves the number of codewords in the specified code occurrence(s); this includes data, overhead, and error correction codewords. This result type is available for 2D and composite code types, with the exception of Data Matrix code types that do not use M_ERROR_CORRECTION with M_ECC_200.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_DATA_CODEWORDS +	Retrieves the number of data codewords in the specified code occurrence(s). This result type is available for 2D and composite code types, with the exception of Data Matrix code types that do not use M_ERROR_CORRECTION with M_ECC_200.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_DECODED_ROWS +	Retrieves the number of successfully decoded rows of the specified code occurrence(s). This result type is available for all 1D and composite code types. For GS1 Databar code types, the number of rows is the number of stacks in the specified code occurrence(s). For composite code types, this is the number of rows in the 1D portion of the code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

	M_NUMBER_OF_DECODED_SCANS +	Retrieves the total number of successfully decoded scanlines of the specified code occurrence(s). This is the sum of the number of scanlines per row, for every row within the code occurrence. For composite code types, this is the number of scanlines in the 1D portion of the code occurrence(s). This result type is available for all 1D and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NUMBER_OF_ERASURES +	Retrieves the number of erasures in the specified code occurrence(s). Erasures are missing or unreadable codewords at known positions. This result type is available for 2D and composite code types, with the exception of Data Matrix code types that do not use M_ERROR_CORRECTION with M_ECC_200.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NUMBER_OF_ERROR_CORRECTION_CODEWORDS +	Retrieves the number of error correction codewords in the specified code occurrence(s). Error correction can be used to compensate for defects found during the decoding process. This result type is available for 2D and composite code types, with the exception of Data Matrix code types that do not use M_ERROR_CORRECTION with M_ECC_200.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NUMBER_OF_ERRORS +	Retrieves the number of errors found in the specified code occurrence(s). Note that the number of errors does not include the number of erasures. This result type is available for 2D and composite code types, with the exception of Data Matrix code types that do not use M_ERROR_CORRECTION with M_ECC_200.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_POSITION_X +	Retrieves the X-coordinate of the specified code occurrence(s). For all types of codes, except the Data Matrix and composite code types, this result is the center of the code in the X-direction. For Data Matrix codes, after an McodeRead() or McodeGrade() operation this result is the center, in the X-direction, of the top-left cell. For composite codes, this result is the center, in the X-direction, of the 1D component of the code. Note that to retrieve the top-left and bottom-right coordinates, use M_BOTTOM... and M_TOP.... For an occurrence of a 1D code type from an McodeRead() operation, this value is dependent on McodeControl() with M_POSITION_ACCURACY.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_Y +	Retrieves the Y-coordinate of the specified code occurrence(s). For all types of codes, except the Data Matrix and composite code types, this result is the center of the code in the Y-direction. For Data Matrix codes, after an McodeRead() or McodeGrade() operation this result is the center, in the Y-direction, of the top-left cell. For composite codes, this result is the center, in the Y-direction, of the 1D component of the code. Note that to retrieve the top-left and bottom-right coordinates, use M_BOTTOM... and M_TOP.... For an occurrence of a 1D code type from an McodeRead() operation, this value is dependent on McodeControl() with M_POSITION_ACCURACY.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_BOTTOM_LEFT_X +	Retrieves the bottom-left position along the X-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_BOTTOM_LEFT_Y +	Retrieves the bottom-left position along the Y-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_BOTTOM_RIGHT_X +	Retrieves the bottom-right position along the X-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_BOTTOM_RIGHT_Y +		Retrieves the bottom-right position along the Y-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_INCLUDED +		Retrieves whether the quiet zone was included in the source image of the code operation, for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Indicates the quiet zone was not included.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Indicates the quiet zone was included.	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_TOP_LEFT_X +		Retrieves the top-left position along the X-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_TOP_LEFT_Y +		Retrieves the top-left position along the Y-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_TOP_RIGHT_X +		Retrieves the top-right position along the X-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_TOP_RIGHT_Y +		Retrieves the top-right position along the Y-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_RECOMMENDED_APERTURE_SIZE +	Retrieves the recommended aperture size for the specified code occurrence(s), following the ISO/IEC 15415 specification for a 2D matrix code type and the ISO/IEC 15416 specification for all other available code types. Note that, for 1D, 2D cross-row, and composite code types, this value depends on McodeControl() with M_PIXEL_SIZE_IN_MM. If M_PIXEL_SIZE_IN_MM is not set (or is set to M_UNKNOWN), the value returned is (0.5 x *cell size*). If the M_PIXEL_SIZE_IN_MM is set to a value, the recommended aperture size is computed using the ISO/IEC 15416 specification. For 2D matrix code types, the value returned is (0.8 x *cell size*), regardless of the value of M_PIXEL_SIZE_IN_MM.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE +	Retrieves the confidence score of the code operation for the specified code occurrence(s). The code type determines how this value is calculated. • For linear 1D and GS1 Databar code types, the score is based on: ○ The number of redundant scanlines. Redundant scanlines can be omitted from a code, while still yielding the same result. ○ The difference between the actual code and the theoretical code model. • For 1D Planet, Postnet, and 4-state code types, the score is 1 if the code was read and 0 if it was not. • For 2D code types, the score is calculated using: 1 - (*Number of errors*)/(*Number of error correction codewords*). If the code could not be decoded successfully, the score is 0. • For 2D cross-row types, the score is calculated using: 1 - (*Number of errors + Number of erasures*)/(*Number of codewords*). If the code could not be decoded successfully, the score is 0. • For the M_DATAMATRIX code type, the score is calculated using: 1 - (*Number of errors + Number of erasures*)/(*Number of error correction codewords*). If the code could not be decoded successfully, the score is 0. • For composite codes, the score is the score of either the 1D part or the 2D part; whichever was lower.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the confidence score. 0 indicates 0% confidence and 1 indicates 100% confidence.	c	e	f	g	a	i	j	k	l	m	n	b
M_SIZE_X +		Retrieves the size of the specified code occurrence(s) in the X-direction. For composite code types, this result is the width of the 1D component of the code. For an occurrence of a 1D code type from an McodeRead() operation, this value is dependent on McodeControl() with M_POSITION_ACCURACY.	c	e	f	g	a	i	j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
M_SIZE_Y +		Retrieves the size of the specified code occurrence(s) in the Y-direction. For composite code types, this result is the height of the 1D component of the code. For an occurrence of a 1D code type from an McodeRead() operation, this value is dependent on McodeControl() with M_POSITION_ACCURACY.	c	e	f	g	a	i	j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
M_STRING +		Retrieves the decoded string of the specified code occurrence(s). If the string is decoded from a composite code model, the string is returned in the following format: 1D string|2D string, where | is used as the separator. Note that for this result type, ResultIndex cannot be set to M_ALL or M_DEFAULT.	c	e	f	g	a	i	j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR*	c	e	f	g	a	i	j	k	l	m	n	b
M_THRESHOLD_MODE +		Retrieves the threshold mode used to internally binarize the source image for the specified code occurrence(s).	c	e	f	g	a	i	j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
	M_ADAPTIVE	Specifies the use of a fast dynamic local threshold. INFO	c	e	f	g	a	i	j	k	l	m	n	b
	M_GLOBAL_SEGMENTATION	Specifies the use of a global threshold value. INFO	c	e	f	g	a	i	j	k	l	m	n	b

	M_GLOBAL_WITH_LOCAL_RESEGMENTATION	Specifies that the source image was globally thresholded and then the edges in the binarized image were resegmented according to the intensities of the surrounding bars and spaces in the original source image. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_VALUE +		Retrieves the threshold value used to internally binarize the source image for the specified code occurrence(s). Note that, this result type is only available when M_THRESHOLD_MODE is not set to M_ADAPTIVE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 255	Specifies the threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOP_LEFT_X +		Retrieves the X-coordinate of the top-left corner of the specified code occurrence(s). For an occurrence of a 1D code type from an McodeRead() operation, this value is dependent on McodeControl() with M_POSITION_ACCURACY.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TOP_LEFT_Y +		Retrieves the Y-coordinate of the top-left corner of the specified code occurrence(s). For an occurrence of a 1D code type from an McodeRead() operation, this value is dependent on McodeControl() with M_POSITION_ACCURACY.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TOP_RIGHT_X +		Retrieves the X-coordinate of the top-right corner of the specified code occurrence(s). For an occurrence of a 1D code type from an McodeRead() operation, this value is dependent on McodeControl() with M_POSITION_ACCURACY.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TOP_RIGHT_Y +		Retrieves the Y-coordinate of the top-right corner of the specified code occurrence(s). For an occurrence of a 1D code type from an McodeRead() operation, this value is dependent on McodeControl() with M_POSITION_ACCURACY.	c		e	f	g	a		i		j	k	l	m	n	b

UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Code write result buffer ID for occurrence-specific results	Specifies a code write result buffer, allocated using McodeAllocResult() with M_CODE_WRITE_RESULT, and used to store McodeWrite() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE +	Retrieves the angle, in degrees, of the specified code occurrence(s), relative to the output coordinate system specified using McodeControl() with M_RESULT_OUTPUT_UNITS. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTTOM_LEFT_X +	Retrieves the X-coordinate of the bottom-left corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTTOM_LEFT_Y +	Retrieves the Y-coordinate of the bottom-left corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTTOM_RIGHT_X +	Retrieves the X-coordinate of the bottom-right corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTTOM_RIGHT_Y +	Retrieves the Y-coordinate of the bottom-right corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_NUMBER_X +	Retrieves the number of cells in the X-direction of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the number of cells in the X-direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_NUMBER_Y +		Retrieves the number of cells in the Y-direction of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the number of cells in the Y-direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE +		Retrieves the size of the cell in X (module size (Z)) of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_TYPE +		Retrieves the code type of the specified code occurrence(s). The code type is retrieved as a numeric that can be passed to McodeModel().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_4_STATE	Specifies a 4-state code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AZTEC	Specifies an Aztec code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BC412	Specifies a BC412 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODABAR	Specifies a Codabar code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE39	Specifies a Code 39 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE93	Specifies a Code 93 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE128	Specifies a Code 128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPOSITECODE	Specifies a composite code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DATAMATRIX	Specifies a Data Matrix code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DOTCODE	Specifies a DotCode code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_EAN8	Specifies an EAN 8 code type.	c		e	f	g	a		i		j	k	l	m	n	b

	M_EAN13	Specifies an EAN 13 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_EAN14	Specifies an EAN 14 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GS1_128	Specifies a GS1-128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GS1_DATABAR	Specifies a GS1 Databar code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_IATA25	Specifies an IATA 2 of 5 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INDUSTRIAL25	Specifies an Industrial 2 of 5 (standard 2 of 5) code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERLEAVED25	Specifies an Interleaved 2 of 5 (ITF-14) code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAXICODE	Specifies a Maxicode code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MICROPDF417	Specifies a MicroPDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MICROQRCODE	Specifies a Micro QR code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PDF417	Specifies a PDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PHARMACODE	Specifies a Pharmacode code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PLANET	Specifies a Planet code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_POSTNET	Specifies a Postnet code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_QRCODE	Specifies a QR code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUNCATED_PDF417	Specifies a Truncated PDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPC_A	Specifies a UPC-A code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPC_E	Specifies a UPC-E code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_ELEMENT_NUMBER_X +		Retrieves the number of elements in the X-direction. For 1D code types, this result retrieves the number of bars and spaces. For GS1 Databar Stacked code types and composite code types whose 1D portion uses a GS1 Databar Stacked code type, this result retrieves the number of bars and spaces in all rows. For composite code types, this result retrieves the number of bars and spaces in the 1D portion of the code. For 2D cross-row and 2D matrix code types, this result retrieves the same value as M_CELL_NUMBER_X.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ELEMENT_NUMBER_Y +		Retrieves the number of elements in the Y-direction. For composite code types, this result retrieves the number of rows in the 1D portion of the code. For 2D matrix code types, this result retrieves the number of cells. For 1D and 2D cross-row code types, this result retrieves the same value as M_CELL_NUMBER_Y. Note that for 1D code types, excluding GS1 Databar, this result will always return 1.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ENCODING +		Retrieves the type of encoding of the specified code occurrence(s). Note that since some versions of the GS1 Databar code types only differ by the bar height (and not the structure), the same result is returned for these similar code types. GS1 Databar Omnidirectional and GS1 Databar Truncated will return M_ENC_GS1_DATABAR_OMNI. GS1 Databar Stacked and GS1 Databar Stacked Omnidirectional will return M_ENC_GS1_DATABAR_STACKED. It is possible to obtain a more accurate result to distinguish between these structurally similar code types, using McodeControl() with M_POSITION_ACCURACY set to M_HIGH.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ALPHA	Specifies an encoding scheme that supports uppercase alphabetical characters, along with the space. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ALPHANUM	Specifies an encoding scheme that supports alphanumeric characters, as well as the space. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ALPHANUM_PUNC	Specifies a similar encoding scheme to M_ENC_ALPHANUM, except it also supports the following characters: (,), (-), (/) and (. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ASCII	Specifies an encoding scheme that supports ASCII characters. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_AUSTRALIA_MAIL_C	Specifies an encoding scheme for a 4-state format used with the C encoding table by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b

		M_ENC_AUSTRALIA_MAIL_N	Specifies an encoding scheme for a 4-state format used with the N encoding table by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AUSTRALIA_MAIL_RAW	Specifies an encoding scheme for a 4-state format used by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AZTEC_COMPACT	Specifies an encoding scheme for a compact Aztec code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AZTEC_FULL_RANGE	Specifies an encoding scheme for a full-range (not compact) Aztec code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_AZTEC_RUNE	Specifies an encoding scheme for an Aztec rune (the smallest version of an Aztec code). INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN8	Specifies an encoding scheme for a composite code whose 1D portion uses an EAN 8 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN8_ADDON	Specifies an encoding scheme for an EAN 8 format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN13	Specifies an encoding scheme for a composite code whose 1D portion uses an EAN 13 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_EAN13_ADDON	Specifies an encoding scheme for an EAN 13 format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_128_MICROPDF417	Specifies an encoding scheme for a composite code whose 1D portion uses a GS1 128 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_128_PDF417	Specifies an encoding scheme for a composite code whose 1D portion uses a GS1 128 format and whose 2D portion uses a PDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_EXPANDED	Specifies an encoding scheme that uses a GS1 Databar format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_EXPANDED_STACKED	Specifies an encoding scheme that uses a GS1 Databar Expanded Stacked format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_LIMITED	Specifies an encoding scheme that uses a GS1 Databar Limited format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_OMNI	Specifies an encoding scheme that uses a GS1 Databar format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_STACKED	Specifies an encoding scheme that uses a GS1 Databar Stacked format. INFO	c		e	f	g	a		i		j	k	l	m	n	b

		M_ENC_GS1_DATABAR_STACKED_OMNI	Specifies an encoding scheme that uses a GS1 Databar Stacked Omnidirectional format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_GS1_DATABAR_TRUNCATED	Specifies an encoding scheme that uses a GS1 Databar Truncated format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_ISO8	Specifies a similar encoding scheme as M_ENC_ASCII, but supports the extended ASCII character set. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_KOREA_MAIL	Specifies an encoding scheme for a 4-state format used by the Korean Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE2	Specifies an encoding scheme that requires a Structured Carrier Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE3	Specifies an encoding scheme that requires a Structured Carrier Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE4	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE5	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_MODE6	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_NUM	Specifies an encoding scheme that only supports numbers. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_QRCODE_MODEL1	Specifies an encoding scheme that uses an older version of the QR code format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_QRCODE_MODEL2	Specifies an encoding scheme that uses a newer version of the QR code format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_STANDARD	Specifies different types of encoding schemes, depending on what code type is used. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UK_MAIL	Specifies an encoding scheme for a 4-state format used by the UK Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENC_UPCA	Specifies an encoding scheme for a composite code whose 1D portion uses an UPC-A format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENC_UPCA_ADDON	Specifies an encoding scheme for an UPC-A format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_UPCE	Specifies an encoding scheme for a composite code whose 1D portion uses an UPC-E format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_UPCE_ADDON	Specifies an encoding scheme for an UPC-E format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_US_MAIL	Specifies the Intelligent Mail Barcode encoding scheme for a 4-state format used by the US Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	5 <= *Value* <= 95	Specifies the minimum amount of the symbol that contains error correction information, as a percentage. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ERROR_CORRECTION +		Retrieves the type of error correction scheme of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_4STATE	Specifies the Reed-Solomon-based algorithm or a check digit type of error correction scheme, depending on the specification of the encoding. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_200	Specifies a Reed-Solomon-based algorithm as an error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_CHECK_DIGIT	Specifies an additional digit to check whether there is an error or not. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_COMPOSITE	Specifies the default error correction scheme for the 1D and 2D portions of the composite code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_H	Specifies the highest-level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_L	Specifies the lowest-level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_M	Specifies a medium-low level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_NONE	Specifies no error correction. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_Q	Specifies a medium-high level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_ECC_REED_SOLOMON	Specifies a Reed-Solomon type of error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_REED_SOLOMON_*n*	Specifies a Reed-Solomon type of error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_UNKNOWN	Specifies an unknown error correction scheme.	c		e	f	g	a		i		j	k	l	m	n	b
	5 <= *Value* <= 95	Specifies the minimum percentage of the symbol that contains error correction information. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_IS_GS1 +		Retrieves whether the specified code occurrence(s) follows the industry standard for a GS1 code. This result type is available for Aztec, Data Matrix, Code 128, EAN 14, GS1-128, GS1-Databar, QR code, DotCode, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the specified code occurrence(s) does not follow the industry standard for a GS1 code.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the specified code occurrence(s) follows the industry standard for a GS1 code.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_X +		Retrieves the X-coordinate of the specified code occurrence(s). For all types of codes, except composite code types, this result is the center of the code in the X-direction. For composite codes, this result is the center, in the X-direction, of the 1D component of the code. Note that to retrieve the top-left and bottom-right coordinates, use M_BOTTOM... and M_TOP....	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_Y +		Retrieves the Y-coordinate of the specified code occurrence(s). For all types of codes, except composite code types, this result is the center of the code in the Y-direction. For composite codes, this result is the center, in the Y-direction, of the 1D component of the code. Note that to retrieve the top-left and bottom-right coordinates, use M_BOTTOM... and M_TOP....	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_QUIET_ZONE_BOTTOM_LEFT_X +	Retrieves the bottom-left position along the X-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_BOTTOM_LEFT_Y +	Retrieves the bottom-left position along the Y-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_BOTTOM_RIGHT_X +	Retrieves the bottom-right position along the X-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_BOTTOM_RIGHT_Y +	Retrieves the bottom-right position along the Y-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_TOP_LEFT_X +	Retrieves the top-left position along the X-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_TOP_LEFT_Y +	Retrieves the top-left position along the Y-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_TOP_RIGHT_X +	Retrieves the top-right position along the X-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_QUIET_ZONE_TOP_RIGHT_Y +	Retrieves the top-right position along the Y-axis of the specified code occurrence(s), including the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_SIZE_X +	Retrieves the size of the specified code occurrence(s) in the X-direction. For composite code types, this result is the width of the 1D component of the code.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Y +	Retrieves the size of the specified code occurrence(s) in the Y-direction. For composite code types, this result is the height of the 1D component of the code.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING +	Retrieves the decoded string of the specified code occurrence(s). If the string is decoded from a composite code model, the string is returned in the following format: 1D string|2D string, where | is used as the separator. Note that for this result type, ResultIndex cannot be set to M_ALL or M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR*	c		e	f	g	a		i		j	k	l	m	n	b
M_TOP_LEFT_X +	Retrieves the X-coordinate of the top-left corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TOP_LEFT_Y +	Retrieves the Y-coordinate of the top-left corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TOP_RIGHT_X +	Retrieves the X-coordinate of the top-right corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TOP_RIGHT_Y +	Retrieves the Y-coordinate of the top-right corner of the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_WRITE_SIZE_X +	Specifies the minimum width for the destination image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_WRITE_SIZE_Y +	Specifies the minimum height for the destination image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve a row result that is returned from an McodeRead() or McodeGrade() operation, the ResultCodeId and ResultType parameters can be set to the following values. In this case, the ResultIndex parameter must be set to the index of a specific occurrence or M_ALL, and the RowOrScanIndex parameter must be set to the index of a specific row or to M_ALL (or M_DEFAULT). If RowOrScanIndex is set to an index of a specific row, ResultIndex cannot be set to M_ALL; it must be set to an index of a specific occurrence.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving row-specific results from an McodeRead() or McodeGrade() operation																	
ResultCodeId ResultType UserVarArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Code grade result buffer ID for row-specific results	Specifies a code grade result buffer, allocated using McodeAllocResult() with M_CODE_GRADE_RESULT, and used to store McodeGrade() results.															
M_ROW_NUMBER_OF_DECODED_SCANS +	Retrieves the number of successfully decoded scanlines per row, for the specified row(s) within the specified code occurrence(s). For composite code types, this is the number of scanlines per row in the 1D portion of the code occurrence(s). This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the code occurrence.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of successfully decoded scanlines for the row.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROW_NUMBER_OF_SCANS +	Retrieves the number of scanlines analyzed per row, for the specified row(s) within the specified code occurrence(s). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the code occurrence.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of scanlines analyzed for the row.	c		e	f	g	a		i		j	k	l	m	n	b

M_ROW_OVERALL_GRADE +	Retrieves the overall grade per row, for the specified row(s) within the specified code occurrence(s). To retrieve the average of all the row overall grades, use M_OVERALL_SYMBOL_GRADE. For stacked sub-types of the GS1 Databar code type, the overall row grade is the overall grade per stack of the code. For all other 1D code types, the overall row grade is the overall grade for the entire code. For 2D cross-row code types, the overall row grade is the worst grade of the row. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the code occurrence.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 4.0	Specifies the overall grade for the row.	c		e	f	g	a		i		j	k	l	m	n	b
Code read result buffer ID for row-specific results	Specifies a code read result buffer, allocated using McodeAllocResult() with M_CODE_READ_RESULT, and used to store McodeRead() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROW_NUMBER_OF_DECODED_SCANS +	Retrieves the number of successfully decoded scanlines per row, for the specified row(s) within the specified code occurrence(s). For composite code types, this is the number of scanlines per row in the 1D portion of the code occurrence(s). This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve a scanline result that is returned from an McodeRead() or McodeGrade() operation, the ResultCodeId and ResultType parameters can be set to the following values. In this case, the ResultIndex parameter must be set to the index of a specific occurrence or M_ALL, and the RowOrScanIndex parameter must be set to the index of a specific scanline or to M_ALL (or M_DEFAULT). If RowOrScanIndex is set to an index of a specific scanline, ResultIndex cannot be set to M_ALL; it must be set to an index of a specific occurrence.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving scanline-specific results from an McodeRead() or McodeGrade() operation																
ResultCodeId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultType																
UserVarArrayPtr - Possible values returned																
Code grade result buffer ID for scanline-specific results	Specifies a code grade result buffer, allocated using McodeAllocResult() with M_CODE_GRADE_RESULT, and used to store McodeGrade() results.															

M_DECODED_SCANS_END_X +	Retrieves the X-coordinate of the end point of the specified decoded scanline(s), for the specified code occurrence(s). For composite code types, this is for the scanline(s) in the 1D portion of the code occurrence(s). This result type is available for all 1D and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DECODED_SCANS_END_Y +	Retrieves the Y-coordinate of the end point of the specified decoded scanline(s), for the specified code occurrence(s). For composite code types, this is for the scanline(s) in the 1D portion of the code occurrence(s). This result type is available for all 1D and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DECODED_SCANS_SCORE +	Retrieves the score of the specified decoded scanline(s) of the specified code occurrence(s). For composite code types, this is for the scanline(s) in the 1D portion of the code occurrence(s). This result type is available for all 1D and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the score of the decoded scanline.	c		e	f	g	a		i		j	k	l	m	n	b
M_DECODED_SCANS_START_X +	Retrieves the X-coordinate of the start point of the specified decoded scanline(s), for the specified code occurrence(s). For composite code types, this is for the scanline(s) in the 1D portion of the code occurrence(s). This result type is available for all 1D and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DECODED_SCANS_START_Y +	Retrieves the Y-coordinate of the start point of the specified decoded scanline(s), for the specified code occurrence(s). For composite code types, this is for the scanline(s) in the 1D portion of the code occurrence(s). This result type is available for all 1D and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_SCAN_DECODABILITY +	Retrieves the decodability result for the scan reflectance profile. To retrieve the decodability for each codeword, use M_CODEWORD_DECODABILITY. To retrieve the decodability of the specified code occurrence(s) as a grade, use M_DECODABILITY_GRADE. Note that certain code types have additional decodability measures that are taken into account when computing the decodability result. See the ISO 15417 (Code 128) and ISO 15420 (EAN and UPC code types) specifications. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the decodability value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_DECODABILITY_GRADE +	Retrieves the decodability result for the scan reflectance profile, as a grade. When M_GRADING_STANDARD_EDITION is set to M_ISO_15416_2016, M_ISO_15415_2011_15416_2016, or M_DEFAULT, the grade is retrieved as an interpolated value. When M_GRADING_STANDARD_EDITION is set to a grading standard edition prior to ISO/IEC 15416:2016, the grade is retrieved as a letter grade. To retrieve the decodability for each codeword as a grade, use M_CODEWORD_DECODABILITY_GRADE. To retrieve the decodability for a cross-row code occurrence as a grade, use M_DECODABILITY_GRADE. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 4.0	Specifies the decodability grade as an interpolated value rounded to the nearest 0.1 when M_GRADING_STANDARD_EDITION is set to M_ISO_15416_2016, M_ISO_15415_2011_15416_2016, or M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_DECODE_GRADE +		Retrieves whether the decoding algorithm succeeded or failed for the scan reflectance profile, as a grade. To retrieve the error status of the McodeGrade() operation, use M_STATUS. To retrieve the confidence score of the McodeGrade() operation, use M_SCORE. Note that M_SCAN_EDGE_DETERMINATION_GRADE is used in calculating this value. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies that the decoding algorithm succeeded.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies that the decoding algorithm failed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_DEFECTS +		Retrieves a measure of the defects in the scan reflectance profile. The result is based on the ERN maximum (M_SCAN_ERN_MAXIMUM) and the symbol contrast value (M_SCAN_SYMBOL_CONTRAST). To retrieve the measure for each codeword, use M_CODEWORD_DEFECTS. To retrieve the measure for the specified code occurrence(s) as a grade, use M_DEFECTS_GRADE. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies a measure of the defects. A value of 1.0 indicates that the scan is perfect; whereas, a value of 0.0 indicates that scan is very bad.	c		e	f	g	a		i		j	k	l	m	n	b

M_SCAN_DEFECTS_GRADE +	Retrieves a measure of the defects in the scan reflectance profile, as a grade. When M_GRADING_STANDARD_EDITION is set to M_ISO_15416_2016, M_ISO_15415_2011_15416_2016, or M_DEFAULT, the grade is retrieved as an interpolated value. When M_GRADING_STANDARD_EDITION is set to a grading standard edition prior to ISO/IEC 15416:2016, the grade is retrieved as a letter grade. To retrieve the measure of defects in each codeword, as a grade, use M_CODEWORD_DEFECTS_GRADE. To retrieve the measure of defects in the specified code occurrence(s), use M_DEFECTS_GRADE. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 4.0	Specifies the defects grade as an interpolated value rounded to the nearest 0.1 when M_GRADING_STANDARD_EDITION is set to M_ISO_15416_2016, M_ISO_15415_2011_15416_2016, or M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_EDGE_CONTRAST_MINIMUM +	Retrieves the minimum edge contrast value of the scan reflectance profile. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

	0.0 <= *Value* <= 1.0	Specifies the minimum edge contrast value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_EDGE_CONTRAST_MINIMUM_GRADE +		Retrieves the minimum edge contrast value of the scan reflectance profile, as a grade. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_EDGE_DETERMINATION_GRADE +		Retrieves the edge determination of the scan reflectance profile, as a grade. Edge determination examines the number of bars and spaces in your code. The presence of scratches on the bars and spaces, including the quiet zone, could affect this value. Note that this grade is not considered when calculating the overall code grade (M_OVERALL_SYMBOL_GRADE), instead it is included in the decode grade (M_SCAN_DECODE_GRADE). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

M_SCAN_EDGE_DETERMINATION_WARNING +	Retrieves the edge determination warning of the scan reflectance profile when M_GRADING_STANDARD_EDITION is set to M_ISO_15416_2016, M_ISO_15415_2011_15416_2016, or M_DEFAULT. An edge determination warning is issued when the minimum reflectance margin for any element is less than 5% of the symbol contrast value. This warning might indicate that the symbol is close to a failing grade for edge determination. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that an edge determination warning was not issued.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that an edge determination warning was issued.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_ERN_MAXIMUM +	Retrieves the highest ERN (element reflectance non-uniformity) in the scan reflectance profile. The ERN is an unexpected peak in the scan reflectance profile. The result is expressed as the ratio of the maximum ERN relative to the symbol contrast value (ERN^{max}/SC). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the element reflectance non-uniformity maximum.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_GUARD_PATTERN +	Retrieves the size of the interior guard pattern, expressed as a factor of the cell (module size (*Z*)), in the scan reflectance profile. Guard patterns consist of two one-cell wide groupings at the end of the specified code occurrence(s). GS1 Databar code types have guard patterns at the ends of each row of the specified code occurrence(s). See the ISO 24724 specification. Note that the value is negative if the left guard pattern is greater than the right guard pattern, to permit easy identification. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D GS1 Databar code types and composite code types containing a GS1 Databar code.	c		e	f	g	a		i		j	k	l	m	n	b

		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* | *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		Value >= 0.0	Specifies the size of the interior guard pattern, expressed as a factor of the cell (module size (*Z*)) in the scan reflectance profile.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SCAN_GUARD_PATTERN_GRADE +		Retrieves the largest interior guard pattern in the scan reflectance profile, as a grade. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D GS1 Databar code types and composite code types containing a GS1 Databar code.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* | *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	M_SCAN_INTERCHARACTER_GAP +		Retrieves the size of the largest inter-character gap in the scan reflectance profile, expressed as a factor of the cell size (module size (*Z*)) in the scan reflectance profile. See ISO 16388 and AIM BC3-1995 for more details. Note that this result type will always return the value of M_CODE_GRADE_NOT_AVAILABLE when McodeControl() with M_PIXEL_SIZE_IN_MM is not set, and the following condition is true: (5.3 *Z*<= *Largest inter-character gap* <= 3*Z*). When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for Code 39 and Codabar code types.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* | *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

	Value >= 0.0	Specifies the largest inter-character gap.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_INTERCHARACTER_GAP_GRADE +		Retrieves the largest inter-character gap in the scan reflectance profile, as a grade. The largest inter-character gap is calculated as a factor of the cell size (module size (*Z*)) in the scan reflectance profile. See ISO 16388 and AIM BC3-1995 for more details. Note that this result type will always return the value of M_CODE_GRADE_NOT_AVAILABLE when McodeControl() with M_PIXEL_SIZE_IN_MM is not set, and the following condition is true: (5.3 *Z*<= *Largest inter-character gap* <= 3*Z*). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for Code 39 and Codabar code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies that the maximum inter-character gap is under three times the module size (3Z).	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies that the maximum inter-character gap is over 5.3 times the module size (5.3Z).	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_MINIMUM_REFLECTANCE_MARGIN +		Retrieves the minimum reflectance margin found in the scan reflectance profile when M_GRADING_STANDARD_EDITION is set to M_ISO_15416_2016, M_ISO_15415_2011_15416_2016, or M_DEFAULT. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the minimum reflectance margin.	c		e	f	g	a		i		j	k	l	m	n	b

M_SCAN_MODULATION +	Retrieves the modulation (MOD) of the scan reflectance profile. The modulation is the ratio of the minimum edge contrast value (M_SCAN_EDGE_CONTRAST_MINIMUM) to the symbol contrast value (M_SCAN_SYMBOL_CONTRAST). To retrieve the modulation of each codeword, use M_CODEWORD_MODULATION. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 2D cross-row and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the modulation value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_MODULATION_GRADE +	Retrieves the modulation (MOD) of the scan reflectance profile, as a grade. When M_GRADING_STANDARD_EDITION is set to M_ISO_15416_2016, M_ISO_15415_2011_15416_2016, or M_DEFAULT, the grade is retrieved as an interpolated value. When M_GRADING_STANDARD_EDITION is set to a grading standard edition prior to ISO/IEC 15416:2016, the grade is retrieved as a letter grade. To retrieve the modulation of each codeword as a grade, use M_CODEWORD_MODULATION_GRADE. To retrieve the modulation for the specified code occurrence(s) as a grade, use M_MODULATION_GRADE. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

	0.0 <= *Value* <= 4.0	Specifies the modulation grade as an interpolated value rounded to the nearest 0.1 when M_GRADING_STANDARD_EDITION is set to M_ISO_15416_2016, M_ISO_15415_2011_15416_2016, or M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_PRINT_CONTRAST_SIGNAL +		Retrieves the print contrast signal (PCS) of the scan reflectance profile. Note that this is an approximation based on the maximum reflectance (M_SCAN_REFLECTANCE_MAXIMUM) and the symbol contrast value (M_SCAN_SYMBOL_CONTRAST). For more details, see the ISO/IEC 15416 specification. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the print contrast signal.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_PROFILE_END_X +		Retrieves the X-coordinate of the end of the scan reflectance profile. This result type relates to the position of the scan profiles, enabling you to visualize or draw the available analysis profiles using McodeDraw() with M_DRAW_SCAN_PROFILES. Note that the start and stop positions of any given scan are related to the steps of the analysis were performed. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the X-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_PROFILE_END_Y +		Retrieves the Y-coordinate of the end of the scan reflectance profile. This result type relates to the position of the scan profiles, enabling you to visualize or draw the available analysis profiles using McodeDraw() with M_DRAW_SCAN_PROFILES. Note that the start and stop positions of any given scan are related to the steps of the analysis were performed. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b

		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		Value >= 0.0	Specifies the Y-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SCAN_PROFILE_START_X +		Retrieves the X-coordinate of the start of the scan reflectance profile. This result type relates to the position of the scan profiles, enabling you to visualize or draw the available analysis profiles using McodeDraw() with M_DRAW_SCAN_PROFILES. Note that the start and stop positions of any given scan are related to the steps of the analysis were performed. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		Value >= 0.0	Specifies the X-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SCAN_PROFILE_START_Y +		Retrieves the Y-coordinate of the start of the scan reflectance profile. This result type relates to the position of the scan profiles, enabling you to visualize or draw the available analysis profiles using McodeDraw() with M_DRAW_SCAN_PROFILES. Note that the start and stop positions of any given scan are related to the steps of the analysis were performed. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
		Value >= 0.0	Specifies the Y-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b

M_SCAN_QUIET_ZONE +	Retrieves the ratio, for the scan reflectance profile, between the expected quiet zone of the theoretical code model and the measured quiet zone of the specified code occurrence(s) (*measured quiet zone size / expected quiet zone size*). The measurement is taken from both sides. Note that if the returned value is negative, the left quiet zone is worse than the right. Refer to the appropriate specifications for more details (ISO 15417, ISO 15420, ISO 16388, and ISO 16390, respectively). When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D code types (except GS1 Databar) and composite code types encoded with a format of EAN/UPC or UCC/EAN-128.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the ratio.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_QUIET_ZONE_GRADE +	Retrieves the quiet zone size of the scan reflectance profile, as a grade. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D code types (except GS1 Databar) and composite code types encoded with a format of EAN/UPC or UCC/EAN-128.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

M_SCAN_REFLECTANCE_MAXIMUM +	Retrieves the highest reflectance (R_{max}) of the scan reflectance profile, as a percentage. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the highest reflectance (Rmax) of the scan reflectance profile, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_REFLECTANCE_MINIMUM +	Retrieves the lowest reflectance (R_{min}) of the scan reflectance profile, as a percentage. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the lowest reflectance (Rmin) of the scan reflectance profile, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_REFLECTANCE_MINIMUM_GRADE +	Retrieves a pass or fail grade for the scan reflectance profile. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies that the scan reflectance profile passed.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies that the scan reflectance profile failed.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b

M_SCAN_REFLECTANCE_PROFILE_GRADE +	Retrieves for the scan reflectance profile, its lowest grade among all its M_SCAN_..._GRADE results (except for M_SCAN_EDGE_DETERMINATION_GRADE). When M_GRADING_STANDARD_EDITION is set to M_ISO_15416_2016, M_ISO_15415_2011_15416_2016, or M_DEFAULT, the grade is retrieved as an interpolated value. When M_GRADING_STANDARD_EDITION is set to a grading standard edition prior to ISO/IEC 15416:2016, the grade is retrieved as a letter grade. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 4.0	Specifies the scan reflectance profile grade as an interpolated value rounded to the nearest 0.1 when M_GRADING_STANDARD_EDITION is set to M_ISO_15416_2016, M_ISO_15415_2011_15416_2016, or M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_REFLECTANCE_PROFILE_LENGTH +	Retrieves the number of values in the scan reflectance profile. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of values in the scan reflectance profile.	c		e	f	g	a		i		j	k	l	m	n	b

M_SCAN_REFLECTANCE_PROFILE_VALUES +	Retrieves the reflectance values in the specified scan reflectance profile(s). This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 255	Specifies a reflectance value of the scan reflectance profile.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_SYMBOL_CONTRAST +	Retrieves the symbol contrast value (*SC*) of the scan reflectance profile. Note that this value is used in conjunction with the maximum reflectance (M_SCAN_REFLECTANCE_MAXIMUM to determine the print contrast signal (M_SCAN_PRINT_CONTRAST_SIGNAL). For more details, see ISO/IEC 15416. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. When dealing with 2D matrix codes, to retrieve the symbol contrast value for the entire code occurrence, use M_SYMBOL_CONTRAST. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the symbol contrast value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_SYMBOL_CONTRAST_GRADE +	Retrieves the symbol contrast value of the scan reflectance profile, as a grade. When M_GRADING_STANDARD_EDITION is set to M_ISO_15416_2016, M_ISO_15415_2011_15416_2016, or M_DEFAULT, the grade is retrieved as an interpolated value. When M_GRADING_STANDARD_EDITION is set to a grading standard edition prior to ISO/IEC 15416:2016, the grade is retrieved as a letter grade. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for 1D, 2D cross-row, and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 4.0	Specifies the symbol contrast grade as an interpolated value rounded to the nearest 0.1 when M_GRADING_STANDARD_EDITION is set to M_ISO_15416_2016, M_ISO_15415_2011_15416_2016, or M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_WIDE_TO_NARROW_RATIO +		Retrieves the ratio between the average of the widest and the average of the narrowest bar/space, for the scan reflectance profile of the specified code occurrence(s). See ISO 16388 and ISO 16390 for more details. When dealing with results that are derived from other results, retrieve the results for the associated results first to narrow down where your code occurrence (or code context configuration) is lacking. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for Code 39, Codabar, Industrial 2 of 5, and Interleaved 2 of 5.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the ratio of the average of the widest and the average of the narrowest bar/space in the scan reflectance profile.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_WIDE_TO_NARROW_RATIO_GRADE +		Retrieves the wide to narrow ratio, for the scan reflectance profile of the specified code occurrence(s), as a grade. This result type is only available if McodeControl() with M_GRADING_STANDARD is set to M_ISO_GRADING (the default) before McodeGrade() is called. This result type is available for Code 39, Codabar, Industrial 2 of 5, and Interleaved 2 of 5.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_A	Specifies the best grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_B	Specifies a good grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b

	M_CODE_GRADE_C	Specifies a fair grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_D	Specifies a poor grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_F	Specifies the worst grade for the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE_GRADE_NOT_AVAILABLE	Specifies that the result is not available for the specified code occurrence(s).	c		e	f	g	a		i		j	k	l	m	n	b
Code read result buffer ID for scanline-specific results		Specifies a code read result buffer, allocated using McodeAllocResult() with M_CODE_READ_RESULT, and used to store McodeRead() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_DECODED_SCANS_END_X +		Retrieves the X-coordinate of the end point of the specified decoded scanline(s), for the specified code occurrence(s). For composite code types, this is for the scanline(s) in the 1D portion of the code occurrence(s). This result type is available for all 1D and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* | *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DECODED_SCANS_END_Y +		Retrieves the Y-coordinate of the end point of the specified decoded scanline(s), for the specified code occurrence(s). For composite code types, this is for the scanline(s) in the 1D portion of the code occurrence(s). This result type is available for all 1D and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* | *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DECODED_SCANS_SCORE +		Retrieves the score of the specified decoded scanline(s) of the specified code occurrence(s). For composite code types, this is for the scanline(s) in the 1D portion of the code occurrence(s). This result type is available for all 1D and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* | *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the score of the decoded scanline.	c		e	f	g	a		i		j	k	l	m	n	b
M_DECODED_SCANS_START_X +		Retrieves the X-coordinate of the start point of the specified decoded scanline(s), for the specified code occurrence(s). For composite code types, this is for the scanline(s) in the 1D portion of the code occurrence(s). This result type is available for all 1D and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* | *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_DECODED_SCANS_START_Y +	Retrieves the Y-coordinate of the start point of the specified decoded scanline(s), for the specified code occurrence(s). For composite code types, this is for the scanline(s) in the 1D portion of the code occurrence(s). This result type is available for all 1D and composite code types.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* | *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve information about the code context or code model control types that were enabled for training using the McodeTrain() operation, the ResultCodeId and ResultType parameters can be set to the following values, where ResultIndex is set to M_GENERAL in the case of a code context, and to a specific model index or M_ALL in the case of a code model. RowOrScanIndex must be set to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving information about the code context or code model control types that were enabled for training using the McodeTrain() operation																
ResultCodeId ResultType UserVarArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Code train result buffer ID for trained control type results	Specifies a code train result buffer, allocated using McodeAllocResult() with M_CODE_TRAIN_RESULT, and used to store McodeTrain() results.															
M_TRAIN_ENABLED_CONTROL_TYPES +	Retrieves the code context or code model control types that were enabled for training.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_ENABLED_CONTROL_TYPES_ORIGINAL_VALUE +	Retrieves the original values of the code context or code model control types that were enabled for training.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_ENABLED_CONTROL_TYPES_STATE +	Retrieves the states of the code context or code model control types that were enabled for training.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_OPTIMIZABLE	Specifies that the original value is the same as the trained value.	c		e	f	g	a		i		j	k	l	m	n	b
M_OPTIMIZABLE	Specifies that the original value is different from the trained value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_ENABLED_CONTROL_TYPES_TRAINED_VALUE +	Retrieves the trained values of the code context or code model control types that were enabled for training.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

	M_TRAINED_CONTROL_TYPES +	Retrieves the code context or code model control types that can be modified by the training results; that is, the code model control types that will be changed after calling McodeControl() with M_RESET_FROM_TRAINED_RESULTS.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when ResultCodeId=Code grade result buffer ID for occurrence-specific results: M_STRING;], [when ResultCodeId=Code read result buffer ID for occurrence-specific results: M_STRING;], [when ResultCodeId=Code write result buffer ID for occurrence-specific results: M_STRING].

You can add the following value to the above-mentioned values to specify whether unprintable characters should be returned.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For use with string result types																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ESCAPE_SEQUENCE	Retrieves the string with its unprintable characters represented by their ASCII character codes. Any back slashes (\) in the string are doubled, while unprintable characters, such as carriage returns, are put in a `\xNN` format, where *NN* is the hexadecimal value of the ASCII character code for the unprintable character. For example, the string "\Hello<CR>" (where <CR> is a carriage return) is retrieved as "\\Hello\x0D".	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when ResultCodeId=Code grade result buffer ID for occurrence-specific results: M_BOTTOM_LEFT_X; M_BOTTOM_LEFT_Y; M_BOTTOM_RIGHT_X; M_BOTTOM_RIGHT_Y; M_CELL_NUMBER_X; M_CELL_NUMBER_Y; M_CELL_SIZE; M_CODEWORD_DECODABILITY; M_CODEWORD_DECODABILITY_GRADE; M_CODEWORD_DEFECTS; M_CODEWORD_DEFECTS_GRADE; M_CODEWORD_MODULATION; M_CODEWORD_MODULATION_GRADE; M_CODEWORD_YIELD; M_CODEWORD_YIELD_GRADE; M_DATA_CODEWORDS; M_DECODABILITY_GRADE; M_DECODE_GRADE; M_DEFECTS_GRADE; M_ELEMENT_NUMBER_X; M_ELEMENT_NUMBER_Y; M_ENCODING; M_ERROR_CORRECTION; M_MODULATION_GRADE; M_NUMBER_OF_CODEWORDS; M_NUMBER_OF_DATA_CODEWORDS; M_NUMBER_OF_ERASURES; M_NUMBER_OF_ERROR_CORRECTION_CODEWORDS; M_NUMBER_OF_ERRORS; M_NUMBER_OF_ROWS; M_NUMBER_OF_SCANS; M_OVERALL_SYMBOL_GRADE; M_POSITION_X; M_POSITION_Y; M_START_STOP_PATTERN_GRADE; M_TOP_LEFT_X; M_TOP_LEFT_Y; M_TOP_RIGHT_X; M_TOP_RIGHT_Y; M_UNUSED_ERROR_CORRECTION; M_UNUSED_ERROR_CORRECTION_GRADE;], [when ResultCodeId=Code read result buffer ID for occurrence-specific results: M_BOTTOM_LEFT_X; M_BOTTOM_LEFT_Y; M_BOTTOM_RIGHT_X; M_BOTTOM_RIGHT_Y; M_CELL_NUMBER_X; M_CELL_NUMBER_Y; M_CELL_SIZE; M_DATA_CODEWORDS; M_ELEMENT_NUMBER_X; M_ELEMENT_NUMBER_Y; M_ENCODING; M_ERROR_CORRECTION; M_NUMBER_OF_CODEWORDS; M_NUMBER_OF_DATA_CODEWORDS; M_NUMBER_OF_ERASURES; M_NUMBER_OF_ERROR_CORRECTION_CODEWORDS; M_NUMBER_OF_ERRORS; M_POSITION_X; M_POSITION_Y; M_TOP_LEFT_X; M_TOP_LEFT_Y; M_TOP_RIGHT_X; M_TOP_RIGHT_Y;], [when ResultCodeId=Code grade result buffer ID for row-specific results: M_ROW_NUMBER_OF_SCANS; M_ROW_OVERALL_GRADE;], [when ResultCodeId=Code grade result buffer ID for scanline-specific results: M_SCAN_DECODABILITY; M_SCAN_DECODABILITY_GRADE; M_SCAN_DECODE_GRADE; M_SCAN_DEFECTS; M_SCAN_DEFECTS_GRADE; M_SCAN_EDGE_CONTRAST_MINIMUM; M_SCAN_EDGE_CONTRAST_MINIMUM_GRADE; M_SCAN_EDGE_DETERMINATION_GRADE; M_SCAN_EDGE_DETERMINATION_WARNING; M_SCAN_ERN_MAXIMUM; M_SCAN_GUARD_PATTERN; M_SCAN_GUARD_PATTERN_GRADE; M_SCAN_MINIMUM_REFLECTANCE_MARGIN; M_SCAN_MODULATION; M_SCAN_MODULATION_GRADE; M_SCAN_PRINT_CONTRAST_SIGNAL; M_SCAN_PROFILE_END_X; M_SCAN_PROFILE_END_Y; M_SCAN_PROFILE_START_X; M_SCAN_PROFILE_START_Y; M_SCAN_QUIET_ZONE; M_SCAN_QUIET_ZONE_GRADE; M_SCAN_REFLECTANCE_MAXIMUM; M_SCAN_REFLECTANCE_MINIMUM; M_SCAN_REFLECTANCE_MINIMUM_GRADE; M_SCAN_REFLECTANCE_PROFILE_GRADE; M_SCAN_REFLECTANCE_PROFILE_LENGTH; M_SCAN_REFLECTANCE_PROFILE_VALUES; M_SCAN_SYMBOL_CONTRAST; M_SCAN_SYMBOL_CONTRAST_GRADE].

You can add the following value to the above-mentioned values to determine the specified result for the 2D component of a composite code occurrence, from an McodeRead() or McodeGrade() operation. Alternatively, you can add a value from For retrieving the specified result for the linear component (1D component) of a composite code occurrence, from an McodeGrade() operation.

If this combination constant is not added to a result type, the results of the linear component (1D component) are returned followed by those of the 2D component. Note that this does not include general results (number of rows, number of scanlines) where results are compounded into a single value.

Note that this combination constant is only available for results of an McodeRead() or McodeGrade() operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the specified result for the 2D component of a composite code occurrence, from an McodeRead() or McodeGrade() operation

Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_2D_COMPONENT	Retrieves the result of the 2D component of a composite code occurrence.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when ResultCodeId=Code grade result buffer ID for occurrence-specific results: M_CODEWORD_DECODABILITY; M_CODEWORD_DECODABILITY_GRADE; M_CODEWORD_DEFECTS; M_CODEWORD_DEFECTS_GRADE; M_CODEWORD_MODULATION; M_CODEWORD_MODULATION_GRADE; M_CODEWORD_YIELD; M_CODEWORD_YIELD_GRADE; M_DECODABILITY_GRADE; M_DECODE_GRADE; M_DEFECTS_GRADE; M_MODULATION_GRADE; M_NUMBER_OF_ROWS; M_NUMBER_OF_SCANS; M_OVERALL_SYMBOL_GRADE; M_START_STOP_PATTERN_GRADE; M_UNUSED_ERROR_CORRECTION; M_UNUSED_ERROR_CORRECTION_GRADE;], [when ResultCodeId=Code grade result buffer ID for row-specific results: M_ROW_NUMBER_OF_SCANS; M_ROW_OVERALL_GRADE;], [when ResultCodeId=Code grade result buffer ID for scanline-specific results: M_SCAN_DECODABILITY; M_SCAN_DECODABILITY_GRADE; M_SCAN_DECODE_GRADE; M_SCAN_DEFECTS; M_SCAN_DEFECTS_GRADE; M_SCAN_EDGE_CONTRAST_MINIMUM; M_SCAN_EDGE_CONTRAST_MINIMUM_GRADE; M_SCAN_EDGE_DETERMINATION_GRADE; M_SCAN_EDGE_DETERMINATION_WARNING; M_SCAN_ERN_MAXIMUM; M_SCAN_GUARD_PATTERN; M_SCAN_GUARD_PATTERN_GRADE; M_SCAN_MINIMUM_REFLECTANCE_MARGIN; M_SCAN_MODULATION; M_SCAN_MODULATION_GRADE; M_SCAN_PRINT_CONTRAST_SIGNAL; M_SCAN_PROFILE_END_X; M_SCAN_PROFILE_END_Y; M_SCAN_PROFILE_START_X; M_SCAN_PROFILE_START_Y; M_SCAN_QUIET_ZONE; M_SCAN_QUIET_ZONE_GRADE; M_SCAN_REFLECTANCE_MAXIMUM; M_SCAN_REFLECTANCE_MINIMUM; M_SCAN_REFLECTANCE_MINIMUM_GRADE; M_SCAN_REFLECTANCE_PROFILE_GRADE; M_SCAN_REFLECTANCE_PROFILE_LENGTH; M_SCAN_REFLECTANCE_PROFILE_VALUES; M_SCAN_SYMBOL_CONTRAST; M_SCAN_SYMBOL_CONTRAST_GRADE].

You can add the following value to the above-mentioned values to determine the specified result of the linear component (1D component) of a composite code occurrence, from an McodeGrade() operation. Alternatively, you can add a value from For retrieving the specified result for the 2D component of a composite code occurrence, from an McodeRead() or McodeGrade() operation.

If this combination constant is not added to a result type, the results of the linear component (1D component) are returned followed by those of the 2D component. Note that this does not include general results (number of rows, number of scanlines) where results are compounded into a single value.

Note that this combination constant is only available for results of an McodeGrade() operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the specified result for the linear component (1D component) of a composite code occurrence, from an McodeGrade() operation																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_LINEAR_COMPONENT	Retrieves the result of the linear component (1D component) of a composite code occurrence.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values of the ResultType parameter listed in For retrieving code model-specific results from an McodeTrain() operation, For retrieving code occurrence-specific results from an McodeDetect(), McodeRead(), McodeGrade(), or McodeWrite() operation, For retrieving row-specific results from an McodeRead() or McodeGrade() operation, For retrieving scanline-specific results from an McodeRead() or McodeGrade() operation, For retrieving information about the code context or code model control types that were enabled for training using the McodeTrain() operation; (exceptions might exist) and for the following values: [when ResultCodeId=Code train result buffer ID for general results: M_CODE_MODEL_ID; M_CODE_RESULT_ID; M_FAILED_IMAGES_ID; M_FAILED_IMAGES_INDEX; M_PASSED_IMAGES_ID; M_PASSED_IMAGES_INDEX; M_TRAIN_ENABLED_NUMBER_OF_MODEL_CONTROL_TYPES; M_TRAINED_NUMBER_OF_MODEL_CONTROL_TYPES;], [when ResultCodeId=Code grade result buffer ID for occurrence-specific results: M_CODEWORD_DECODABILITY; M_CODEWORD_DEFECTS; M_CODEWORD_MODULATION; M_CODEWORD_REFLECTANCE_MARGIN; M_EXTENDED_AREA_CODEWORD_MODULATION; M_FIXED_PATTERN_DAMAGE_B_GRADE; M_UNUSED_ERROR_CORRECTION;], [when ResultCodeId=Code read result buffer ID for occurrence-specific results: M_DATA_CODEWORDS].

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																
ResultType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarArrayPtr - Possible values returned																
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.															

UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when ResultCodeId=Code detect result buffer ID for occurrence-specific results: M_CODE_TYPE_NAME;], [when ResultCodeId=Code grade result buffer ID for occurrence-specific results: M_STRING;], [when ResultCodeId=Code read result buffer ID for occurrence-specific results: M_STRING;], [when ResultCodeId=Code write result buffer ID for occurrence-specific results: M_STRING].

You can add the following value to the above-mentioned values to determine the string length of a string.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the string length																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarArrayPtr - Possible values returned																
M_STRING_SIZE +	Retrieves the length of the string, including the terminating null character ("\0"), and unprintable characters, if the M_ESCAPE_SEQUENCE combination constant is specified.															
UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values of the ResultType parameter listed in For retrieving general results from an McodeDetect(), McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation, For retrieving code model-specific results from an McodeTrain() operation, For retrieving code occurrence-specific results from an McodeDetect(), McodeRead(), McodeGrade(), or McodeWrite() operation, For retrieving row-specific results from an McodeRead() or McodeGrade() operation, For retrieving scanline-specific results from an McodeRead() or McodeGrade() operation, For retrieving information about the code context or code model control types that were enabled for training using the McodeTrain() operation; (exceptions might exist) and for the following values: [when ResultCodeId=Code result buffer ID: M_NUMBER;], and for the following values: M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested results to a required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested information to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]* **|** *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarArrayPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_TEXT_CHAR		Cast the requested results to an *AIL_TEXT_CHAR*.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarArrayPtr extra info	Data type info: Data type: *address of an AIL_TEXT_CHAR* **|** *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcode11.lib.
DLL	Requires ail11.dll; ailcode11.dll.

McodeGrade

Synopsis

Grade the code occurrence(s) in an image.

Syntax

```
void McodeGrade(
    AIL_ID ContextCodeId,                   //in
    AIL_ID ImageBufId,                      //in
    AIL_ID SrcResultCodeId,                 //in
    AIL_INT SrcModelCodeIndexOrResultIndex, //in
    AIL_ID DstResultCodeId,                 //out
    AIL_INT64 ControlFlag                   //in
)
```

Description

This function computes different quality-grades for the code occurrences in an image, as specified in the code context, and writes the results in the destination result buffer. Retrieve results using McodeGetResult(). **McodeGrade**() can also grade code occurrences found using McodeRead().

> Note that **McodeGrade**() does not support M_4_STATE, M_PHARMACODE, M_POSTNET, and M_PLANET code types.

By default, **McodeGrade**() internally performs a code read operation before performing the code grade operation. However, you can skip this step if you pass **McodeGrade**() the result buffer from a previous successful call to McodeRead(); this is more efficient if for some reason, you need to explicitly call McodeRead() prior to calling **McodeGrade**(). In this case for all code types except 2D matrix, ensure that McodeRead() was executed with the control type M_POSITION_ACCURACY set to high to avoid potential failure.

Since the global context settings are required, **McodeGrade**() requires that you pass a code context identifier, and not a code model identifier. The code context passed to **McodeGrade**() can contain multiple code models depending on their code type (see McodeModel() for details). **McodeGrade**() can grade one or more occurrences of 1D and Data Matrix code models; for other types of code models, **McodeGrade**() can grade at most one occurrence. If a target image is specified and the code context contains multiple code models, you can grade all the occurrences of a specific code model or grade all the occurrences of all the code models (M_ALL). If a source code result buffer is specified and it contains the results of several code occurrences, you can specify the index of the code occurrence to grade or grade all the occurrences (M_ALL).

Before calling **McodeGrade**(), ensure that the following control types are set appropriately for your code occurrences.

Control type	Notes
M_FOREGROUND_VALUE	Specifies the foreground color of the code occurrence. This control type must be set if the foreground color is not black; the code occurrence will not be graded if the foreground value is not correctly set.
M_ENCODING	Specifies the type of encoding scheme for the code occurrence. This control type must be set if the default encoding scheme for the code type differs from the one used by the code occurrence or if M_ANY is not supported.
M_ERROR_CORRECTION	Specifies the type of error correction scheme for the code occurrence. This control type must be set if the default error correction scheme for the code type differs from the one used by the code occurrence or if M_ANY is not supported.
M_STRING_SIZE....	Specifies the maximum and minimum size of the string (number of characters) encoded in each code occurrence. These control types must be set if M_STRING_SIZE_MIN and/or M_STRING_SIZE_MAX cannot be set to M_ANY.
M_SEARCH_ANGLE....	Specifies the nominal search angle and angular search range. These control types must be set if the code occurrence is at an angle greater or less than 0.0 ±5 degrees.

For more information on code model and code context settings that can improve results, see the Customizing read and grade operation settings section of Chapter 17: Codes.

If you have associated the target image with a camera calibration context, you can specify that certain input settings be interpreted in world units. To do so, use McodeControl() with M_ABSOLUTE_APERTURE_SIZE_INPUT_UNITS, M_SCANLINE_INPUT_UNITS, M_DOT_SPACING_INPUT_UNITS, M_SEARCH_ANGLE_INPUT_UNITS, and/or M_CELL_SIZE_INPUT_UNITS set to M_WORLD. Note that if you set any of these constants to M_WORLD but you don't pass **McodeGrade**() a calibrated target image, the function will generate an error. If you pass a source code result buffer to **McodeGrade**() instead of a target image, the code context should have the same settings as the context passed to McodeRead(); the settings should not be changed between the two operations. A source code result buffer (passed to this function) should contain results obtained from a calibrated target image for best results.

It is strongly recommended to use a region of interest (ROI), set using MbufSetRegion(), with **McodeGrade**(), especially if your image contains more code occurrences than your context is configured to grade; otherwise, Aurora Imaging Library will randomly select the specified number of code occurrences to grade. Grading only a portion of your image is also recommended because it achieves a fast and robust **McodeGrade**() operation if your image contains other information besides the code occurrences (for example, text). Note that, if your image has an ROI, you can have the nominal angle set to the same angle as that of the image's ROI, using McodeControl() with M_SEARCH_ANGLE set to M_ACCORDING_TO_REGION.

You should note that if the code occurrence cannot be decoded by Aurora Imaging Library, all grades will be returned as F.

If you cannot read a bar code with a cell size that is size 2 or smaller, you can enlarge the image using MimResize() and try again.

Parameters

ContextCodeId

Specifies the identifier of the code context, allocated using McodeAlloc().

ImageBufId

Specifies the identifier of the image buffer containing the code occurrence(s) to be graded. The image buffer must be an 8-bit, unsigned, 1-band image buffer. Note that, this image buffer should contain the same image used for the previous McodeRead() operation.

This image buffer can have a rectangular region of interest (ROI), defined using MbufSetRegion() (with or without rotation). In addition, the region must be defined in vector format from a 2D graphics list (M_VECTOR or M_VECTOR_AND_RASTER). An error will be generated if the ROI is only in raster format (M_RASTER) or is non-rectangular in shape.

SrcResultCodeId

Specifies a source code result buffer from which to retrieve McodeRead() results. The result buffer must have been allocated using McodeAllocResult() with M_CODE_READ_RESULT. Note that M_GRADING_STANDARD and M_EXTENDED_AREA_REFLECTANCE_CHECK should have been set before the McodeRead() operation. In addition for all code types except 2D matrix, the results must have been obtained from the same source image in high accuracy (McodeControl() M_POSITION_ACCURACY set to M_HIGH). This parameter can be set to **M_NULL**.

SrcModelCodeIndexOrResultIndex

Specifies the index of the code model (in the code context) or the result occurrence (in the source code result buffer) to use when grading.

The index of the code model is determined by the order in which it was added to the code context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the model or the result containing information about the code occurrence to grade																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to grade all occurrences of all code models in the code context or all occurrences of code models in the source code result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies which code model (from the code context) or result index (from the result buffer) to use, when grading the code occurrence.	c		e	f	g	a		i		j	k	l	m	n	b

DstResultCodeId

Specifies the result buffer in which to write the resulting grading information. The result buffer must have been allocated using McodeAllocResult() with M_CODE_GRADE_RESULT.

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcode11.lib.
DLL	Requires ail11.dll; ailcode11.dll.

McodeInquire

Synopsis

Inquire about a code context, code model, or code result setting.

Syntax

```
AIL_INT McodeInquire(
    AIL_ID CodeId,         //in
    AIL_INT64 InquireType, //in
    void *UserVarPtr       //out
)
```

Description

This function inquires about a setting of the specified code context, code model, or code result buffer.

Parameters

CodeId

Specifies the identifier of the code context, code model, or code result buffer.

InquireType

Specifies the setting about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address in which to return the value of the inquired setting. Since the **McodeInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring about code context or result buffer settings
- For inquiring about the number of code models in a code context
- For inquiring about code context settings that affect an McodeRead(), McodeGrade(), or McodeTrain() operation
- For inquiring about code context settings that affect an McodeGrade() operation
- For inquiring about code context settings that affect an McodeTrain() operation
- For inquiring about a code model's code type
- For inquiring about code model settings that affect an McodeRead(), McodeGrade(), or McodeTrain() operation
- For inquiring about code model settings that affect an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation
- For inquiring about code model settings that affect an McodeGrade() operation
- For inquiring about code model settings that affect an McodeWrite() operation
- For inquiring about result buffer settings dealing with an McodeRead(), McodeGrade(), McodeDetect(), McodeTrain(), or McodeWrite() operation
- For inquiring about result buffer settings dealing with an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation

To inquire about general code context settings or code result buffer settings, the InquireType parameter can be set to one of the following values. In this case, set the CodeId parameter to a code context or code result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about code context or result buffer settings

InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_OWNER_SYSTEM +	Inquires the identifier of the system on which the code context or code result buffer has been allocated. SET SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED_CODE_TYPES +	Inquires all code types supported by the module.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_4_STATE	Specifies a 4-state code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_AZTEC	Specifies an Aztec code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_BC412	Specifies a BC412 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODABAR	Specifies a Codabar code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE39	Specifies a Code 39 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE93	Specifies a Code 93 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE128	Specifies a Code 128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPOSITECODE	Specifies a composite code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DATAMATRIX	Specifies a Data Matrix code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DOTCODE	Specifies a DotCode code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_EAN8	Specifies an EAN 8 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_EAN13	Specifies an EAN 13 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_EAN14	Specifies an EAN 14 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_GS1_128	Specifies a GS1-128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_GS1_DATABAR	Specifies a GS1 Databar code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_IATA25	Specifies an IATA 2 of 5 code type.	c		e	f	g	a		i		j	k	l	m	n	b

	M_INDUSTRIAL25	Specifies an Industrial 2 of 5 (standard 2 of 5) code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERLEAVED25	Specifies an Interleaved 2 of 5 (ITF-14) code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAXICODE	Specifies a Maxicode code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MICROPDF417	Specifies a MicroPDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MICROQRCODE	Specifies a Micro QR code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PDF417	Specifies a PDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PHARMACODE	Specifies a Pharmacode code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PLANET	Specifies a Planet code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_POSTNET	Specifies a Postnet code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_QRCODE	Specifies a QR code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUNCATED_PDF417	Specifies a Truncated PDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPC_A	Specifies a UPC-A code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPC_E	Specifies a UPC-E code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED_CODE_TYPES_1D +		Inquires all 1D code types supported by the module.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_BC412	Specifies a BC412 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODABAR	Specifies a Codabar code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE39	Specifies a Code 39 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE93	Specifies a Code 93 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE128	Specifies a Code 128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_EAN8	Specifies an EAN 8 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_EAN13	Specifies an EAN 13 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_EAN14	Specifies an EAN 14 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GS1_128	Specifies a GS1-128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GS1_DATABAR	Specifies a GS1 Databar code type.	c		e	f	g	a		i		j	k	l	m	n	b

	M_IATA25	Specifies an IATA 2 of 5 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INDUSTRIAL25	Specifies an Industrial 2 of 5 (standard 2 of 5) code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERLEAVED25	Specifies an Interleaved 2 of 5 (ITF-14) code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PHARMACODE	Specifies a Pharmacode code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPC_A	Specifies a UPC-A code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPC_E	Specifies a UPC-E code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED_CODE_TYPES_2D +		Inquires all 2D code types supported by the module.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_AZTEC	Specifies an Aztec code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DATAMATRIX	Specifies a Data Matrix code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DOTCODE	Specifies a DotCode code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAXICODE	Specifies a Maxicode code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MICROPDF417	Specifies a MicroPDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MICROQRCODE	Specifies a Micro QR code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PDF417	Specifies a PDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_QRCODE	Specifies a QR code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUNCATED_PDF417	Specifies a Truncated PDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED_CODE_TYPES_DETECT +		Inquires all code types supported by McodeDetect().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_BC412	Specifies a BC412 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODABAR	Specifies a Codabar code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE39	Specifies a Code 39 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE93	Specifies a Code 93 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE128	Specifies a Code 128 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_EAN8	Specifies an EAN 8 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_EAN13	Specifies an EAN 13 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_EAN14	Specifies an EAN 14 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_GS1_128	Specifies a GS1-128 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_IATA25	Specifies an IATA 2 of 5 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_INDUSTRIAL25	Specifies an Industrial 2 of 5 (standard 2 of 5) code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERLEAVED25	Specifies an Interleaved 2 of 5 (ITF-14) code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPC_A	Specifies a UPC-A code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPC_E	Specifies a UPC-E code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED_CODE_TYPES_POSTAL +		Inquires all postal code types supported by the module.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_4_STATE	Specifies a 4-state code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PLANET	Specifies a Planet code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_POSTNET	Specifies a Postnet code type.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For inquiring about code context or result buffer settings. (exceptions might exist)

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																	
InquireType combination value		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_NB_ELEMENTS		Retrieves the required array size (number of elements) to store the returned values.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about the number of code models in a code context, set the InquireType parameter to the following value. In this case, set the CodeId parameter to a code context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the number of code models in a code context																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_NUMBER_OF_CODE_MODELS +		Inquires the number of code models in the code context.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	Value >= 0	Specifies the number of code models in the code context.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about general code context settings that affect an McodeRead(), McodeGrade(), or McodeTrain() operation, the InquireType parameter can be set to one of the following values. In this case, set the CodeId parameter to a code context.

> Note that besides code type restrictions listed explicitly in the values below, McodeGrade() does not support M_4_STATE, M_PHARMACODE, M_POSTNET, and M_PLANET code types.

Although the settings of all these inquire types affect the results of McodeTrain(), see the description of the inquire types to determine if their corresponding control type can be activated for training; you can also use filters to limit the inquire types in the table to those whose control type can be trained.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about code context settings that affect an McodeRead(), McodeGrade(), or McodeTrain() operation																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_INITIALIZATION_MODE +		Inquires the initialization mode of the code context. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_TYPICAL_RECOGNITION.	c		e	f	g	a		i		j	k	l	m	n	b
	M_IMPROVED_RECOGNITION	Specifies a code context that might provide a more robust McodeRead(), McodeGrade(), or McodeTrain() operation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_TYPICAL_RECOGNITION	Specifies a code context that might provide a quicker McodeRead() or McodeGrade() operation, but might potentially produce less robust results.	c		e	f	g	a		i		j	k	l	m	n	b
M_MINIMUM_CONTRAST +		Inquires the minimum possible contrast between the foreground and background in the target image. This inquire type is only available for 1D code types (excluding M_PLANET and M_POSTNET), the M_DOTCODE code type, and 2D cross-row code types. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. Note that training the corresponding control type (M_TRAIN) is only supported for the M_DOTCODE code type. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 50. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 255	Specifies the minimum contrast. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SCANLINE_HEIGHT +		Inquires the scanline height (or thickness). This inquire type is only available for 1D code types (excluding M_4_STATE, M_PLANET, and M_POSTNET), 2D cross-row code types, and M_COMPOSITECODE code types. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies a default scanline height that is based on the code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the scanline height, relative to the input coordinate system specified using M_SCANLINE_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SCANLINE_INPUT_UNITS +	Inquires the units with which to interpret the M_SCANLINE_STEP and M_SCANLINE_HEIGHT inquire types. This inquire type is only available for 1D code types (excluding M_4_STATE, M_PLANET, and M_POSTNET), 2D cross-row code types, and M_COMPOSITECODE code types. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the values in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the values in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SCANLINE_STEP +	Inquires the scanline step (or interval). This inquire type is only available for 1D code types (excluding M_4_STATE, M_PLANET, and M_POSTNET), 2D cross-row code types, and M_COMPOSITECODE code types. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies a default scanline step that is based on the value of M_SPEED. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the scanline step, relative to the input coordinate system specified using M_SCANLINE_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_MODE +	Inquires whether the search angular range algorithm for the code context is enabled. Note that the corresponding control type can be automatically activated for training; to inquire if it has been activated for training, use M_SEARCH_ANGLE with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the search angular range algorithm is not used.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the search angular range algorithm is used.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPEED +	Inquires the search speed. This inquire type is available for all code types except M_DOTCODE. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the initialization mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_HIGH	Specifies a high search speed.	c		e	f	g	a		i		j	k	l	m	n	b

M_LOW	Specifies a low search speed.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEDIUM	Specifies a medium search speed.	c		e	f	g	a		i		j	k	l	m	n	b
M_VERY_HIGH	Specifies a very high search speed.	c		e	f	g	a		i		j	k	l	m	n	b
M_VERY_LOW	Specifies a very low search speed.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_MODE +	Inquires the threshold mode used to internally binarize the source image. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type and the initialization mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ADAPTIVE	Specifies to use a fast dynamic local threshold. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_SEGMENTATION	Specifies to use a global threshold value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_WITH_LOCAL_RESEGMENTATION	Specifies that the source image will be globally thresholded and then the edges in the binarized image are resegmented according to the intensities of the surrounding bars and spaces in the original source image. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_VALUE +	Inquires the threshold value used to internally binarize the source image, depending on the threshold mode. Note that the corresponding control type can be automatically activated for training; to inquire if it has been activated for training, use M_THRESHOLD_MODE with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO_COMPUTE.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_COMPUTE	Specifies the threshold value automatically.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 255	Specifies the threshold value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT +	Inquires the maximum decoding time for an McodeRead() or McodeGrade() operation, in msec. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2000 msec.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that there is no maximum decoding time.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum decoding time, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

M_TOTAL_NUMBER +	Inquires the total number of codes to be read in one source image. Note that this number is limited by the maximum number of occurrences of each code model to read or grade (M_NUMBER). Only 1D code types (excluding M_GS1_DATABAR, M_4_STATE, M_PLANET, and M_POSTNET code types), the M_DOTCODE code type, and the M_DATAMATRIX code type support searching for multiple occurrences. Note that for M_DATAMATRIX and M_DOTCODE code types, M_TOTAL_NUMBER and M_NUMBER always return the same value because a code context can only contain multiple code models of the above-mentioned 1D code types. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to read/grade all code occurrences in the source image, up to the maximum number of occurrences to read/grade of each code model (M_NUMBER).	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the maximum number of codes to read/grade in one source image. INFO	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about general code context settings that affect an McodeGrade() operation, the InquireType parameter can be set to one of the following values. In this case, set the CodeId parameter to a code context.

> Note that besides code type restrictions listed explicitly in the values below, McodeGrade() does not support M_4_STATE, M_PHARMACODE, M_POSTNET, and M_PLANET code types.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about code context settings that affect an McodeGrade() operation																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_ABSOLUTE_APERTURE_SIZE +	Inquires the absolute size (diameter) of the aperture. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the absolute aperture size, relative to the input coordinate system specified using M_ABSOLUTE_APERTURE_SIZE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_ABSOLUTE_APERTURE_SIZE_INPUT_UNITS +	Inquires the units with which to interpret the M_ABSOLUTE_APERTURE_SIZE inquire type. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the value in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the value in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_APERTURE_MODE +		Inquires the way in which the aperture size is determined. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_RELATIVE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ABSOLUTE	Specifies to use the absolute aperture size, set using M_ABSOLUTE_APERTURE_SIZE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable the aperture. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_RELATIVE	Specifies to use a relative aperture size, based on the cell size (using M_CELL_SIZE...) and the relative aperture factor (using M_RELATIVE_APERTURE_FACTOR).	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_AREA_REFLECTANCE_CHECK +		Inquires whether the grading must perform an additional reflectance check over an area containing the code occurrence and its extended area (that is, 20 times the cell size beyond the quiet zone on all sides). This inquire type is only available for M_AZTEC, M_DATAMATRIX, M_DOTCODE, M_QRCODE, and M_MICROQRCODE code types. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable the additional reflectance check.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable the additional reflectance check. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GRADE_QUIET_ZONE +		Inquires whether to include the quiet zone when performing an McodeGrade() operation. This inquire type is only available for M_DATAMATRIX, M_DOTCODE, M_QRCODE, and M_MICROQRCODE code types. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to exclude the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to include the quiet zone.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRADING_STANDARD +		Inquires the grading standard used when performing an McodeGrade() operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ISO_DPM_GRADING	Specifies an ISO/IEC 29158 specification. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ISO_GRADING	Specifies an ISO/IEC 15416 or ISO/IEC 15415 specification.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SEMI_T10_GRADING	Specifies a Semi T10 specification. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_INSPECTION_BAND_RATIO +	Inquires the height of the inspection band as a percentage of the average bar height. This inquire type is only available for 1D code types, 2D cross-row code types, and M_COMPOSITECODE code types. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default percentage. INFO	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* < 100.0	Specifies the percentage of the average bar height used as the height of the inspection band.	c		e	f	g	a		i		j	k	l	m	n	b
M_LIGHTING_CONFIGURATION +	Inquires the tilted coaxial lighting and camera angle position (TCL) relative to the marked object. This inquire type is only available for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_MICROQRCODE code types. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_90_DEGREE	Specifies the TCL is 90 degrees relative to the marked object. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSPECIFIED	Specifies that the TCL is none of the other values.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAXIMUM_CALIBRATED_REFLECTANCE +	Inquires the maximum possible grayscale value in the target image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 255.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 255	Specifies the maximum calibrated reflectance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN_LIGHT_CALIBRATION +	Inquires the expected mean light (MLcal). This corresponds to the expected mean intensity of the centers of the white elements of the code occurrence. This value is used during the target grading phase of the ISO/IEC 29158 specification. This inquire type is only available for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_MICROQRCODE code types. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is equal to the setting of M_MAXIMUM_CALIBRATED_REFLECTANCE.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 255	Specifies the mean intensity. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MINIMUM_CALIBRATED_REFLECTANCE +	Inquires the minimum possible grayscale value in the target image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 255	Specifies the minimum calibrated reflectance. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_NUMBER_OF_SCANLINES +	Inquires the number of scanlines inside the inspection band to inspect during the McodeGrade() operation, after the code occurrence has been located. This inquire type is only available for 1D code types, 2D cross-row code types, and M_COMPOSITECODE code types. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to use all possible scanlines inside the inspection band.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of scanlines to inspect. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_SIZE_IN_MM +	Inquires the scale between a pixel and its physical measurement, in millimeters per pixel units. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_UNKNOWN.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNKNOWN	Specifies that the scale between a pixel and its physical measurement is not known, in mm per pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the scale between a pixel and its physical measurement, in mm per pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REFLECTANCE_CALIBRATION +	Inquires the expected reflectance value (Rcal). This value is used during the target grading phase of the ISO/IEC 29158 specification. This inquire type is only available for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_MICROQRCODE code types. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is equal to the setting of M_MAXIMUM_CALIBRATED_REFLECTANCE.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 255	Specifies the reflectance value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_APERTURE_FACTOR +	Inquires the relative aperture factor. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the aperture factor is chosen according to the ISO/IEC 15416 specification for all supported code types except 2D matrix code types; for the latter, the aperture factor is chosen according to the ISO/IEC 15415 or ISO/IEC 29158 specification (depends on M_GRADING_STANDARD). INFO	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 2	Specifies the aperture factor.	c		e	f	g	a		i		j	k	l	m	n	b

M_SYSTEM_RESPONSE_CALIBRATION +	Inquires the System Response value derived during the reflectance calibration phase (SRcal) of the ISO/IEC 29158 specification. This inquire type is only available for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_MICROQRCODE code types. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the System Response value for the reference code.	c		e	f	g	a		i		j	k	l	m	n	b
M_SYSTEM_RESPONSE_TARGET +	Inquires the System Response value derived during the target grading phase (SRtarget) of the ISO/IEC 29158 specification. This inquire type is only available for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_MICROQRCODE code types. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the System Response value for the target code.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about general code context settings that affect an McodeTrain() operation, the InquireType parameter can be set to one of the following values. In this case, set the CodeId parameter to a code context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about code context settings that affect an McodeTrain() operation																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TRAIN_TIMEOUT +	Inquires the maximum training time for an McodeTrain() operation. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the timeout.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the timeout limit, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about the code type of a code model, set the InquireType parameter to the following value. In this case, set the CodeId parameter to a code model.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a code model's code type																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_CODE_TYPE +	Inquires the code type of the specified model. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_4_STATE	Specifies a 4-state code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AZTEC	Specifies an Aztec code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BC412	Specifies a BC412 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODABAR	Specifies a Codabar code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE39	Specifies a Code 39 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE93	Specifies a Code 93 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CODE128	Specifies a Code 128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPOSITECODE	Specifies a composite code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DATAMATRIX	Specifies a Data Matrix code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DOTCODE	Specifies a DotCode code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_EAN8	Specifies an EAN 8 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_EAN13	Specifies an EAN 13 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_EAN14	Specifies an EAN 14 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GS1_128	Specifies a GS1-128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GS1_DATABAR	Specifies a GS1 Databar code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_IATA25	Specifies an IATA 2 of 5 code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INDUSTRIAL25	Specifies an Industrial 2 of 5 (standard 2 of 5) code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERLEAVED25	Specifies an Interleaved 2 of 5 (ITF-14) code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAXICODE	Specifies a Maxicode code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MICROPDF417	Specifies a MicroPDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MICROQRCODE	Specifies a Micro QR code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PDF417	Specifies a PDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PHARMACODE	Specifies a Pharmacode code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PLANET	Specifies a Planet code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_POSTNET	Specifies a Postnet code type.	c		e	f	g	a		i		j	k	l	m	n	b

	M_QRCODE	Specifies a QR code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUNCATED_PDF417	Specifies a Truncated PDF417 code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPC_A	Specifies a UPC-A code type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPC_E	Specifies a UPC-E code type.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about code model settings that affect an McodeRead(), McodeGrade(), or McodeTrain() operation, the InquireType parameter can be set to one of the following values. In this case, set the CodeId parameter to a code model.

> Note that besides code type restrictions listed explicitly in the values below, McodeGrade() does not support M_4_STATE, M_PHARMACODE, M_POSTNET, and M_PLANET code types.

Although the settings of all these inquire types affect the results of McodeTrain(), see the description of the inquire types to determine if their corresponding control type can be activated for training; you can also use filters to limit the inquire types in the table to those whose control type can be trained.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about code model settings that affect an McodeRead(), McodeGrade(), or McodeTrain() operation																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_BEARER_BAR +		Inquires whether bearer bars run along the top and bottom of the code occurrences to read (such as, the edge of a sticker). This value is available only for 1D code types. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ABSENT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ABSENT	Specifies that no bearer bars are above and below the code.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PRESENT	Specifies that there are bearer bars above and below the code.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_NUMBER_X_MAX +		Inquires the maximum number of cells for which to search in the X-direction. This inquire type is only available for 2D code types. Note that the corresponding control type can be automatically activated for training; to inquire if it has been activated for training, use M_CELL_NUMBER_X with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ANY	Specifies to search for code occurrences with any number of cells.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the maximum number of cells for which to search. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_CELL_NUMBER_X_MIN +	Inquires the minimum number of cells for which to search in the X-direction. This inquire type is only available for 2D code types. Note that the corresponding control type can be automatically activated for training; to inquire if it has been activated for training, use M_CELL_NUMBER_X with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to search for code occurrences with any number of cells.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the minimum number of cells for which to search. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_NUMBER_Y_MAX +	Inquires the maximum number of cells for which to search in the Y-direction. This inquire type is only available for 2D code types. Note that the corresponding control type can be automatically activated for training; to inquire if it has been activated for training, use M_CELL_NUMBER_Y with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to search for code occurrences with any number of cells.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the maximum number of cells for which to search. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_NUMBER_Y_MIN +	Inquires the minimum number of cells for which to search in the Y-direction. This inquire type is only available for 2D code types. Note that the corresponding control type can be automatically activated for training; to inquire if it has been activated for training, use M_CELL_NUMBER_Y with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to search for code occurrences with any number of cells.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the minimum number of cells for which to search. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE_MAX +	Inquires the maximum cell size. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies to automatically select an appropriate size, which is dependent on the code type and the initialization mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum cell size, relative to the input coordinate system specified using M_CELL_SIZE_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_CELL_SIZE_MIN +	Inquires the minimum cell size. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default cell size, in pixels, which is dependent on the operation and code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the minimum cell size, relative to the input coordinate system specified using M_CELL_SIZE_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CHECK_FINDER_PATTERN +	Inquires whether checking for a false Data Matrix pattern is enabled. This inquire type is only available for the M_DATAMATRIX code type. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the Code module will not check for false M_DATAMATRIX code types.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the Code module will check for false M_DATAMATRIX code types.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHECK_QUIET_ZONE +	Inquires whether the presence of the quiet zone is necessary for a successful McodeRead() or McodeGrade() operation of this code type. Note that the specifications of a code's quiet zone are dependent upon the code type. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that a quiet zone is not necessary. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that a quiet zone is necessary.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_FLIP +	Inquires whether code occurrences need to be flipped or read in the opposite direction to be read properly. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the initialization mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Allows Aurora Imaging Library to decide whether a code occurrence needs to be flipped or read in the opposite direction to be read properly.	c		e	f	g	a		i		j	k	l	m	n	b
M_FLIP	Specifies that code occurrences need to be flipped or read in the opposite direction to be read properly. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_NO_FLIP	Specifies that code occurrences don't need to be flipped or read in the opposite direction. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_SEARCH_MODE +	Inquires the behavior of the decoding algorithm. This inquire type is available for 1D code types (excluding M_4_STATE, M_PLANET, and M_POSTNET), the M_DOTCODE code type, and M_COMPOSITECODE code types. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_BEST	Specifies to search and return the best quality code occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAST	Specifies to search and return the first code occurrences decoded.	c		e	f	g	a		i		j	k	l	m	n	b
M_DATAMATRIX_SHAPE +	Inquires the shape of the Data Matrix code. This inquire type is only available for the M_DATAMATRIX code type. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies that the M_DATAMATRIX code type can be any shape.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECTANGLE	Specifies that the M_DATAMATRIX code has a rectangular shape. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SQUARE	Specifies that the M_DATAMATRIX code has a square shape. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DECODE_ALGORITHM +	Inquires the decoding algorithm used to read the code occurrences. This inquire type is available for M_AZTEC, M_DATAMATRIX, M_DOTCODE, M_QRCODE, M_PDF417, and M_TRUNCATED_PDF417 code types. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. Note that training the corresponding control type (M_TRAIN) is not supported for the M_DOTCODE code type. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type and the initialization mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_DEFORMED	Specifies to use the algorithm to decode deformed code occurrences. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_CODE_NOT_DEFORMED	Specifies to use the algorithm to decode non-deformed code occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCANLINE_AT_ANGLE	Specifies to use the algorithm to decode M_PDF417 or M_TRUNCATED_PDF417 code occurrences using rotated scanlines, without any localization. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SIZE_INPUT_UNITS +	Inquires the units with which to interpret the M_DOT_SIZE_MAX and M_DOT_SIZE_MIN inquire types. This inquire type is only available for the M_DOTCODE code type. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the values in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the values in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SIZE_MAX +	Inquires the maximum diameter of the dots for which to search. This inquire type is only available for the M_DOTCODE code type. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the initialization mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to select an appropriate size, automatically.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the maximum dot size, relative to the input coordinate system specified using M_DOT_SIZE_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SIZE_MIN +	Inquires the minimum diameter of the dots for which to search. This inquire type is only available for the M_DOTCODE code type. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to select an appropriate size, automatically.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the minimum dot size, relative to the input coordinate system specified using M_DOT_SIZE_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_DOT_SPACING_INPUT_UNITS +	Inquires the units with which to interpret the M_DOT_SPACING_MAX and M_DOT_SPACING_MIN inquire types. This inquire type is only available for M_AZTEC, M_DATAMATRIX, M_MAXICODE, M_QRCODE, and M_MICROQRCODE code types. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies what convention is used, automatically.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the value in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the value in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SPACING_MAX +	Inquires the maximum distance between 2 dots in a matrix code type composed of dots. This inquire type is only available for M_AZTEC, M_DATAMATRIX, M_MAXICODE, M_QRCODE, and M_MICROQRCODE code types. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the initialization mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
-256 <= *Value* <= 256	Specifies the distance, relative to the input coordinate system specified using M_DOT_SPACING_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SPACING_MIN +	Inquires the minimum distance between 2 dots in a matrix code type composed of dots. This inquire type is only available for M_AZTEC, M_DATAMATRIX, M_MAXICODE, M_QRCODE, and M_MICROQRCODE code types. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the initialization mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
-256 <= *Value* <= 256	Specifies the distance, relative to the input coordinate system specified using M_DOT_SPACING_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_CORRECTED_NUMBER +	Inquires whether McodeRead() or McodeGrade() are forced to perform a more robust operation to minimize the number of errors to correct. This inquire type is only available for M_PDF417 and M_TRUNCATED_PDF417 code types. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform a more robust operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_ENABLE	Specifies to perform a more robust operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_FINDER_PATTERN_EXHAUSTIVE_SEARCH +	Inquires whether to search for the L-shaped finder pattern (the gray boxed area in the following image) to help localize the Data Matrix code. This inquire type is only available for the M_DATAMATRIX code type. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to not perform an exhaustive search.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform an exhaustive search.	c		e	f	g	a		i		j	k	l	m	n	b
M_FINDER_PATTERN_INPUT_UNITS +	Inquires the units with which to interpret the M_FINDER_PATTERN_EXHAUSTIVE_SEARCH and M_FINDER_PATTERN_MINIMUM_LENGTH inquire types. This inquire type is only available for the M_DATAMATRIX code type. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the values in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the values in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FINDER_PATTERN_MAX_GAP +	Inquires the maximum tolerable gap in the finder pattern of a Data Matrix code. This inquire type is only available for the M_DATAMATRIX code type. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the maximum tolerable gap in the finder pattern is 6 times the minimum cell size specified using M_CELL_SIZE_MIN.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum gap allowed, in input units specified using M_FINDER_PATTERN_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_FINDER_PATTERN_MINIMUM_LENGTH +	Inquires the shortest acceptable length of either "arm" of the finder pattern of a Data Matrix code occurrence. This inquire type is only available for the M_DATAMATRIX code type. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the minimum acceptable finder pattern "arm" length is 6 times the minimum cell size specified using M_CELL_SIZE_MIN.	c		e	f	g	a		i		j	k	l	m	n	b

Value	The minimum acceptable finder pattern "arm" length, in input units specified using M_FINDER_PATTERN_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER +	Inquires the maximum number of codes to be read for the specified code model. This number is also limited by M_TOTAL_NUMBER. Only 1D code types (excluding M_GS1_DATABAR, M_4_STATE, M_PLANET, and M_POSTNET code types), the M_DOTCODE code type, and the M_DATAMATRIX code type support searching for multiple occurrences. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. Note that training the corresponding control type (M_TRAIN) is only supported for the M_DOTCODE code type. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type and the initialization mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies that all code model occurrences are read/graded up to the maximum number limited by M_TOTAL_NUMBER.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number of code occurrences of the specified code model to read or grade. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_ACCURACY +	Inquires the accuracy of positional results. Accuracy depends on the settings of the code context and its models. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type and the initialization mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_HIGH	Specifies to report the positional results of the code operation with high accuracy.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOW	Specifies to report the positional results of the code operation with low accuracy.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE +	Inquires the nominal search angle. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0 degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_REGION	Specifies that the nominal angle is set to the angle of the target image's ROI (MbufSetRegion()); recall that for McodeGrade() and McodeRead(), the ROI must be rectangular. INFO	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the nominal angle, in degrees, relative to the input coordinate system specified using M_SEARCH_ANGLE_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_SEARCH_ANGLE_DELTA_NEG +	Inquires the negative angular range of the search. Note that the corresponding control type can be automatically activated for training; to inquire if it has been activated for training, use M_SEARCH_ANGLE with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type and the initialization mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 180.0	Specifies a negative angular range, in degrees, relative to the nominal angle set by M_SEARCH_ANGLE. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_DELTA_POS +	Inquires the positive angular range of the search. Note that the corresponding control type can be automatically activated for training; to inquire if it has been activated for training, use M_SEARCH_ANGLE with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type and the initialization mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 180.0	Specifies a positive angular range, in degrees, relative to the nominal angle set by M_SEARCH_ANGLE. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_INPUT_UNITS +	Inquires the units with which to interpret the M_SEARCH_ANGLE inquire type. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the value in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the value in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_STEP +	Inquires the angle increment/decrement used when searching for codes through an angular range. This inquire type is available for 1D code types (excluding M_4_STATE, M_PLANET, and M_POSTNET). It is also available for M_PDF417 and M_TRUNCATED_PDF417 code types when M_DECODE_ALGORITHM is set to M_SCANLINE_AT_ANGLE. Note that for M_PDF417 and M_TRUNCATED_PDF417 code types, the corresponding control type can be automatically activated for training; to inquire if it has been activated for training, use M_SEARCH_ANGLE with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that no explicit increment/decrement is used. INFO	c		e	f	g	a		i		j	k	l	m	n	b
0.1 <= *Value* <= 180.0	Specifies the explicit angle increment/decrement, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_SIZE_MAX +	Inquires the maximum size (number of characters) of the string encoded in a code occurrence. For M_QRCODE, M_MICROQRCODE, M_COMPOSITECODE, and M_GS1_DATABAR code types, the maximum string size is ignored when performing McodeRead() and McodeGrade() operations. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies that there is no maximum string size.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 65535	Specifies the maximum string size. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SIZE_MIN +	Inquires the minimum size (number of characters) of the string encoded in the code. For M_QRCODE, M_MICROQRCODE, M_COMPOSITECODE, and M_GS1_DATABAR code types, the minimum string size is ignored when performing McodeRead() and McodeGrade() operations. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies that there is no minimum string size.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 65535	Specifies the minimum string size. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_TYPE +	Inquires the particular code sub-types for which to search. This value is available only for M_GS1_DATABAR and M_COMPOSITECODE code types. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to search for all of the code sub-types that can be specified for M_SUB_TYPE.	c		e	f	g	a		i		j	k	l	m	n	b
M_EAN8	Specifies that the EAN 8 code sub-type is enabled. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_EAN13	Specifies that the EAN 13 code sub-type is enabled. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GS1_128	Specifies that the GS1-128 code sub-type is enabled. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GS1_DATABAR_EXPANDED	Specifies that the GS1 Databar Expanded code sub-type is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
M_GS1_DATABAR_EXPANDED_STACKED	Specifies that the GS1 Databar Expanded Stacked code sub-type is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
M_GS1_DATABAR_LIMITED	Specifies that the GS1 Databar Limited code sub-type is enabled.	c		e	f	g	a		i		j	k	l	m	n	b

M_GS1_DATABAR_OMNI	Specifies that the GS1 Databar Omnidirectional code sub-type is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
M_GS1_DATABAR_STACKED	Specifies that the GS1 Databar Stacked code sub-type is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
M_GS1_DATABAR_STACKED_OMNI	Specifies that the GS1 Databar Stacked Omnidirectional code sub-type is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
M_GS1_DATABAR_TRUNCATED	Specifies that the GS1 Databar Truncated code sub-type is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
M_UPC_A	Specifies that the UPC-A code sub-type is enabled. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_UPC_E	Specifies that the UPC-E code sub-type is enabled. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_PRESEARCH +	Inquires whether the localization operation is performed prior to the decoding step of an operation. This inquire type is available for 2D code types, excluding the M_DOTCODE code type. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type and the initialization mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the operation is not performed.	c		e	f	g	a		i		j	k	l	m	n	b
M_FINDER_PATTERN_BASE	Specifies that the localization operation is only performed on the base pattern of the M_DATAMATRIX code (an "L" starting at the top-most left corner, and ending on the bottom-most right corner of the code). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_BASE	Specifies to localize the code within the image with the statistical characteristics of a 2D bar code (for example, local variance and the presence of a lot of edges). INFO	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about code model settings that affect an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation, the InquireType parameter can be set to one of the following values. In this case, set the CodeId parameter to a code model.

> Note that besides code type restrictions listed explicitly in the values below, McodeGrade() does not support M_4_STATE, M_PHARMACODE, M_POSTNET, and M_PLANET code types.

Although the settings of all these inquire types affect the results of McodeTrain(), see the description of the inquire types to determine if their corresponding control type can be activated for training; you can also use filters to limit the inquire types in the table to those whose control type can be trained.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about code model settings that affect an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

M_CELL_NUMBER_X +	Inquires the number of cells to be used in the X-direction. This inquire type is only available for 2D code types. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. SET														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to search for a code with any number of cells, when performing an McodeRead() or McodeGrade() operation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of cells. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_NUMBER_Y +	Inquires the number of cells to be used in the Y-direction. This inquire type is only available for 2D code types. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to search for a code with any number of cells, when performing an McodeRead() or McodeGrade() operation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of cells. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE_INPUT_UNITS +	Inquires the units with which to interpret the M_CELL_SIZE_MAX and M_CELL_SIZE_MIN inquire types. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the values in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the values in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENCODING +	Inquires the type of encoding scheme. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. Note that training the corresponding control type (M_TRAIN) is not supported for the M_DOTCODE code type. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies to use the default encoding scheme, which is dependent on the code type and the initialization mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies any type of encoding scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENC_ALPHA	Specifies an encoding scheme that supports uppercase alphabetical characters, along with the space. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ALPHANUM	Specifies an encoding scheme that supports alphanumeric characters, as well as the space. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ALPHANUM_PUNC	Specifies a similar encoding scheme to M_ENC_ALPHANUM, except it also supports the following characters: (,), (-), (/) and (. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ASCII	Specifies an encoding scheme that supports ASCII characters. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_AUSTRALIA_MAIL_C	Specifies an encoding scheme for a 4-state format used with the C encoding table by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_AUSTRALIA_MAIL_N	Specifies an encoding scheme for a 4-state format used with the N encoding table by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_AUSTRALIA_MAIL_RAW	Specifies an encoding scheme for a 4-state format used by the Australian Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_AZTEC_COMPACT	Specifies an encoding scheme for a compact Aztec code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_AZTEC_FULL_RANGE	Specifies an encoding scheme for a full-range (not compact) Aztec code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_AZTEC_RUNE	Specifies an encoding scheme for an Aztec rune (the smallest version of an Aztec code). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_EAN8	Specifies an encoding scheme for a composite code whose 1D portion uses an EAN 8 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_EAN8_ADDON	Specifies an encoding scheme for an EAN 8 format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_EAN13	Specifies an encoding scheme for a composite code whose 1D portion uses an EAN 13 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_EAN13_ADDON	Specifies an encoding scheme for an EAN 13 format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_GS1_128_MICROPDF417	Specifies an encoding scheme for a composite code whose 1D portion uses a GS1 128 format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_GS1_128_PDF417	Specifies an encoding scheme for a composite code whose 1D portion uses a GS1 128 format and whose 2D portion uses a PDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_GS1_DATABAR_EXPANDED	Specifies an encoding scheme that uses a GS1 Databar format. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENC_GS1_DATABAR_EXPANDED_STACKED	Specifies an encoding scheme that uses a GS1 Databar Expanded Stacked format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_GS1_DATABAR_LIMITED	Specifies an encoding scheme that uses a GS1 Databar Limited format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_GS1_DATABAR_OMNI	Specifies an encoding scheme that uses a GS1 Databar format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_GS1_DATABAR_STACKED	Specifies an encoding scheme that uses a GS1 Databar Stacked format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_GS1_DATABAR_STACKED_OMNI	Specifies an encoding scheme that uses a GS1 Databar Stacked Omnidirectional format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_GS1_DATABAR_TRUNCATED	Specifies an encoding scheme that uses a GS1 Databar Truncated format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_ISO8	Specifies a similar encoding scheme as M_ENC_ASCII, but supports the extended ASCII character set. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_KOREA_MAIL	Specifies an encoding scheme for a 4-state format used by the Korean Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_MODE2	Specifies an encoding scheme that requires a Structured Carrier Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_MODE3	Specifies an encoding scheme that requires a Structured Carrier Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_MODE4	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_MODE5	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_MODE6	Specifies an encoding scheme that requires a Free Format Message. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_NUM	Specifies an encoding scheme that only supports numbers. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_QRCODE_MODEL1	Specifies an encoding scheme that uses an older version of the QR code format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_QRCODE_MODEL2	Specifies an encoding scheme that uses a newer version of the QR code format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_STANDARD	Specifies different types of encoding schemes, depending on what code type is used. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_UK_MAIL	Specifies an encoding scheme for a 4-state format used by the UK Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENC_UPCA	Specifies an encoding scheme for a composite code whose 1D portion uses an UPC-A format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_UPCA_ADDON	Specifies an encoding scheme for an UPC-A format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_UPCE	Specifies an encoding scheme for a composite code whose 1D portion uses an UPC-E format and whose 2D portion uses a MicroPDF417 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_UPCE_ADDON	Specifies an encoding scheme for an UPC-E format with a supplemental 2 or 5 digit add-on. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENC_US_MAIL	Specifies the Intelligent Mail Barcode encoding scheme for a 4-state format used by the US Mail service. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	5 <= *Value* <= 95	Specifies the minimum amount of the symbol that contains error correction information, as a percentage. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ERROR_CORRECTION +		Inquires the type of error correction. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. Note that training the corresponding control type (M_TRAIN) is not supported for the M_DOTCODE code type. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies to use the default error correction scheme for the code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ANY	Specifies that the error correction type for McodeRead() and McodeGrade() operations is detected automatically. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_4STATE	Specifies to use the Reed-Solomon-based algorithm or a check digit type of error correction scheme, depending on the specification of the encoding. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_200	Specifies to use a Reed-Solomon-based algorithm as an error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_CHECK_DIGIT	Specifies to use an additional digit to check whether there is an error or not. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_COMPOSITE	Specifies to use the default error correction scheme for the 1D and 2D portions of the composite code. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_H	Specifies to use the highest-level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_L	Specifies to use the lowest-level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_M	Specifies to use a medium-low level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ECC_NONE	Specifies no error correction. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_ECC_Q	Specifies to use a medium-high level error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_REED_SOLOMON	Specifies to use a Reed-Solomon type of error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ECC_REED_SOLOMON_*n*	Specifies to use a Reed-Solomon type of error correction scheme. INFO	c		e	f	g	a		i		j	k	l	m	n	b
5 <= *Value* <= 95	Specifies the minimum percentage of the symbol that contains error correction information. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_VALUE +	Inquires the foreground color of the code. Note that the corresponding control type can be trained; to inquire if it has been activated for training, combine this inquire type with M_TRAIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type and the initialization mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_ANY	Specifies the foreground color as black or white. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_BLACK	Specifies that the foreground color is black.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_WHITE	Specifies that the foreground color is white.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_MINIMUM_CONTRAST; M_SPEED; M_THRESHOLD_MODE; M_CELL_SIZE_MAX; M_CELL_SIZE_MIN; M_CODE_FLIP; M_CODE_SEARCH_MODE; M_DATAMATRIX_SHAPE; M_DECODE_ALGORITHM; M_DOT_SIZE_MAX; M_DOT_SIZE_MIN; M_DOT_SPACING_MAX; M_DOT_SPACING_MIN; M_NUMBER; M_SEARCH_ANGLE; M_USE_PRESEARCH; M_CELL_NUMBER_X; M_CELL_NUMBER_Y; M_ENCODING; M_ERROR_CORRECTION; M_FOREGROUND_VALUE.

You can add the following value to the above-mentioned values to determine whether the corresponding control type is activated for training.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring whether the corresponding control type is activated for training																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TRAIN +	Inquires whether the corresponding control type is activated for training. You can establish the default activation setting of the corresponding control type, regardless of its current activation setting, using M_TRAIN + M_DEFAULT.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default activation; the default activation depends on the control type and code model type. For a given trainable control type, if an unsupported code model is included in the training, the default value will be M_DISABLE, otherwise it will be M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the control type has not been activated for training.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the control type has been activated for training.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about code model settings that affect an McodeGrade() operation, set the InquireType parameter to one of the following values. In this case, the CodeId parameter must be set to the identifier of a code model.

> Note that besides code type restrictions listed explicitly in the values below, McodeGrade() does not support M_4_STATE, M_PHARMACODE, M_POSTNET, and M_PLANET code types.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about code model settings that affect an McodeGrade() operation																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_GRADING_STANDARD_EDITION +	Inquires the grading standard edition used when performing an McodeGrade() operation. This inquire type is available for all code types except M_MAXICODE. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ISO_15415_2011_15416_2000	Specifies the ISO/IEC 15415:2011 and ISO/IEC 15416:2000 specifications. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ISO_15415_2011_15416_2016	Specifies the ISO/IEC 15415:2011 and ISO/IEC 15416:2016 specifications. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ISO_15416_2000	Specifies the ISO/IEC 15416:2000 specification. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ISO_15416_2016	Specifies the ISO/IEC 15416:2016 specification. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ISO_29158_2011	Specifies the ISO/IEC 29158:2011 specification. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ISO_29158_2020	Specifies the ISO/IEC 29158:2020 specification. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SEMI_T10_0701	Specifies the Semi T10-0701 specification. INFO	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about code model settings that affect an McodeWrite() operation, set the InquireType parameter to one of the following values. In this case, the CodeId parameter must be set to the identifier of a code model.

> Note that you cannot set CodeId to the identifier of a code context when using the following inquire types.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about code model settings that affect an McodeWrite() operation																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_CELL_SIZE_MODE +	Inquires how to establish the cell size to use when performing an McodeWrite() operation. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to use the largest possible cell size such that the code will be resized as to just fit into the target image of the operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_USER_DEFINED	Specifies that the cell size corresponds to the value of M_CELL_SIZE_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE_VALUE +	Inquires the cell size to use when performing an McodeWrite() operation and M_CELL_SIZE_MODE is set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the cell size. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SHAPE +	Inquires the shape of the dots used to draw foreground cells when performing an McodeWrite() operation. This inquire type is only available for M_AZTEC, M_DATAMATRIX, M_DOTCODE, M_QRCODE, and M_MICROQRCODE code types. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CIRCLE	Specifies that the foreground cells will be drawn with circular dots.	c		e	f	g	a		i		j	k	l	m	n	b
M_SQUARE	Specifies that the foreground cells will be drawn with square dots.	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_SIZE +	Inquires the size of the dot (or square) used to draw foreground cells when performing an McodeWrite() operation. This inquire type is only available for M_AZTEC, M_DATAMATRIX, M_DOTCODE, M_QRCODE, and M_MICROQRCODE code types. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value, which is dependent on the code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the size of the dot or square. INFO	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about code result buffer settings dealing with an McodeRead(), McodeGrade(), McodeDetect(), McodeTrain(), or McodeWrite() operation, set the InquireType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about result buffer settings dealing with an McodeRead(), McodeGrade(), McodeDetect(), McodeTrain(), or McodeWrite() operation																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_RESULT_OUTPUT_UNITS +	Inquires the units with which to interpret the results. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ACCORDING_TO_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

	M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about code result buffer settings dealing with an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation, set the InquireType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about result buffer settings dealing with an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_STRING_FORMAT +		Inquires the format in which to return the string, retrieved using McodeGetResult() with M_STRING. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO_FORMAT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO_FORMAT	Specifies that the string is returned in the string format associated with the code type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_GS1_HUMAN_READABLE	Specifies that the string is returned in a format that is human-readable. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_JAPANESE	[This is only applicable to Windows] Specifies that the string is returned in the Japanese (Windows-932) encoding. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_KOREAN	[This is only applicable to Windows] Specifies that the string is returned in the Korean (Windows-949) encoding. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_LATIN	[This is only applicable to Windows] Specifies that the returned string uses Latin (Windows-1252) encoding. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_RAW_DATA	Specifies that the string is returned in a raw data format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIMPLIFIED_CHINESE	[This is only applicable to Windows] Specifies that the returned string uses Simplified Chinese (Windows-936) encoding. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPCE_COMPRESSED	Specifies that the returned string is in the UPCE compressed string format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_UTF8	Specifies that the returned string is in the UTF-8 string format. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for

Tables:

- For inquiring about code context or result buffer settings
- For inquiring about the number of code models in a code context
- For inquiring about code context settings that affect an McodeRead(), McodeGrade(), or McodeTrain() operation
- For inquiring about code context settings that affect an McodeGrade() operation
- For inquiring about code context settings that affect an McodeTrain() operation
- For inquiring about code model settings that affect an McodeRead(), McodeGrade(), or McodeTrain() operation
- For inquiring about code model settings that affect an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation
- For inquiring about code model settings that affect an McodeGrade() operation
- For inquiring about code model settings that affect an McodeWrite() operation
- For inquiring about result buffer settings dealing with an McodeRead(), McodeGrade(), McodeDetect(), McodeTrain(), or McodeWrite() operation

- For inquiring about result buffer settings dealing with an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation

the values listed in all the parameter association tables except **For inquiring about a code model's code type**.

You can add the following value to the above-mentioned values to determine whether an inquire type is supported.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring whether an inquire type is supported																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_SUPPORTED	Inquires whether the specified inquire type is supported.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For inquiring about code context settings that affect an McodeRead(), McodeGrade(), or McodeTrain() operation, For inquiring about code context settings that affect an McodeGrade() operation, For inquiring about code context settings that affect an McodeTrain() operation, For inquiring about code model settings that affect an McodeRead(), McodeGrade(), or McodeTrain() operation, For inquiring about code model settings that affect an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation, For inquiring about result buffer settings dealing with an McodeRead(), McodeGrade(), McodeDetect(), McodeTrain(), or McodeWrite() operation, For inquiring about result buffer settings dealing with an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation; and for the following value: M_TRAIN.

You can add the following value to the above-mentioned values to determine the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the default value of an inquire type																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Inquires the default value of the specified inquire type.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For inquiring about code context or result buffer settings, For inquiring about the number of code models in a code context, For inquiring about code context settings that affect an McodeRead(), McodeGrade(), or McodeTrain() operation, For inquiring about code context settings that affect an McodeGrade() operation, For inquiring about code context settings that affect an McodeTrain() operation, For inquiring about a code model's code type, For inquiring about code model settings that affect an McodeRead(), McodeGrade(), or McodeTrain() operation, For inquiring about code model settings that affect an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation, For inquiring about code model settings that affect an McodeGrade() operation, For inquiring about code model settings that affect an McodeWrite() operation, For inquiring about result buffer settings dealing with an McodeRead(), McodeGrade(), McodeDetect(), McodeTrain(), or McodeWrite() operation, For inquiring about result buffer settings dealing with an McodeRead(), McodeGrade(), McodeTrain(), or McodeWrite() operation; and for the following value: M_TRAIN.

You can add one of the following values to the above-mentioned values to cast the requested information to a required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

M_TYPE_AIL_DOUBLE	Casts the requested data to an *AIL_DOUBLE*.														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested data to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID* **|** *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested data to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested data to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32* **|** *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested data to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64* **|** *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcode11.lib.
DLL	Requires ail11.dll; ailcode11.dll.

McodeModel

Synopsis

Add, find, or delete a code model within a code context.

Syntax

```
AIL_ID McodeModel(
    AIL_ID ContextCodeId,  //in
    AIL_INT64 Operation,   //in
    AIL_INT64 CodeType,    //in
    AIL_INT Instance,      //in
    AIL_INT64 ControlFlag, //in
    AIL_ID *ModelCodeIdPtr //in-out
)
```

Description

This function adds a code model to a code context, finds a code model of a particular type in a code context, or deletes a code model from a code context.

Specify the code type of the code model when adding the model. A code context can contain multiple code models of 1D code types (excluding M_GS1_DATABAR, M_PLANET, M_POSTNET, and M_4_STATE); for other code types, a code context can contain at most one code model. Although a code context can contain only one 2D code model, you can specify to perform a read, grade, or train operation on more than one occurrence of an M_DATAMATRIX or M_DOTCODE code model (or any 1D code model that is not an M_GS1_DATABAR, M_PLANET, M_POSTNET, and M_4_STATE code model), using McodeControl() with M_NUMBER.

Instead of individually adding models and specifying their code type, you can automatically detect the code type and encoding scheme of most 1D code occurrences in an image using McodeDetect(), and automatically add a code model for each type found. To do so, set the Operation parameter to M_RESET_FROM_DETECTED_RESULTS, and pass the code detect result buffer to the ControlFlag parameter. This first deletes all existing code models in the code context, and then adds a code model for each detected code type. If two code occurrences with the same code type but different encoding schemes are detected, only one code model of this type is added and its encoding scheme is set to M_ANY (if supported by the code type).

Use McodeControl() and McodeInquire() to configure or retrieve information about a code model or code context.

When you add a code model, it is assigned a code model identifier. To control or inquire about the code model, or to use the code model to write a symbol, you need its code model identifier. To retrieve the identifier of an existing code model in a code context (for example, in a restored code context), you can perform an M_FIND operation in one of two ways. If you know the code type, you can specify this and the instance of the code model of this type in the code context. Alternatively, you can pass M_ANY as the code type and specify the index of the code model in the code context. Aside from being associated with a code model identifier, a code model is given a sequential index number within the code context, in the order that the code model was added and starting from 0. If a code model is deleted, all code models with higher indices are shifted down one.

The added code models cannot be individually freed using McodeFree(). They are either automatically freed when their context is freed, or when they are removed from the context by calling **McodeModel**() with M_DELETE.

Parameters

ContextCodeId

Specifies the code context in which to add, find, or delete the code model. The code context must have been previously allocated on the required system using McodeAlloc().

Operation

Specifies the operation to perform within the specified code context. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of operation to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ADD +	Adds a new code model to the code context. Specify the code type of the code model using the CodeType parameter.	c		e	f	g	a		i		j	k	l	m	n	b

M_ADD_IF_NOT_THERE	Adds a new code model to the code context if its type is not already present in the context. Specify the type of the code model using the CodeType parameter. In case a code model of the same type is already present, its Aurora Imaging Library identifier is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_DELETE	Deletes a code model within the code context, whose model identifier is specified by the ContextCodeId parameter. For an M_DELETE operation, the CodeType and Instance parameters must be passed **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_DELETE_ALL	Deletes all code models from the context. For this operation, the CodeType and Instance parameters must be passed M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIND	Finds a code model within the code context, whose code type is specified by the CodeType parameter. If there are many code models having the same specified code type, you must specify which instance of the specified type using the Instance parameter.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESET_FROM_DETECTED_RESULTS	Resets the code context and adds new code models to the context based on the results of an McodeDetect() operation. A code model is added for each code type that was detected by the McodeDetect() operation; if it detected two code occurrences with the same code type but different encoding schemes, only one code model of this type is added and its encoding scheme is set to M_ANY (if supported by the code type). All existing code models in the context are deleted. When resetting the code context, M_INITIALIZATION_MODE is set to M_IMPROVED_RECOGNITION. For all code models added, M_POSITION_ACCURACY is set to M_HIGH, and M_CODE_SEARCH_MODE is set to M_BEST. You must pass the code detect result buffer to the ControlFlag parameter.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_ADD.

You can add the following value to the above-mentioned value to determine whether a code type can be added to the code context, given the code type of the code models that already exist in the context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring whether a code type can be added to the code context																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SUPPORTED	Inquires whether the code type can be added to the code context, given the code type(s) that already exists in the context.	c		e	f	g	a		i		j	k	l	m	n	b

CodeType INQ

Specifies the code type of the code model to add or find in the code context. Note that if the Operation parameter is set to M_RESET_FROM_DETECTED_RESULTS or M_DELETE, set this parameter to **M_NULL**. Possible values for CodeType are listed below.

To find a code model of any type, the CodeType parameter must be set to the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For finding code models of any type																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ANY	Matches all code types. This value can only be used when the Operation parameter is set to M_FIND.	c		e	f	g	a		i		j	k	l	m	n	b

To specify the type of a 1D code model to add or find, the CodeType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a 1D code type

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_4_STATE	Specifies a 4-state code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_BC412	Specifies a BC412 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODABAR	Specifies a Codabar code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE39	Specifies a Code 39 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE93	Specifies a Code 93 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE128	Specifies a Code 128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_EAN8	Specifies an EAN 8 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_EAN13	Specifies an EAN 13 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_EAN14	Specifies an EAN 14 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_GS1_128	Specifies a GS1-128 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_GS1_DATABAR	Specifies a GS1 Databar code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_IATA25	Specifies an IATA 2 of 5 code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_INDUSTRIAL25	Specifies an Industrial 2 of 5 (standard 2 of 5) code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERLEAVED25	Specifies an Interleaved 2 of 5 (ITF-14) code type. M_INTERLEAVED25 is best used in a fixed-length application, with all reading equipment programmed to accept messages only of the correct length.	c		e	f	g	a		i		j	k	l	m	n	b
M_PHARMACODE	Specifies a Pharmacode code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANET	Specifies a Planet code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSTNET	Specifies a Postnet code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_UPC_A	Specifies a UPC-A code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_UPC_E	Specifies a UPC-E code type.	c		e	f	g	a		i		j	k	l	m	n	b

To specify the type of a 2D code model to add or find, the CodeType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a 2D code type																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AZTEC	Specifies an Aztec code type. This is a matrix code type.	c		e	f	g	a		i		j	k	l	m	n	b

M_DATAMATRIX	Specifies a Data Matrix code type. This is a matrix code type. Note that this code type includes the Extended Rectangular Data Matrix (DMRE) type.	c		e	f	g	a		i		j	k	l	m	n	b
M_DOTCODE	Specifies a DotCode code type. This is a matrix code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAXICODE	Specifies a Maxicode code type. This is a matrix code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_MICROPDF417	Specifies a MicroPDF417 code type. This is a cross-row code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_MICROQRCODE	Specifies a Micro QR code type. This is a matrix code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_PDF417	Specifies a PDF417 code type. This is a cross-row code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_QRCODE	Specifies a QR code type. This is a matrix code type.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUNCATED_PDF417	Specifies a Truncated PDF417 code type. This is a cross-row code type.	c		e	f	g	a		i		j	k	l	m	n	b

To specify that the code model to add or find is a composite code, the CodeType parameter must be set to the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a composite code type																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPOSITECODE	Specifies a composite code type. The code type is a composite of a 1D and a 2D code type; the particular combination of code types is determined by the specified encoding scheme.	c		e	f	g	a		i		j	k	l	m	n	b

Instance

Specifies the instance or the index of the code model, depending on the operation and whether a specific code type is specified. Note that if the Operation parameter is set to M_ADD or M_DELETE, set this parameter to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the instance of a code model																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALL	Specifies to add a code model for each code type in the code detect result buffer, when performing an M_RESET_FROM_DETECTED_RESULTS operation.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the instance or index of the individual code model on which to perform the operation when performing an M_FIND operation. When performing an M_FIND operation and a specific code type is specified using the CodeType parameter, the Instance parameter must indicate the instance, starting at zero, of that particular code type. If CodeType is set to M_ANY, the Instance parameter must specfy the index of the code model.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies additional information required to perform the operation. When not performing an M_RESET_FROM_DETECTED_RESULTS operation, set this parameter to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying additional information required to perform the operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies no additional information is required.	c		e	f	g	a		i		j	k	l	m	n	b

DetectResultCodeId	Specifies the identifier of the code detect result buffer to use to add models, when performing an M_RESET_FROM_DETECTED_RESULTS operation. The code detect result buffer must have been previously allocated using McodeAllocResult() with M_CODE_DETECT_RESULT and store the results of an McodeDetect() operation.	c		e	f	g	a		i		j	k	l	m	n	b

ModelCodeIdPtr

Specifies the address in which to return or from which to read the code model identifier, depending on operation. For M_ADD, M_ADD_IF_NOT_THERE, M_RESET_FROM_DETECTED_RESULTS, and M_FIND operations, you can set this parameter to M_NULL, since this function also returns the code model identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying/retrieving the code model identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies to ignore this parameter.	c		e	f	g	a		i		j	k	l	m	n	b
Address containing code model identifer to delete	Specifies the address containing the identifier of the code model to delete from the context, when performing an M_DELETE operation.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier of code model added	Specifies the address in which to write the identifier of the code model added, when performing an M_ADD, M_ADD_IF_NOT_THERE, or M_RESET_FROM_DETECTED_RESULTS operation. In the latter case, if multiple code models are added, only the identifier of the first code model is returned; to retrieve the identifier of the other code models, perform an M_FIND operation for each additional code model, specifying its index and an M_ANY code type.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier of code model found	Specifies address in which to write the identifier of the code model found, when performing an M_FIND operation.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the identifier of the (first) code model for a successful M_ADD, M_ADD_IF_NOT_THERE, M_RESET_FROM_DETECTED_RESULTS, or M_FIND operation; if the model could not be added/found, **M_NULL** is returned, unless performing an M_ADD_IF_NOT_THERE operation. In the latter case, if the model already exists, the Aurora Imaging Library identifier of the existing model will be returned. The returned value is **M_NULL** for an M_DELETE or an M_DELETE_ALL operation.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcode11.lib.
DLL	Requires ail11.dll; ailcode11.dll.

McodeRead

Synopsis

Read the code occurrence(s) in an image.

Syntax

```
void McodeRead(
    AIL_ID ContextCodeId, //in
    AIL_ID ImageBufId,    //in
    AIL_ID ResultCodeId   //out
)
```

Description

This function searches for code occurrences in an image, as specified in the code context, and writes the results in the specified result buffer. The code context's control settings determine how to perform the operation. Retrieve results using McodeGetResult().

Since the global context settings are required, **McodeRead**() requires that you pass a code context identifier, and not a code model identifier. The code context passed to **McodeRead**() can contain multiple code models depending on their code type (see McodeModel() for details). **McodeRead**() can read one or more occurrences of 1D and Data Matrix code models; for other types of code models, **McodeRead**() can read at most one occurrence.

It is strongly recommended to use a region of interest (ROI), set using MbufSetRegion(), with read operations, especially if your image contains more code occurrences than your context is configured to read; otherwise, Aurora Imaging Library will randomly select which code occurrences to read, up to the specified number, from among all the code occurrences in the image. Reading only a portion of your source image is also recommended because it achieves a fast and robust read operation if your image contains other information besides the code occurrences. Note that, if your target image has an ROI, you can have the nominal angle set to the same angle of the target image's ROI, using McodeControl() with M_SEARCH_ANGLE set to M_ACCORDING_TO_REGION.

If you have associated the target image with a camera calibration context, you can specify that certain input settings be interpreted in world units. To do so, use McodeControl() with M_SCANLINE_INPUT_UNITS, M_DOT_SPACING_INPUT_UNITS, M_FINDER_PATTERN_INPUT_UNITS, M_SEARCH_ANGLE_INPUT_UNITS, and/or M_CELL_SIZE_INPUT_UNITS set to M_WORLD. Note that if you set any of these constants to M_WORLD but you don't pass **McodeRead**() a calibrated target image, the function will generate an error.

Before performing a read operation, certain control types might have to be set globally for the entire code context or individually for each code model in McodeControl(). When reading multiple occurrences of a Data Matrix code model, setting one or more of these values and reducing the search speed (using McodeControl() with M_SPEED) will produce more robust results.

Control type	Notes
M_CELL_NUMBER_X, M_CELL_NUMBER_Y	For 2D code types only, improves the robustness of the operation.
M_CELL_SIZE_MIN, M_CELL_SIZE_MAX	To improve the robustness of the read operation for some code types, especially Data Matrix and Maxicode. Note that Aurora Imaging Library might have difficulty reading code occurrences if the cell size is less than 2 pixels, even if the size is specified.
M_ENCODING	For code types where M_ANY is not supported.
M_ERROR_CORRECTION	For code types where M_ANY is not supported.
M_FOREGROUND_VALUE	This control type is essential for all code types, and the image will not be decoded if the foreground value is not correctly set.

M_SEARCH_ANGLE, M_SEARCH_ANGLE_DELTA_NEG, M_SEARCH_ANGLE_DELTA_POS	If the code occurrences to read are not within 5° of the horizontal axis, and/or if the code occurrences have bearer bars.
M_STRING_SIZE_MIN and M_STRING_SIZE_MAX	For the M_BC412 code type, the string size must be specified.
M_SUB_TYPE	For M_COMPOSITECODE and M_GS1_DATABAR code type, specifying the code sub-type can help increase the speed of read operations.

Note that if the image contains Structured Append sequence of separate but logically linked ECC 200 code occurrences, Aurora Imaging Library will treat the code occurrences as separate, independent code occurrences.

Parameters

ContextCodeId

Specifies the identifier of the code context.

ImageBufId

Specifies the 8-bit unsigned 1-band image buffer in which to search.

This image buffer can only have a rectangular region of interest (ROI) defined using MbufSetRegion() (with or without rotation). In addition, the region must be defined in vector format from a 2D graphics list (M_VECTOR or M_VECTOR_AND_RASTER). An error will be generated if the ROI is only in raster format (M_RASTER) or a non-rectangular shape.

ResultCodeId

Specifies the result buffer in which to write the results of the read.

The result buffer must have been allocated using McodeAllocResult() with M_CODE_READ_RESULT.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcode11.lib.
DLL	Requires ail11.dll; ailcode11.dll.

McodeRestore

Synopsis

Restore a code context from disk.

Syntax

```
AIL_ID McodeRestore(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SysId,                //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID *ContextCodeIdPtr     //out
)
```

Description

This function restores a code context that has been saved to a file, using McodeSave() or McodeStream(). This function can also restore a code context from a predefined code context file distributed with Aurora Imaging Library (such as, *C:\Program Files\Aurora Imaging Library\<version number>\Contexts\SEMI_T1-95r303.mco*). For more information, refer to the Predefined SEMI code contexts subsection of the Supported code types section of Chapter 17: Codes.

This function restores all of the code context's controls and models that were in effect when the code context was saved.

When the code context is no longer required, release it using McodeFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the code context's lifetime and you must not manually free it.

Parameters

FileName

Specifies the path and name of the file from which to restore the code context (for example, "*C:\mydirectory\myfile*"). The function handles (internally) the opening and closing of the file.

The predefined code context files, distributed with Aurora Imaging Library, are in the *C:\Program Files\Aurora Imaging Library\<version number>\Contexts* folder.

To retrieve the file from the remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").

This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Open** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Filename.mco*") 1	Specifies to restore a user-defined code context.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*SEMI_T1-95r303.mco*") 1	Specifies to restore a predefined BC-412 1D code context that matches the "SEMI *SEMI T1-95 (reapproved 0303) specification for back surface bar code marking of silicon wafers* . USA: SEMI, 2003. 10.".	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("SEMI_T2-0298E.mco") 1	Specifies to restore a predefined Data Matrix 2D code context with an ECC_200 encoding scheme that matches the "SEMI *SEMI T2-0298 (reapproved 1104) specification for marking of wavers with a two-dimensional matrix code symbol* . USA: SEMI, 2004. 9.".	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("SEMI_T7-0303.mco") 1	Specifies to restore a predefined Data Matrix 2D code context that matches the "SEMI *SEMI T7-0303 specification for back surface marking of double-side polished wafers with a two-dimensional matrix code symbol* . USA: SEMI, 2003. 4.".	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the system on which to allocate the context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

ContextCodeIdPtr

Specifies the address of the variable in which to write the code context identifier or specifies the data type that the function should use to return the code context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated code context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated code context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_CODE_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the code context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored code context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the code context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_CODE_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcode11.lib.
DLL	Requires ail11.dll; ailcode11.dll.

McodeSave

Synopsis

Save the specified code context in a file.

Syntax

```
void McodeSave(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID ContextCodeId,        //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function saves a code context in a file so that it can be reloaded with McodeRestore() or McodeStream().

Parameters

FileName

Specifies the name and path of the file in which to save the code context. It is recommended that you use the MCO file extension for easier use with other Aurora Imaging software products. The function internally handles the opening and closing of this file. If this file already exists, it will be overwritten.

This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Save As** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, code context files have an MCO extension. To save the file on the remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ContextCodeId

Specifies the identifier of the data buffer to save.

ControlFlag

Specifies the function's control flag. Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcode11.lib.
DLL	Requires ail11.dll; ailcode11.dll.

McodeStream

Synopsis

Load, restore, or save a code context from/to a file or a memory stream, save a grading report to a file, or save a training report to a file.

Syntax

```
void McodeStream(
    AIL_TEXT_PTR MemPtrOrFileName,    //in-out
    AIL_ID SysId,                     //in
    AIL_INT64 Operation,              //in
    AIL_INT64 StreamType,             //in
    AIL_DOUBLE Version,               //in
    AIL_INT64 ControlFlag,            //in
    AIL_ID *ContextOrResultCodeIdPtr, //in-out
    AIL_INT *SizeByteVarPtr           //out
)
```

Description

This function can load, restore, or save a code context from/to a file or memory stream. This function can also save a grading report to a file or save a training report to a file.

To inquire the number of bytes necessary to save a code context to memory stream, you should first call this function (**McodeStream**()) with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. In addition, any file saved using this function is equivalent to a file saved with McodeSave().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file.

Using **McodeStream**(), you can choose to save a backwards-compatible version of the code context, which will work using a specified version of Aurora Imaging Library that is up to one major release older than the current version. However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore a code context saved using Aurora Imaging Library version 7.0 or above. Settings that do not exist in the lower version will be filled with default values when the code context is loaded or restored using the current version.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the system on which to restore the code context.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform on the code context.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the code context. This parameter must be set to one of the following values:

For specifying the type of stream																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream. Note that this is the only value available when performing an M_SAVE_REPORT operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the code context.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

ContextOrResultCodeIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the code context. In the case of an M_SAVE_REPORT operation, this parameter specifies the address of the variable from which to read the identifier of the code result buffer.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the code context, in bytes. If the size is not required, or if performing an M_SAVE_REPORT operation, set this parameter to **M_NULL**.

> Note that the size of a code context will vary depending on the Aurora Imaging Library version specified.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **ContextOrResultCodeIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.																
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
MemPtrOrFileName - Possible values passed or returned																
SysId																
Version																
ContextOrResultCodeIdPtr - Possible values passed or returned																
M_INQUIRE_SIZE_BYTE	Inquires the number of bytes required to save a code context to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.															
MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the code context must be compatible when saved. The size of a code context might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
ContextOrResultCodeIdPtr		Specifies the address of the variable containing the identifier of the code context.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD		Loads the content of a specified file or memory stream into a previously allocated code context.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName		Specifies the file or memory stream from which to load the context.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Code contexts typically have an MCO file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextOrResultCodeIdPtr		Specifies the address of the variable containing the identifier of the code context. All previous settings are overwritten with those of the loaded context, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the context is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE		Restores a code context from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName		Specifies the file or memory stream from which to restore the context.	c		e	f	g	a		i		j	k	l	m	n	b

M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Code contexts typically have an MCO file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	Specifies the system on which to allocate the code context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextOrResultCodeIdPtr	Specifies the address of the variable in which to write the identifier of the code context. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE	Saves a code context to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. For easier use with other Aurora Imaging software products, when saving a code context to a file, use the MCO file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b

SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the code context must be compatible when saved. Note that the code context can only be up to one major release older than the current version.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
ContextOrResultCodeIdPtr	Specifies the address of the variable containing the identifier of the code context to save.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE_REPORT	Saves a report containing most results from an McodeGrade() or McodeTrain() operation.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextOrResultCodeIdPtr	Specifies the address of the variable containing the identifier of the result buffer to save. The result buffer must have been allocated using McodeAllocResult() with M_CODE_GRADE_RESULT or M_CODE_TRAIN_RESULT, and it must contain the result of a successful McodeGrade() or McodeTrain() operation, respectively. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcode11.lib.

DLL	Requires ail11.dll; ailcode11.dll.

McodeTrain

Synopsis

Train the control type settings of a code context and its code models so that they are optimized for the specified set of sample images.

Syntax

```
void McodeTrain(
    AIL_ID ContextCodeId,        //in
    AIL_INT NumImages,           //in
    const AIL_ID *ImageArrayPtr, //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID TrainResultCodeId     //out
)
```

Description

This function trains the settings of the selected control types of a code context and its code models so that the settings are optimized for performing an McodeRead() or McodeGrade() operation on the specified set of sample images. The function saves the results in the specified code train result buffer; the specified code context is not directly modified.

Activate the required control types for training using McodeControl() with the combination constant M_TRAIN or with M_SET_TRAINING_STATE_ALL. The following control types are supported for training:

- M_CELL_NUMBER_....
- M_CELL_SIZE_....
- M_CODE_FLIP.
- M_DATAMATRIX_SHAPE.
- M_DECODE_ALGORITHM.
- M_DOT_SIZE_....
- M_DOT_SPACING_....
- M_ENCODING.
- M_ERROR_CORRECTION.
- M_FOREGROUND_VALUE.
- M_MINIMUM_CONTRAST.
- M_NUMBER.
- M_SEARCH_ANGLE....
- M_SPEED.
- M_THRESHOLD_....
- M_USE_PRESEARCH.

You can limit the code occurrences used for training to those within specific regions of the training images. If the training images have a rectangular region of interest (ROI), defined using MbufSetRegion(), **McodeTrain**() only uses occurrences within those regions to establish the recommended settings. The regions must be defined in vector format from a 2D graphics list (M_VECTOR or M_VECTOR_AND_RASTER). An error will be generated if any of the images have an ROI that is only in raster format (M_RASTER) or is a non-rectangular shape.

This function writes the recommended settings of the activated control types to the specified code train result buffer. You can reconfigure any code context with these results using McodeControl() with M_RESET_FROM_TRAINED_RESULTS. This control type discards any existing code models from the context, and then adds the code models used for training to the context. If a control type was not trained, it is set to its value used during training; if a control type was trained, it is set to its trained value. Prior to using M_RESET_FROM_TRAINED_RESULTS, you should ensure that the train operation was successful using McodeGetResult() with M_STATUS. If required, you can retrieve the recommended setting for a specific control type using McodeGetResult(). This is especially useful if you want to reconfigure the code context that was used during training. In this case, you can use the retrieved values to selectively change the control types of the context to their trained value. Note that some control types are inter-dependent; changing the setting of one control type to its trained value might not be optimal if the settings of other control types are not also changed. This is especially true for control types that are automatically activated

for training if another control type is activated (for example, M_CELL_NUMBER_...).

You can retrieve or draw the results of the read operation that **McodeTrain**() internally performed on each training image. You can use this extra information to complement the status result. First, call McodeGetResult() with M_CODE_RESULT_ID. This returns the identifiers of the internal code read result buffers; there is one per training image. You can then pass one of these result buffer identifiers to McodeGetResult(), and retrieve any type of result available for an McodeRead() operation. You can also pass one of the result buffer identifiers to McodeDraw() to visualize the results for a specific training image.

The recommended control type settings for a code context and its code models are based on the code occurrences found in all the training images. **McodeTrain**() must find at least one occurrence of one of the code models among all the sample images; otherwise, McodeGetResult() with M_STATUS returns M_STATUS_TRAIN_FAILED. You can retrieve the buffer identifiers of images in which no code occurrence was found using M_FAILED_IMAGES_ID.

Parameters

ContextCodeId

Specifies the identifier of the code context to train.

NumImages

Specifies the number of training images passed to ImageArrayPtr.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the ImageArrayPtr parameter.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of training images																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Value >= 1	Specifies the number of images passed to ImageArrayPtr.	c		e	f	g	a		i		j	k	l	m	n	b

ImageArrayPtr
Accepts the address of one of the following:

- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>] (and a maximum array rank for C# of 1)
  Required array size:(NumImages)

Specifies the address of the array containing the buffer identifiers of the training images. The image buffers must be 1-band, 8-bit unsigned buffers.

Any of the image buffers can have a rectangular region of interest (ROI) defined using MbufSetRegion() (with or without rotation). The region must be defined in vector format from a 2D graphics list (M_VECTOR or M_VECTOR_AND_RASTER). An error will be generated if the ROI is only in raster format (M_RASTER) or a non-rectangular shape.

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

TrainResultCodeId

Specifies the train result buffer identifier. The result buffer must have been allocated using McodeAllocResult() with M_CODE_TRAIN_RESULT.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcode11.lib.
DLL	Requires ail11.dll; ailcode11.dll.

McodeWrite

Synopsis

Encode an ASCII string into a symbol.

Syntax

```
void McodeWrite(
    AIL_ID ModelCodeId,           //in
    AIL_ID ImageBufId,            //out
    AIL_CONST_TEXT_PTR StringPtr, //in
    AIL_INT64 ControlFlag,        //in
    AIL_ID WriteResultCodeId      //out
)
```

Description

This function encodes a null-terminated ASCII string into a symbol (code) using the code type and encoding scheme, specified by the code model, and draws the code into a specified image.

For a given code model and string, you can inquire the minimum buffer size required to draw the code, using **McodeWrite**() with its ImageBufId parameter set to M_NULL, and then using McodeGetResult() with M_WRITE_SIZE_X and M_WRITE_SIZE_Y.

The control type settings of the specified code model determine how to perform the operation. Before performing a **McodeWrite**() operation, you must set certain controls using McodeControl() or verify that their default settings are appropriate, specifically:

Control	Notes
M_CELL_NUMBER_X, M_CELL_NUMBER_Y	Specifies the number of cells in the X and Y directions. Improves the robustness of the operation, for 2D code types only. If these settings are set to M_DEFAULT, then the number of cells will be selected automatically so as to minimize the size of the resulting code.
M_CELL_SIZE_MODE	Specifies how to establish the cell size.
M_CELL_SIZE_VALUE	Specifies the cell size to use when M_CELL_SIZE_MODE is set to M_USER_DEFINED.
M_DOT_SHAPE	Specifies the shape of the dots used to draw foreground cells. This control type can be set for M_AZTEC, M_DATAMATRIX, M_QRCODE, and M_MICROQRCODE code types.
M_DOT_SIZE	Specifies the size of the dot (or square) used to draw foreground cells. This control type can be set for M_AZTEC, M_DATAMATRIX, M_MAXICODE, M_QRCODE, and M_MICROQRCODE code types.
M_ENCODING	Specifies the encoding scheme. This control type must be set for code types where M_ANY is not supported as the encoding scheme.
M_ERROR_CORRECTION	Specifies the type of error correction. This control type must be set for code types where M_ANY is not a supported error correction type.
M_FOREGROUND_VALUE	Specifies the foreground color of the code. This control type is essential for all code types.

If you have associated the target image with a camera calibration context, you can specify that certain input settings be interpreted in world units. To do so, use McodeControl() with M_CELL_SIZE_INPUT_UNITS set to M_WORLD. Note that if you set this constant to M_WORLD but you don't pass **McodeWrite**() a calibrated target image, the function will generate an error.

Parameters

ModelCodeId

Specifies the identifier of the code model.

ImageBufId

Specifies the 8-bit unsigned 1-band image buffer in which to write the symbol. Set this parameter to one of the following.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the image buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Allows you to inquire about the minimum buffer size required. Using this value means that **McodeWrite**() will not write into an image buffer. After calling **McodeWrite**() with M_NULL, call McodeGetResult() to retrieve the minimum width (M_WRITE_SIZE_X) and height (M_WRITE_SIZE_Y) required for the image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies that **McodeWrite**() will write into the image buffer, and in which image buffer to write. This buffer must be 8-bit unsigned. In addition, it should be large enough to hold the symbol.	c		e	f	g	a		i		j	k	l	m	n	b

StringPtr

Specifies the address of the string to be encoded.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string of the code																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that a sample string will be encoded. This can be used if you need to test or use the functionality of **McodeWrite**() without needing to encode a specific string (for example, to draw the code type on a button in an interface). The sample string that is encoded depends on the control values set by McodeControl().	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*String*") [1]	Specifies the address of the string. You need to know the string specifications of the code to write. Note the following restrictions for certain code types: • If the code type is a Codabar code, the string must be numeric, but can contain the following ASCII characters: minus (-), dollar sign ($), colon (:), forward slash (/), period (.), and plus (+). In addition, the string must start and end with any of the following characters: a, b, c, or d. • If the code type is an Interleaved 2 of 5 code, the string must have an even number of characters. • If the code type is a Maxicode code, with M_ENCODING set to M_ENC_MODE2 or M_ENC_MODE3, the string must respect the structured carrier message format (the portion of the string that contains postal code, country code, and class of service information). • If the code type is a composite code, the string must be passed in the following format: 1D string|2D string, where | is used as the separator. • If the code type uses an M_ENC_MODE2 or M_ENC_MODE3 encoding scheme, the string is truncated so that only the string that can fit in a single code is written. Aurora Imaging Library does not support a Structured Append sequence of separate but logically linked ECC 200 codes. • If the code type is a composite code, the FNC1 separator is represented by the unprintable character with ASCII value 29. In addition, add parentheses around the application identifiers in your string. Note that this is required by the **McodeWrite**() operation; the parentheses are not actually coded.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ControlFlag

Specifies the function's control flag. This parameter can be set to one or more of the following value: (for example, M_ESCAPE_SEQUENCE

+ M_GS1_FORMAT).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the function's control flag																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the string to encode does not use ASCII character codes for unprintable characters. Unprintable characters are not encoded. In addition, the string is not written in GS1 format. This value cannot be combined with any other value.	c		e	f	g	a		i		j	k	l	m	n	b
M_ESCAPE_SEQUENCE	Specifies that the string to encode uses ASCII character codes for unprintable characters. You must use a code type that supports unprintable characters specified using ASCII codes. When typing the string to encode, the unprintable character's ASCII character code must be in hexadecimal format, preceded by \x (for example, \x0D for ASCII 13, which is the carriage return). Note that if you want to encode the "\" character in escape sequence mode, type "\\".	c		e	f	g	a		i		j	k	l	m	n	b
M_GS1_FORMAT	Specifies that the string should be written as a GS1 code. This value is only available when dealing with M_AZTEC, M_DATAMATRIX, M_CODE128, M_EAN14, M_GS1_128, and M_QRCODE code types.	c		e	f	g	a		i		j	k	l	m	n	b

WriteResultCodeId

Specifies the identifier of the code write result buffer. The result buffer must have been allocated using McodeAllocResult() with M_CODE_WRITE_RESULT.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcode11.lib.
DLL	Requires ail11.dll; ailcode11.dll.